

Chapter 3 Analysis

3.1. Determining System Requirements:

- Introduction;
- Performing Requirements Determination;
- Traditional Methods for Determining Requirements;
- Contemporary Methods for Determining System Requirements;
- Radical Methods for Determining System Requirements

3.2. Structuring System Process Requirements:

- Introduction;
- Process Modeling;
- Data Flow Diagrams;
- Modeling Logic with Decision Tables, Decision Trees, and Pseudocodes

3.3. Structuring System Data Requirements:

- Introduction;
- Conceptual Data Modeling;
- Gathering Information for Conceptual Data Modeling;
- Introduction to E-R Modeling

Introduction

- Analysis is the first systems development life cycle (SDLC) phase where you begin to understand, in depth, the need for system changes.
- Systems analysis involves a substantial amount of effort and cost, and is therefore undertaken only after management has decided that the systems development project under consideration has merit and should be pursued through this phase.
- Most observers would agree that many of the errors in developed systems are directly traceable to inadequate efforts in the analysis and design phases of the life cycle.
- The **purpose of analysis is to determine what information and information processing services** are needed to support selected objectives and functions of the organization.
- **Gathering this information is called requirements determination.**

Introduction

- The results of the requirements determination can be **structured according to three essential views** of the current and replacement information systems:
 - **Process**: The sequence of data movement and handling operations within the system. (DFD)
 - **Logic and timing**: The rules by which data are transformed and manipulated and an indication of what triggers data transformation. ([Decision Tree](#), [Decision table](#), [Structured English](#))
 - **Data**: The inherent structure of data independent of how or when they are processed. (ERD)

Determining System Requirements

- **Introduction:**

- Systems analysis is the part of the systems development life cycle in which you determine how the current information system functions and assess what users would like to see in a new system.
- Analysis has **two** sub phases:
 - **requirements determination and**
 - **requirements structuring.**

- **Requirements: Necessity not Luxury**

- A defective requirement is 20-50 times more expensive to repair at end of a project than at beginning (\$100 -> \$2000-5000)
- Requirements analysis is something we must afford to do
- So why do we not do it?
- So why do we not do it well?
Instead ... code and test, code and test

Project Stakeholders Perspectives on Requirement

Stakeholder

- Owner
- Users
- Designers
- Builders

Perspective

Scope (purpose)

Requirements (what)

Arch./Design (how)

Implementation /Testing (results)

What the Users wanted



What the Owners
are willing to pay



What the Designers and
Builders developed



Why must requirements be written down?

- Define the goals of the project
- Sets customer expectations
- Basis of the contract between customer and supplier
- Writing is thinking
- Allows thorough inspection and testing
- Can be kept up to date
- Allows tracking - estimates/actuals
- Team communication tool

Each requirement must be ...

- Expressed properly,
- Validated
- Made easily accessible,
- Numbered,
- Accompanied by tests that verify it,
- Provided for in the design,
- Accounted for by code,
- Tested in isolation,
- Tested in concert with other requirements
- Validated by the user

Performing Requirements Determination

- Gather information on **what** the system should do from many sources
 - Users
 - Reports
 - Forms
 - Procedures
- **Characteristics for gathering requirements**
 - Impertinence
 - Question everything
 - Impartiality
 - Find the best organizational solution
 - Relaxation of constraints
 - Assume anything is possible
 - Attention to detail
 - Every fact must fit in with every other fact
 - Reframing
 - View the organization in new ways

Methods of requirement Determination

- **Three methods:**

- **Traditional methods:** Interviews, Questionnaire, Observation of workers, Document Analysis
- **Modern methods(contemporary):** Joint Application Development(JAD),Facilitated Application Specification Technique(FAST),Prototyping, Brainstorming, Use Case approach
- **Radical Methods:** Business Process Reengineering(BPR),Agile techniques

TABLE 5-1: Deliverables for Requirements Determination

Types of Deliverables	Specific Deliverables
Information collected from conversations with users	Interview transcripts Notes from observations Meeting notes
Existing documents and files	Business mission and strategy statement Sample business forms and reports and computer displays Procedure manuals Job descriptions Training manuals Flowcharts and documentation of existing systems Consultant reports
Computer-based information	Results from Joint Application Design sessions CASE repository contents and reports of existing systems Displays and reports from system prototypes

TABLE 5-2: Traditional Methods of Collecting System Requirements

Traditional Method	Activities Involved
Interviews with individuals	Interview individuals informed about the operation and issues of the current system and needs for systems in future organizational activities.
Observations of workers	Observe workers at selected times to see how data are handled and what information people need to do their jobs.
Business documents	Study business documents to discover reported issues, policies, rules, and directions as well as concrete examples of the use of data and information in the organization.

Traditional methods of Requirement Determination

• Interviews and listening:

- Discover issues with the current system and needs for the future
- requirements engineering teams put the questions to the stakeholder about the system that's currently used, and the system to be developed, and hence they can gather the requirements from the answers.
- Gather facts, opinions and speculations
- Observe body language and emotions
- Interview can be of one to one (direct) and indirect
- Questions can be of
 - Open ended: generic and unstructured questions
 - Close ended: predefined and structured questions
- Guidelines for making interview
 - Plan
 - Checklist
 - Appointment
 - Be neutral
 - Listen
 - Seek a diverse view

Basic Rules of Interview:

- The overall purpose of performing the interviews should be clear.
- Identify the interviewees in advance.
- Interview goals should be communicated to the interviewee.
- Interview questions should be prepared before the interview.
- The location of the interview should be predefined.
- The time limit should be described.
- The interviewer should organize the information and confirm the results with the interviewees as soon as possible after the interview.

Benifits

- Interactive discussion with stakeholders.
- The immediate follow-up to ensure the interviewer's understanding.
- Encourage participation and build relationships by establishing rapport with the stakeholder.

Drawbacks

- Time is required to plan and conduct interviews.
- Commitment is required from all the participants.
- Sometimes training is required to conduct effective interviews.

Traditional Methods for Determining Requirements

TABLE 5-3: Guidelines for Effective Interviewing

Guidelines	What Is Involved
Plan the interview	Prepare interviewee by making an appointment and explaining the purpose of the interview. Prepare a checklist, an agenda, and questions.
Be neutral	Avoid asking leading questions.
Listen and take notes	Give your undivided attention to the interviewee and take notes or tape-record the interview (if permission is granted).
Review notes	Review your notes within 48 hours of the meeting. If you discover follow-up questions or need additional information, contact the interviewee.
Seek diverse views	Interview a wide range of people, including potential users and managers.

Interview Outline	
Interviewee: <i>Name of person being interviewed</i>	Interviewer: <i>Name of person leading interview</i>
Location/Medium: <i>Office, conference room, or phone number</i>	Appointment Date: Start Time: End Time:
Objectives: <i>What data to collect</i> <i>On what to gain agreement</i> <i>What areas to explore</i>	Reminders: <i>Background/experience of interviewee</i> <i>Known opinions of interviewee</i>
Agenda: Introduction Background on Project Overview of Interview Topics to Be Covered Permission to Tape Record Topic 1 Questions Topic 2 Questions ... Summary of Major Points Questions from Interviewee Closing	Approximate Time: 1 minute 2 minutes 1 minute 5 minutes 7 minutes ... 2 minutes 5 minutes 1 minute
General Observations: Interviewee seemed busy—probably need to call in a few days for follow-up questions because he gave only short answers. PC was turned off—probably not a regular PC user.	
Unresolved Issues, Topics Not Covered: He needs to look up sales figures from 1998. He raised the issue of how to handle returned goods, but we did not have time to discuss.	

FIGURE 5-2
A Typical Interview Guide

(continues on next page)

Interviewee:		Date:	
Questions:		Notes:	
<i>When to ask question, if conditional</i> <i>Question: 1</i> Have you used the current sales tracking system? If so, how often? <i>If yes, go to Question 2</i>		<i>Answer</i> Yes, I ask for a report on my product line weekly. <i>Observations</i> Seemed anxious—may be overestimating usage frequency	
<i>Question: 2</i> What do you like least about this system?		<i>Answer</i> Sales are shown in units, not dollars. <i>Observations</i> System can show sales in dollars, but user does not know this.	

FIGURE 5-2
(continued)

Traditional Methods for Determining Requirements

- **Questionnaire/Survey**

- For Survey/Questionnaire, a set of questions is given to stakeholders to quantify their thoughts. After collecting the responses from stakeholders, data is analyzed to identify the area of interest of stakeholders.
- Questions should be based on high priority risks. Questions should be direct and unambiguous. Once the survey is ready, notify the participants and remind them to participate.
- Choosing respondents
 - Should be representative of all users
 - Types of samples
 - Convenient
 - Random sample
 - Purposeful sample
 - Stratified sample
- **Two types of questions can be used here:**
 - **Open-Ended:** Respondent is given the freedom to provide answers in their own words rather than selecting from predefined responses. This is useful but at the same time, this is time-consuming as interpreting the responses is difficult.
 - **Close Ended:** It includes a predefined set of answers for all the questions and the respondent has to choose from those answers. Questions can be multiple choice or can be ranked from not important to very important.
- **Benefits:**
 - More cost-effective than interviews
 - Easy to get data from a large audience.
 - Less time is required for the participants to respond.
 - You can get more accurate information as compared to interviews.
- **Drawback:**
 - All the Stakeholders might not participate in the surveys.
 - Questions may not be clear to all the participants.
 - Open-ended questions require more analysis.
 - Follow up surveys might be required based on the responses provided by participants.

Traditional Methods for Determining Requirements

- **Directly Observing Users(Observation)**

- Serves as a good method to supplement interviews
- The main objective of the observation session is to understand the activity, task, tools used, and events performed by others.
- The plan for observation ensures that all stakeholders are aware of the purpose of the observation session, they agree on the expected outcomes, and that the session meets their expectations. You need to inform the participants that their performance is not judged.
- During the session, the observer should record all the activities and the time taken to perform the work by others so that he/she can simulate the same. After the session, the BA will review the results and will follow up with the participants.
- Observation can be either active or passive.
 - **Active observation** is to ask questions and try to attempt the work that other persons are doing.
 - **Passive observation** is silent observation i.e. you sit with others and just observe how they are doing their work without interpreting them.
- **Benefits:**
 - The observer will get a practical insight into the work.
 - Improvement areas can be easily identified.
 - It produces more accurate and reliable data.
- **Drawbacks:**
 - Participants might get disturbed.
 - Often difficult to obtain unbiased data
 - People often work differently when being observed
 - Participants might change their way of working during observation and the observer might not get a clear picture.
 - Knowledge-based activities cannot be observed.

Traditional Methods for Determining Requirements

- **Analyzing Procedures and Other Documents(Document Observation)**

- Document analysis includes reviewing the business plans, technical documents, problem reports, existing requirement documents, etc.
- This is useful when the plan is to update an existing system. This technique is useful for migration projects.
- This technique is important in identifying the gaps in the system i.e. to compare the AS-IS process with the TO-BE process.
- This analysis also helps when the person who has prepared the existing documentation is no longer present in the system.
- Types of information to be discovered by this method are:
 - Problems with existing system
 - Opportunity to meet new need
 - Organizational direction
 - Names of key individuals
 - Values of organization
 - Special information processing circumstances
 - Rules for processing data
- **Four types of useful documents to SA:**
 - **Written work procedures**
 - For an individual or work group.
 - Describes how a particular job or task is performed.
 - Includes data and information used and created in the process
 - **Business form**
 - Explicitly indicate data flow in or out of a system
 - **Report generated by current systems**
 - Enables the analyst to work backwards from the report to the data that generated it
 - **Description of current information system, how they were designed and how they work**

Other two types of documents

- **Quantitative Document**
 - Almost of data are number
 - Such as financial or accounting statement
- **Qualitative Document**
 - Almost of data are text Almost of data are text
 - Such as business's policy, Business vision

Benefits:

- Existing documents can be used to compare current and future processes.
- Existing documents can be used as a base for future analysis.

Drawbacks:

- Existing documents might not be updated.
- Existing documents might be completely outdated.
- Resources worked on the existing documents might not be available to provide information.
- This process is time-consuming.

TABLE 5-4: Comparison of Observation and Document Analysis

Characteristic	Observation	Document Analysis
Information richness	High (many channels)	Low (passive) and old
Time required	Can be extensive	Low to moderate
Expense	Can be high	Low to moderate
Chance for follow-up and probing	Good: Opportunity for probing and clarification questions during or after observation	Limited: Probing possible only if original author is available
Confidentiality	Observee is known to interviewer; observee may change behavior when observed	Depends on nature of document; does not change simply by being read
Involvement of subject	Interviewees' involvement committed dependent on whether they know they are being observed	None, no clear commitment
Potential audience	Limited numbers and limited time (snapshot) of each	Potentially biased by which documents were kept or because document not created for this purpose

Modern Methods for Determining Requirements

• **Joint Application Development(JAD)**

- This technique is more process-oriented and formal as compared to other techniques. These are structured meetings involving end-users, PMs, SMEs. This is used to define, clarify, and complete requirements.
 - Brings together key users, managers and systems analysts
 - Purpose: collect system requirements simultaneously from key people
 - Conducted off-site
- **Participants**
 - **Session Leader:** facilitates group process.
 - **Users:** active, speaking participants
 - **Managers:** active, speaking participants
 - **Sponsor:** high-level champion, limited participation.
 - **Systems Analysts:** should mostly listen.
 - **Scribe:** record session activities.
 - **IS Staff:** should mostly listen.
- **End Result**
 - Documentation detailing existing system
 - Features of proposed system
- **This technique can be divided into the following categories:**
 - **Formal Workshops:** These workshops are highly structured and are usually conducted with the selected group of stakeholders. The main focus of this workshop is to define, create, refine, and reach closure on business requirements.
 - **Business Process Improvement Workshops:** These are less formal as compared to the above one. Here, existing business processes are analyzed and process improvements are identified.
- **Benefits:**
 - Documentation is completed within hours and is provided quickly back to participants for review.
 - You can get on the spot confirmation on requirements.
 - Successfully gathered requirements from a large group in a short period.
 - Consensus can be achieved as issues and questions are asked in the presence of all the stakeholders.
- **Drawbacks:**
 - Stakeholder's availability might ruin the session.
 - The success rate depends on the expertise of the facilitator.
 - A workshop motive cannot be achieved if there are too many participants.

JAD Layout

CASE Tools During JAD

- Upper CASE tools are used
- Enables analysts to enter system models directly into CASE during the JAD session
- Screen designs and prototyping can be done during JAD and shown to users

Supporting JAD with GSS

- Group support systems (GSS) can be used to enable more participation by group members in JAD
- Facilitate sharing of ideas and voicing of opinions about system requirements.
- Members **type** their answers into the computer
- All members of the group see what other members have been typing

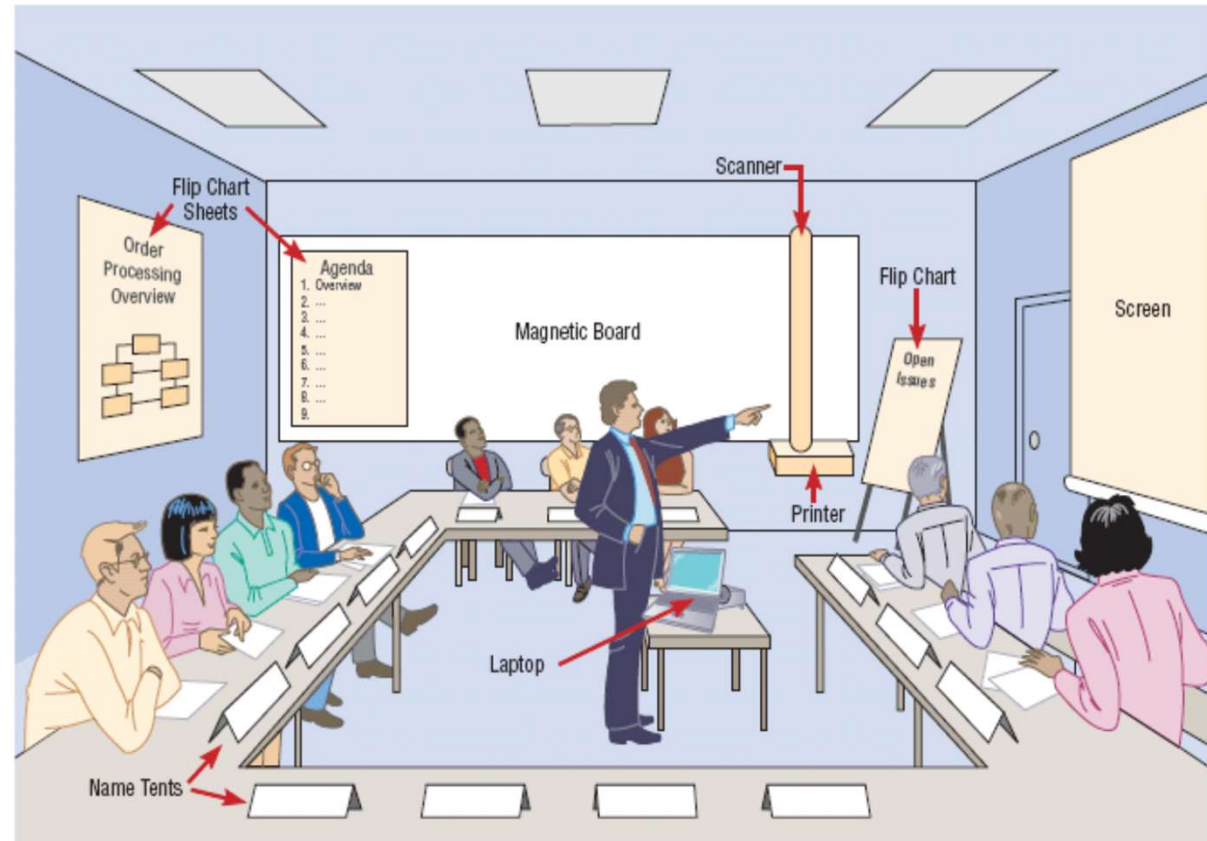


FIGURE 5-6
A Typical Room Layout for a JAD Session

Source: Adapted from Wood and Silver, 1989.

Modern Methods for Determining Requirements

- **Prototyping**

- Prototyping is used to identify missing or unspecified requirements.
- In this technique, frequent demos are given to the client by creating the prototypes so that client can get an idea of how the product will look like.
- Prototypes can be used to create a mock-up of sites, and describe the process using diagrams.
- Quickly converts requirements to working version of system.
- Once the user sees requirements converted to system, will ask for modifications or will generate additional requests.
- Prototyping is:
 - Repetitive process
 - Rudimentary version of system is built
 - Replaces or augments SDLC
 - Goal: to develop concrete specifications for ultimate system
- Quickly converts requirements to working version of system
- Once the user sees requirements converted to system, will ask for modifications or will generate additional requests
- Most useful when:
 - User requests are not clear
 - Few users are involved in the system
 - Designs are complex and require concrete form
 - History of communication problems between analysts and users
 - Tools are readily available to build prototype
- **Benefits:**
 - Gives a visual representation of the product.
 - Stakeholders can provide feedback early.
- **Drawbacks:**
 - Tendency to avoid formal documentation
 - Difficult to adapt to more general user audience
 - Sharing data with other systems is often not considered
 - Systems Development Life Cycle (SDLC) checks are often bypassed

Modern Methods for Determining Requirements

• Prototyping

• Prototyping can be performed using two approaches:

- Throwing Prototyping:

- The prototype can serve only as a model, which is then used as a reference for the construction of the actual system. In this process, called throwing prototyping, the prototype is discarded after it has been used.

- Evolutionary Prototyping

- As the prototype changes through each iteration, more and more of the design specifications for the system are captured in the prototype. The prototype can then serve as the basis for the production system in a process called evolutionary prototyping.

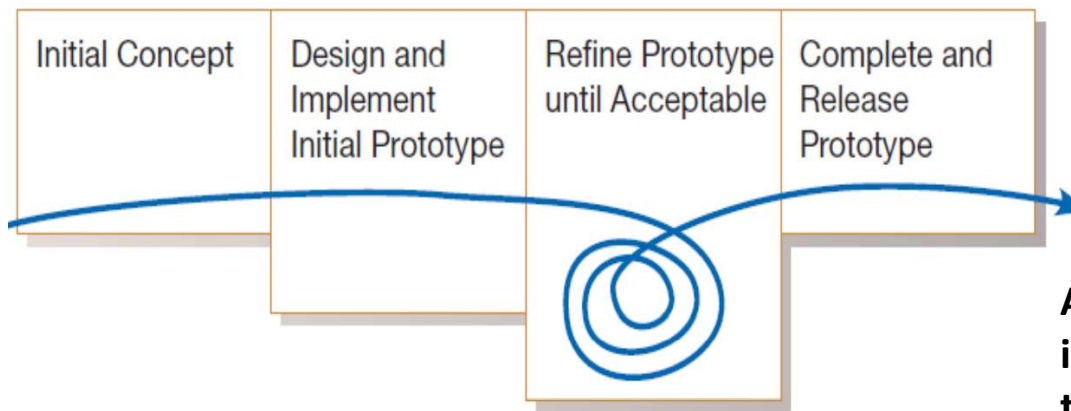


Fig: Evolutionary Prototyping Model

A life-cycle model of evolutionary prototyping illustrates the iterative nature of the process and the tendency to refine the prototype until it is ready to release

Modern Methods for Determining Requirements

- **Brainstorming**

- This technique is used to generate new ideas and find a solution for a specific issue.
- The members included for brainstorming can be domain experts, subject matter experts.
- Multiple ideas and information give you a repository of knowledge and you can choose from different ideas.
- This session is generally conducted around the table discussion. All participants should be given an equal amount of time to express their ideas.
- **Brainstorming technique is used to answer the below questions:**
 - What is the expectation of a system?
 - What are the risk factors that affect the proposed system development and what to do to avoid that?
 - What are the business and organization rules required to follow?
 - What are the options available to resolve the current issues?
 - What should we do so that this particular issue does not happen in the future?
- **There are some basic rules for this technique which should be followed to make it a success:**
 - The time limit for the session should be predefined.
 - Identify the participants in advance. One should include 6-8 members for the session.
 - The agenda should be clear enough for all the participants.
 - Clear expectations should be set with the participants.
 - Once you get all the information, combine the ideas, and remove the duplicate ideas.
 - Once the final list is ready, distribute it among other parties.
- **Benefits:**
 - Creative thinking is the result of the brainstorming session.
 - Plenty of ideas in a short time.
 - Promotes equal participation.
- **Drawbacks:**
 - Participants can be involved in debating ideas.
 - There can be multiple duplicate ideas.

Modern Methods for Determining Requirements

- **Facilitated Application Specification Technique (FAST)**

- It's objective is to bridge the expectation gap – difference between what the developers think they are supposed to build and what customers think they are going to get.
- A team oriented approach is developed for requirements gathering. Each attendee is asked to make a list of objects that are-
 - Part of the environment that surrounds the system
 - Produced by the system
 - Used by the system
- Each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, team is divided into smaller sub-teams to develop mini-specifications and finally a draft of specifications is written down using all the inputs from the meeting.

- **Nominal Group Technique (NGT)**

- A facilitated process that supports idea generation by groups.
- Process
 - Members come together as a group, but initially work separately.
 - Each person writes ideas.
 - Facilitator reads ideas out loud, and they are written on a blackboard or flipchart.
 - Group openly discusses the ideas for clarification.
 - Ideas are prioritized, combined, selected, reduced.

Modern Methods for Determining Requirements

- **Use Case Approach**

- This technique combines text and pictures to provide a better understanding of the requirements.
- The use cases describe the 'what', of a system and not 'how'. Hence they only give a functional view of the system.
- The components of the use case design includes three major things – Actor, Use cases, use case diagram.
 - **Actor** – It is the external agent that lies outside the system but interacts with it in some way. An actor maybe a person, machine etc. It is represented as a stick figure. Actors can be primary actors or secondary actors.
 - Primary actors – It requires assistance from the system to achieve a goal.
 - Secondary actor – It is an actor from which the system needs assistance.
 - **Use cases** – They describe the sequence of interactions between actors and the system. They capture who(actors) do what(interaction) with the system. A complete set of use cases specifies all possible ways to use the system.
 - **Use case diagram** – A use case diagram graphically represents what happens when an actor interacts with a system. It captures the functional aspect of the system.
 - A stick figure is used to represent an actor.
 - An oval is used to represent a use case.
 - A line is used to represent a relationship between an actor and a use case.

Radical Methods for Determining Requirements

- **Business Process Reengineering**

- In some organizations, management is looking for new ways to perform current tasks.
- Business Process Reengineering (BPR) is the search for, and implementation of, radical change in business processes to achieve breakthrough improvements in products and services.
- As part of the BPR effort, key business processes should be identified to Reengineer.
- Concerned with **re-designing business processes** to make them more responsive and more efficient
- Often reliant on the introduction of new computer systems to support the revised processes
- Three Analysis strategies:
 - Business Process Automation
 - Business Process Improvement
 - Business Process Re-engineering

- **BPR Goals**

- Reorganize complete flow of data in major sections of an organization
- Eliminate unnecessary steps
- Combine steps
- Become more responsive to future change

- **BPR Principles**

- Organize around outcomes, not tasks.
- Have those who use the output of the process perform the process.
- Incorporate information processing work into the real work that produces the raw information.
- Treat geographically dispersed resources as though they were centralized.
- Link parallel activities instead of integrated their results when different
- Put the decision point where the work is performed, and build control into the process.
- Capture data once, at its source.

Business Process Reengineering

- **Identification of processes to Reengineer**

- **Key business processes**

- Set of activities designed to produce specific output for a particular customer or market
 - **Focused on customers and outcome** :Key business processes are the structured, measured set of activities designed to produce a specific output for a particular customer or market.
 - **Key business process includes all activities of design, build, deliver and support a product.**
 - Benefits of BPR include radical improvements in speed, quality, and customer satisfaction.

- To identify activities that can be improved through BPR ask:

- How important is the activity to delivering an outcome?
 - How feasible is changing the activity?
 - How dysfunctional is the activity?

- Identify specific activities that can be **improved** through BPR, once it have been identified, Information Technology must be applied to **radically improve** business process.

- **Disruptive technologies**

- are technologies that enable the breaking of long-held business rules that inhibit organizations from making radical business changes.
 - Tools and technology includes: decision **support tools, wireless data communication, high performance computing can provide real-time updating**

TABLE 6-6 Long-Held Organizational Rules That Are Being Eliminated through Disruptive Technologies

Rule	Disruptive Technology
Information can appear in only one place at a time.	Distributed databases allow the sharing of information.
Businesses must choose between centralization and decentralization.	Advanced telecommunications networks can support dynamic organizational structures.
Managers must make all decisions.	Decision-support tools can aid nonmanagers.
Field personnel need offices where they can receive, store, retrieve, and transmit information.	Wireless data communication and portable computers provide a "virtual" office for workers.
The best contact with a potential buyer is personal contact.	Interactive communication technologies allow complex messaging capabilities.
You have to find out where things are.	Automatic identification and tracking technology knows where things are.
Plans get revised periodically.	High-performance computing can provide real-time updating.

BPR comprises:

- **Business definition.** Business goals are identified within the context of four key drivers: cost reduction, time reduction, quality improvement, and personnel development and empowerment.
- **Process identification.** Processes that are critical to achieving the goals defined in the business definition are identified.
- **Process evaluation.** The existing process is thoroughly analyzed and measured.

- **Process specification and design.** Based on information obtained during the first three BPR activities, use-cases are prepared for each process that is to be redesigned.
- **Prototyping.** A redesigned business process must be prototyped before it is fully integrated into the business.
- **Refinement and instantiation.** Based on feedback from the prototype, the business process is refined and then instantiated within a business system.

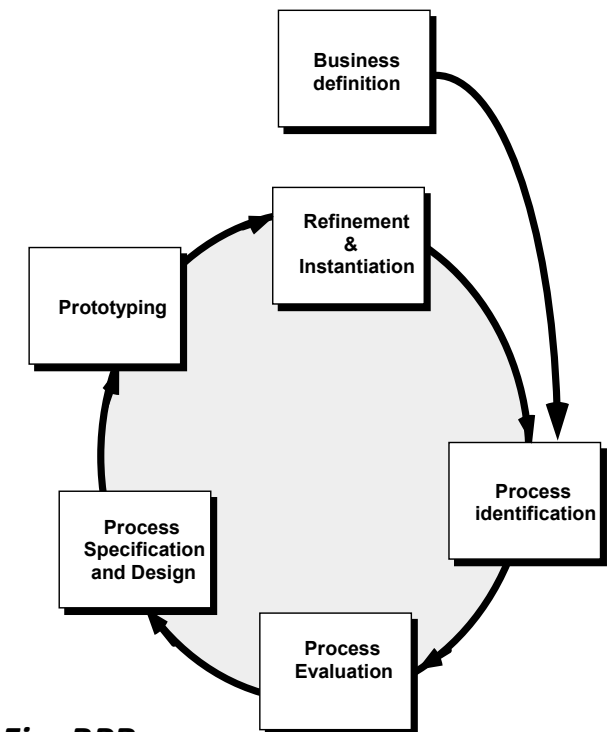


Fig: BPR

Requirements determining using Agile Methodologies

- **Agile Focus on:**

- **Continual user involvement**

- Replace traditional SDLC waterfall with iterative analyze – design – code – test cycle

- **Agile usage-centered design**

- Focuses on user goals, roles, and tasks
 - Gather a group of people all stakeholders in one room.
 - Give everyone a chance to talk about current and new system.
 - Determine user roles and goals
 - Determine task needs to be completed to achieve the goal.
 - Task cards will be grouped together based on similarity.
 - For each task, list steps that are necessary to complete the step.
 - Treat each set of tasks to be supported by a single aspect of user interface (partition task)
 - Prototype and refine the prototype

- **The Planning Game**

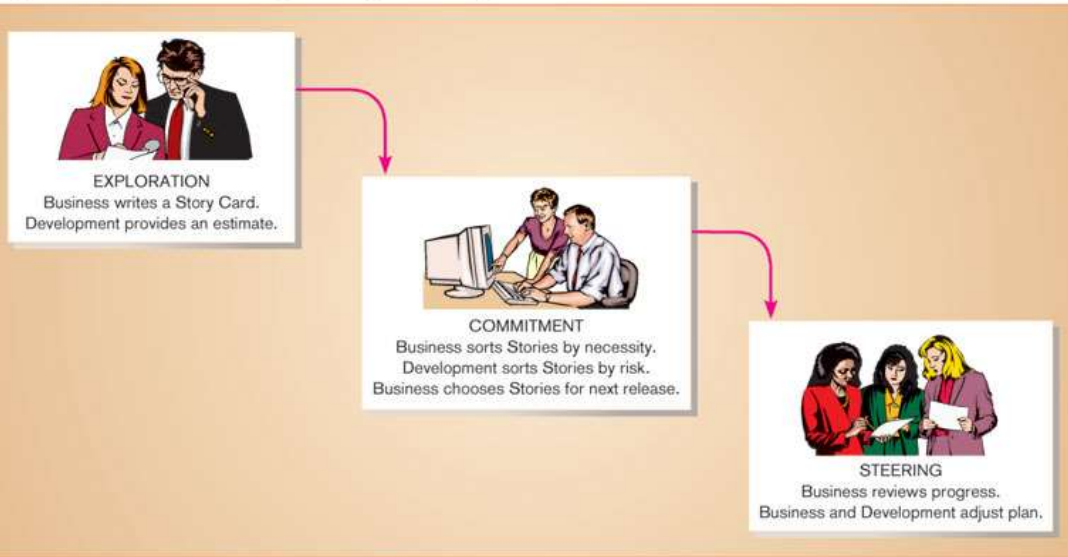
- Based on eXtreme programming
 - Exploration, steering, commitment

- **Agile Usage-Centered Design Steps**

- Gather group of programmers, analysts, users, testers, facilitator.
 - Document complaints of current system.
 - Determine important user roles.
 - Determine, prioritize, and describe tasks for each user role.
 - Group similar tasks into interaction contexts.
 - Associate each interaction context with a user interface for the system, and prototype the interaction context.
 - Step through and modify the prototype.

Requirements determining using Agile Methodologies

Figure 6-8 eXtreme Programming's Planning Game

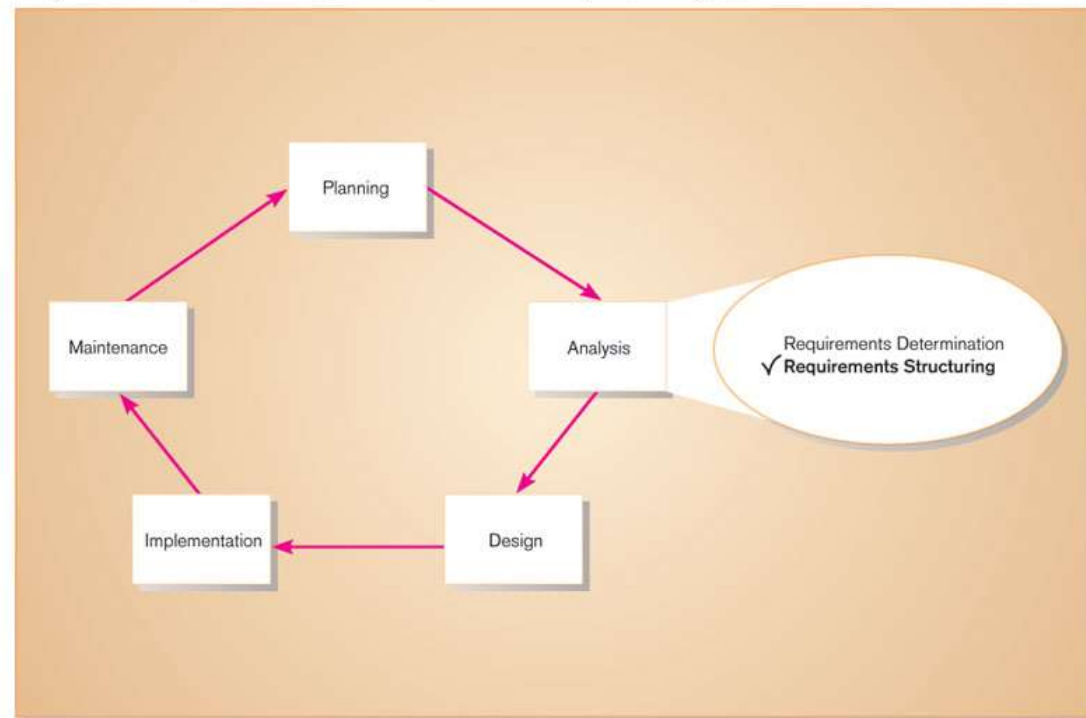


- The planning game from extreme programming: (phases)
 - **Exploration**, business create story cards, development with an estimation of how long it would take to implement.
 - **Commitment**, sort the story cards and split them to essential, not essential, and nice to have.
 - **Steering**, to see how the development process is progressing.
- The planning game is followed by iteration planning game, played by programmers.

• 3.2. Structuring System Process Requirements:

- Introduction;
- Process Modeling;
- Data Flow Diagrams;
- Modeling Logic with Decision Tables, Decision Trees, and Pseudocodes

Figure 7-1 Systems development life cycle with the analysis phase highlighted



Structuring System Process Requirements

- **Introduction**

- **Structured Analysis**

- Structured Analysis is a development method that allows the analyst to understand the system and its activities in a logical way.
- It is a systematic approach, which uses graphical tools that analyze and refine the objectives of an existing system and develop a new system specification which can be easily understandable by user.
- It has following attributes –
 - It is graphic which specifies the presentation of application.
 - It divides the processes so that it gives a clear picture of system flow.
 - It is logical rather than physical i.e., the elements of system do not depend on vendor or hardware.
 - It is an approach that works from high-level overviews to lower-level details.
- During Structured Analysis, various tools and techniques are used for system development. They are –
 - Data Flow Diagrams
 - Data Dictionary
 - Decision Trees
 - Decision Tables
 - Structured English
 - Pseudocode

- **Process Modeling:**

- Process modeling is the graphical representation of business processes or workflows.
- Graphically represent the processes that capture, manipulate, store, and distribute data between a system and its environment and among system components
- Utilize information gathered during requirements determination
- Processes and data structures are modeled
- A common form of process model is Data Flow Diagram(DFD).
- Like a flow chat, individual steps of the process are drawn out so there is an end-to-end overview of the tasks in the process within the context of the business environment.
- Modeling allows visualization of business processes so organizations can better understand their internal business procedures so that they can be managed and made more efficient.
- Process modeling is a vital component of process automation, as a process model needs to be created first to define tasks and optimize the workflow before it is automated.

- **Benefits of Process Modeling**

- **Improve efficiency** – process modeling helps to improve the process, helping business workers to be more productive by saving time
- **Gain transparency** – modeling provides a clear overview of the process, identifying the start and end point and all the steps in between
- **Ensure best practice** – using process models ensures consistency and standardization across the organization
- **Create understanding** – by using the common language of process, it makes it easier for users across the organization to communicate with each other
- **Business orchestration** – supports the coordination of people, systems and information across the organization to support business strategy

Deliverables and Outcomes of Process Modeling

- Context data flow diagram (DFD)
 - Scope of system
- DFDs of current physical and logical system
 - Enables analysts to understand current system
- DFDs of new logical system
 - Technology independent
 - Show data flows, structure, and functional requirements of new system
- Thorough description of each DFD component

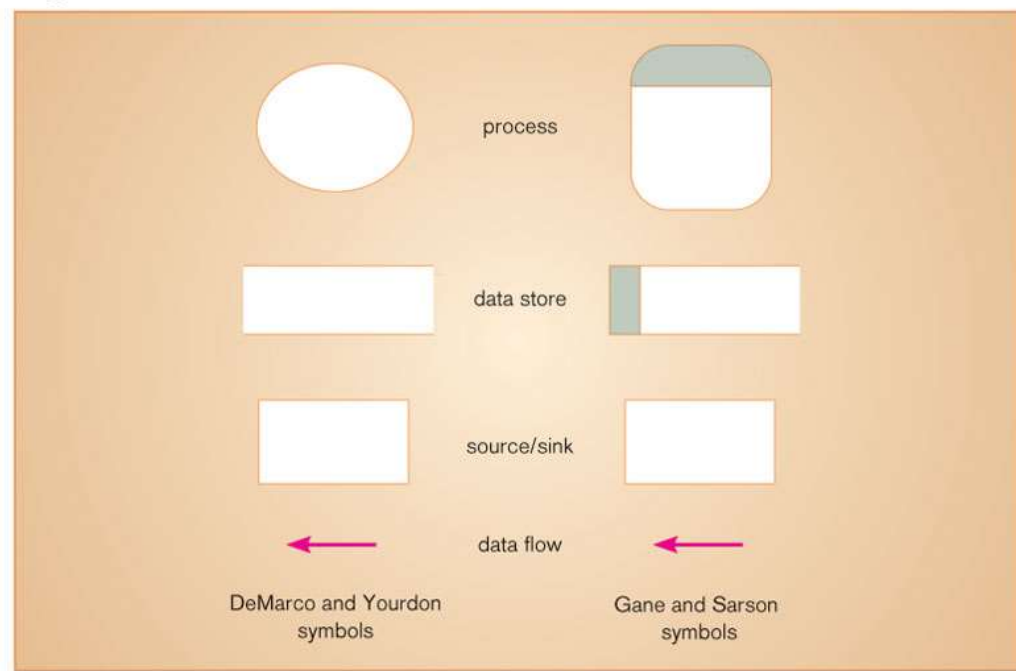
S.No.	Flow Chart	Data Flow Diagram (DFD)
1.	The main objective is to represents the flow of control in program. Flowcharts depict details of physical systems	The main objective is to represent the processes and data flow between them. DFDs depict logical data flow independent of technology.
2.	It has only a single type of arrow is used to show the control flow in flow chart.	It defines the flow and process of data input, data output, and storing data.
3	It is the view of the system at a lower level.	It is the view of the system at a high level.
4.	These are represented by three symbols.	These are represented by five symbols.
5.	It deals with the physical aspect of the action.	It deals with the logical aspect of the action.
6.	It shows the how to make system function.	It defines the functionality of the system.
7.	It is not very suitable for a complex system.	It is used for complex systems.

Data Flow Diagram (DFD)

- A picture of the movement of data between external entities and the processes and data stores within a system.
- Data Flow Diagram=Data+Flow+Diagram ;where,
 - Data=Information
 - Flow=movement
 - Diagram=a picture to represent something.
- So, DFD is basically the graphical representation of the flow of data or information.
- It is a framework or pattern of the data systems.
- It includes storing data, data input, data output.
- It is describes as the process of taking the data as input, storing the data, giving the data as output.
- It describes the path of data that completes the process.

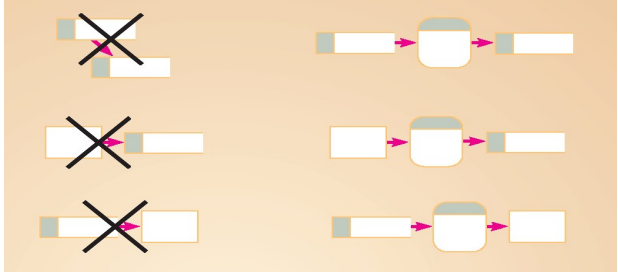
DFD Symbols

Figure 7-2 Comparison of DeMarco and Yourdon and Gane and Sarson DFD symbol sets



- Process: work or actions performed on data (inside the system)
- Data store: data at rest (inside the system)
- Source/sink: external entity that is origin or destination of data (outside the system)
- Data flow: arrows depicting movement of data

DFD Diagramming Rules:

S.N.	Rules	Depiction
1	- No process can have only outputs(called Miracle) or only inputs(called blackhole)...processes must have both outputs and inputs. - Process labels should be verb phrases.	
2	- All flows to or from a data store must move through a process. - Data store labels should be noun phrases	
3	- No data moves directly between external entities without going through a process. - Interactions between external entities without intervening processes are outside the system and therefore not represented in the DFD. - Source and sink labels should be noun phrases.	

DFD Diagramming Rules:

S.N.	Rules	Depiction
4	- Bidirectional flow between process and data store is represented by <u>two separate arrows</u>. - Forked data flow must refer to <u>exact same data item</u> (not different data items) from a common location to multiple destinations.	
5	- Joined data flow must refer to <u>exact same data item</u> (not different data items) from multiple sources to a common location. - Data flow cannot go directly from a process to itself, must go through intervening processes.	
6	Data flow from a process to a data store means update (insert, delete or change).	
7.	Data flow from a data store to a process means retrieve or use.	

DFD Decomposition

- **Functional Decomposition**

- An iterative process of breaking a system description down into finer and finer detail
- High-level processes described in terms of lower-level sub-processes
- DFD charts created for each level of detail

- **DFD Level**

- Context DFD
 - Overview of the organizational system
- Level-0 DFD
 - Representation of system's major processes at high level of abstraction
- Level-1 DFD
 - Results from decomposition of Level 0 diagram
- Level- n DFD
 - Results from decomposition of Level $n-1$ diagram

- **Primitive DFD Level:**

- Lowest Level of DFD

Context Diagram

- It represents the system with a single process and then shows the external agents with the system interacts.
- It shows the top-level view of System.
- Context diagrams show the system boundaries, external entities that interact with the system, and major information flows between entities and the system.
- It's a first DFD in every business process, which shows the context into which the business process fits.
- In Context diagram the overall business process is shown as just *one* process (process 0), Shows all the external entities that receive information from or contribute information to the system.
- Context Diagram is an overview of an organizational system that shows:
 - The system boundaries;
 - External entities that interact with the system;
 - Major information flows between the entities and the system.
 - **Note:** only one process symbol, and no data stores shown.

Benefits:

- Shows the scope and boundaries of a system at a glance including the other systems that interface with it
- No technical knowledge is assumed or required to understand the diagram
- Easy to draw and amend due to its limited notation
- Easy to expand by adding different levels of DFDs
- Can benefit a wide audience including stakeholders, business analyst, data analysts, developers

Drawback:

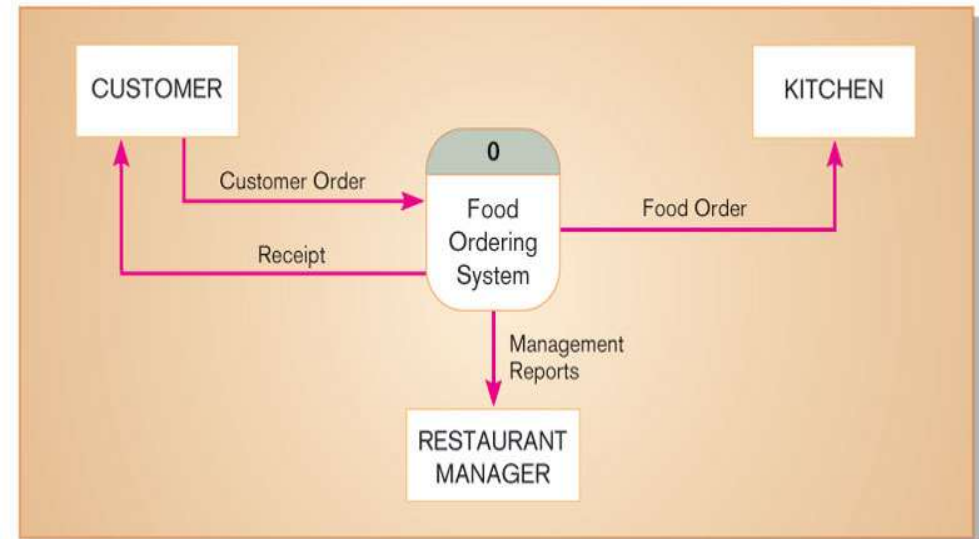
- fail to give details about the sequence or timing of project processes.

How to construct Context diagram

- Identify data-flows by listing the major documents and information flows associated with the system
- Identify external entities by identifying sources and recipients of the data-flows, which lie outside of the system under investigation.
- Draw and label a process box representing the entire system.
- Draw and label the external entities around the outside of the process box.
- Add the data-flows between the external entities and the system box.

Example of Context diagram: Food Ordering System

Figure 7-4 Context diagram of Hoosier Burger's food ordering system



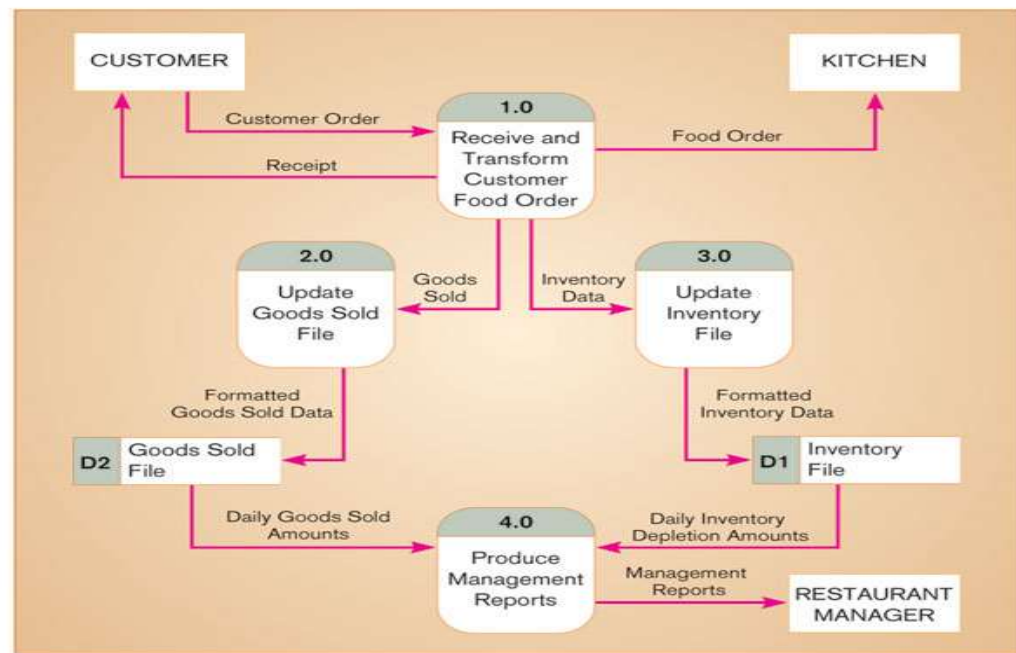
Level-0 DFD

- Level-0 DFD shows the system's major processes, data flows, and data stores at a high level of abstraction.
- Processes are labeled 1.0, 2.0, etc. These will be decomposed into more primitive (lower-level) DFDs.

Note:

- Level 0 is the decomposition of the Process in context level.
- The source and sink are same as context diagram. No source or sinks can be added.
- The incoming and outgoing process from source and to sinks also have the same labels as in context diagram. We can not change it. We only can change the labels between the sub-processes.
- We can use the database for storage, retrieval and updating of information.

Figure 7-5 Level-0 DFD of Hoosier Burger's food ordering system



Level 1 DFD

- Level-1 DFD shows the sub-processes of one of the processes in the Level-0 DFD.
- Here the Process 4.0 of level 0 has been further decomposed into 4.1, 4.2 and 4.3.
- These can be further decomposed in more primitive (lower-level) DFDs if necessary.

Level-N DFD

- Level- n DFD shows the sub-processes of one of the processes in the Level $n-1$ DFD.
- This is a Level-2 DFD for Process 4.3.
- Processes are labeled 4.3.1, 4.3.2, etc. If this is the lowest level of the hierarchy, it is called a *primitive DFD*

Figure 7-8 Level-1 diagram showing the decomposition of Process 4.0 from the level-0 diagram for Hoosier Burger's food ordering system

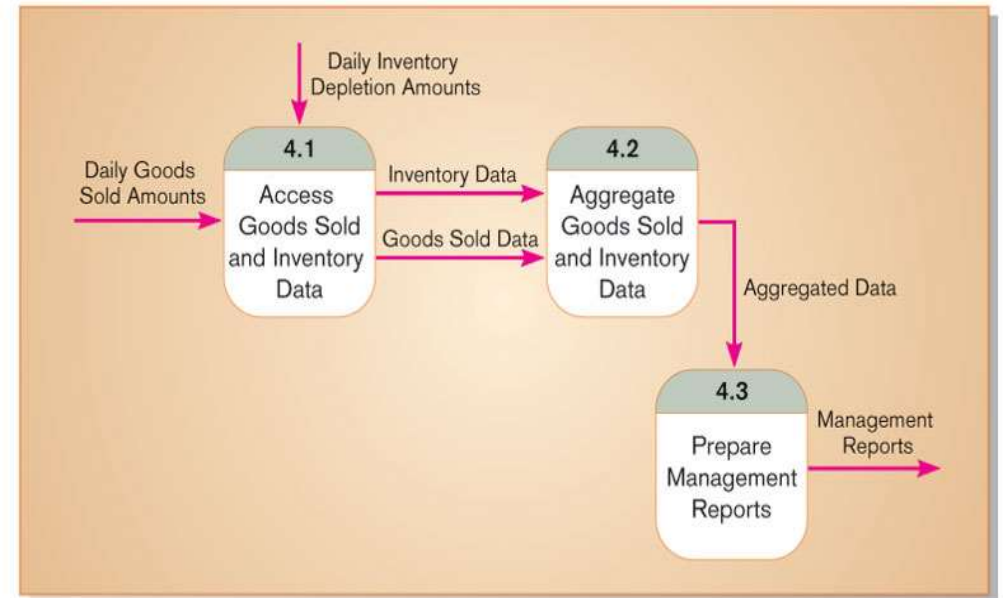
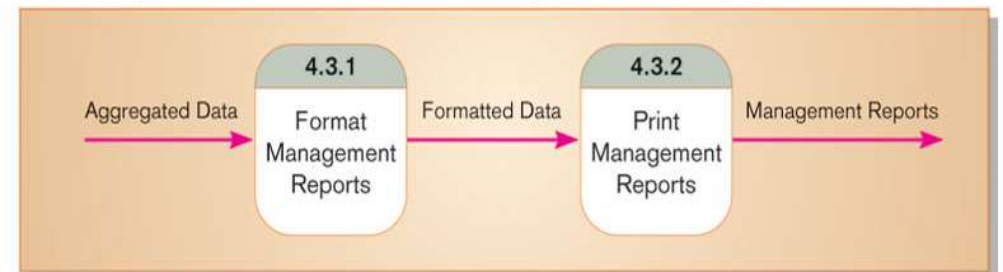


Figure 7-9 Level-2 diagram showing the decomposition of Process 4.3 from the level-1 diagram for Process 4.0 for Hoosier Burger's food ordering system



DFD Balancing

- **Conservation Principle:** conserve inputs and outputs to a process at the next level of decomposition.
- **Balancing:** conservation of inputs and outputs to a data flow diagram process when that process is decomposed to a lower level.
- **Balanced means:**
 - Number of inputs to lower level DFD equals number of inputs to associated process of higher level DFD
 - Number of outputs to lower level DFD equals number of outputs to associated process of higher level DFD

Figure 7-10a An unbalanced set of data flow diagrams - Context diagram

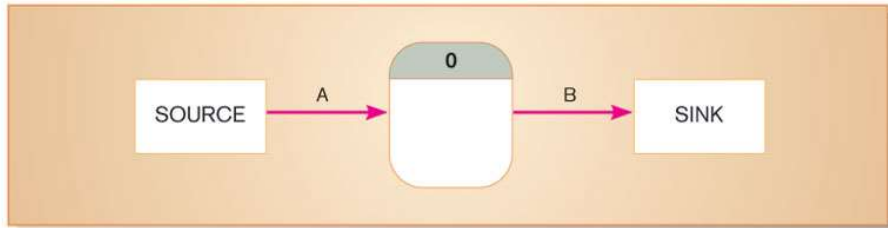
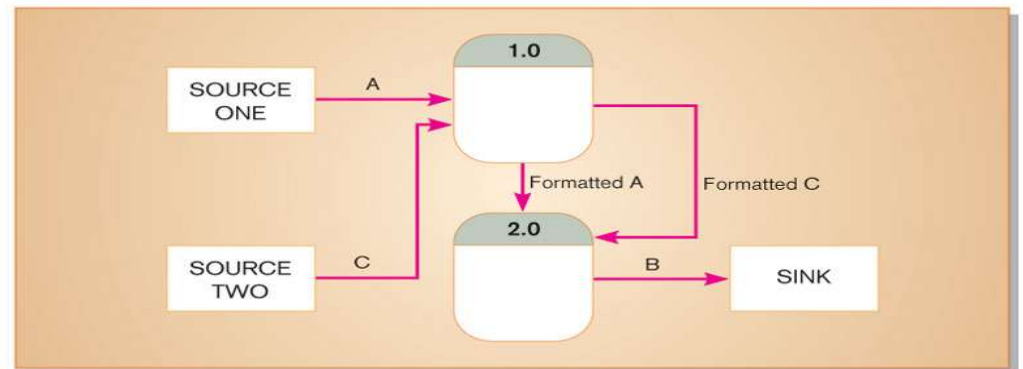
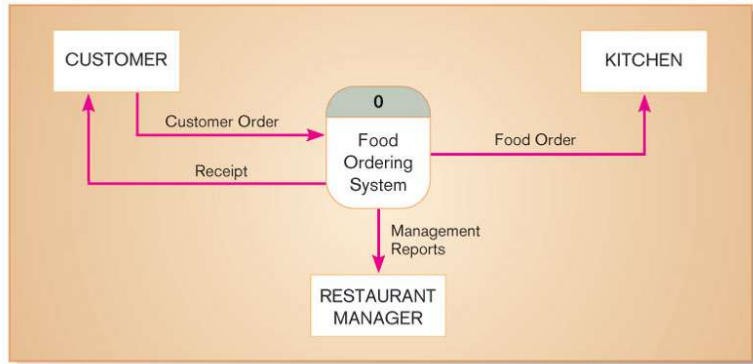


Figure 7-10b An unbalanced set of data flow diagrams - Level-0 diagram



This is unbalanced because the process of the context diagram has only one input but the Level-0 diagram has two inputs.

Figure 7-4 Context diagram of Hoosier Burger's food ordering system



These are balanced because the numbers of inputs and outputs of context diagram process equal the number of inputs and outputs of Level-0 diagram.

Data Flow Splitting

- A composite data flow at a higher level may be split if different parts go to different processes in the lower level DFD.

Figure 7-11a Example of data flow splitting - Composite data flow

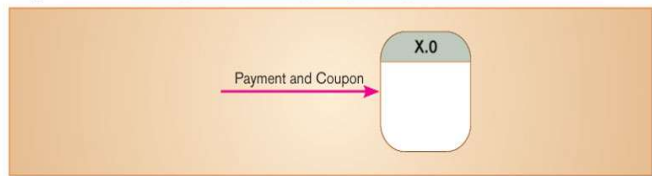


Figure 7-5 Level-0 DFD of Hoosier Burger's food ordering system

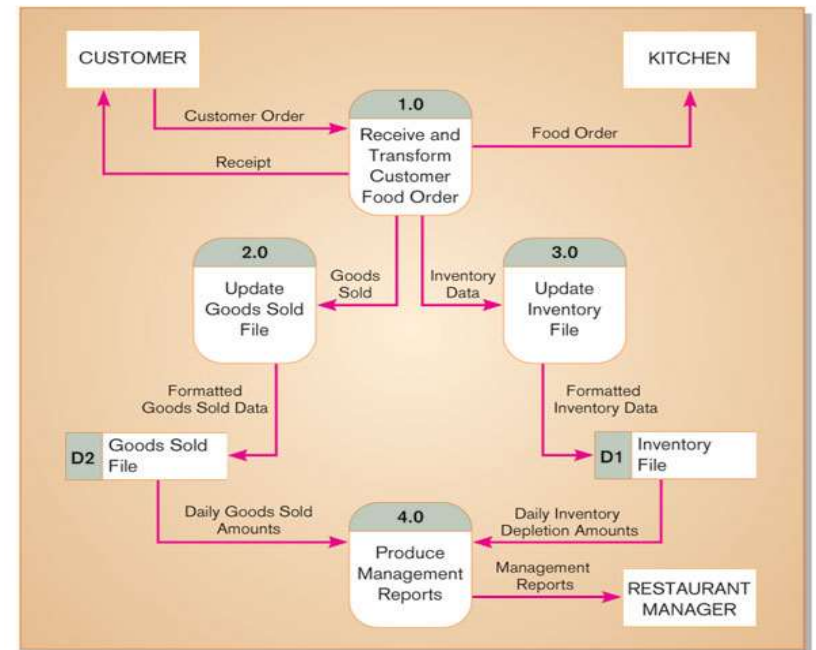
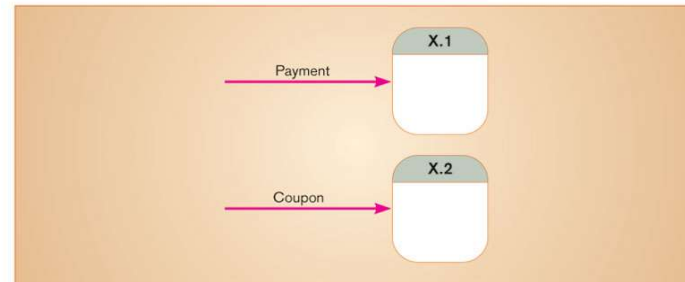


Figure 7-11b Example of data flow splitting - Disaggregated data flows



This remains balanced because the same data is involved, but split into two parts.

Four Different Types of DFDs

- Current Physical
 - Process labels identify technology (people or systems) used to process the data.
 - Data flows and data stores identify actual name of the physical media.
- Current Logical
 - Physical aspects of system are removed as much as possible.
 - Current system is reduced to data and processes that transform them.
- New Logical
 - Includes additional functions
 - Obsolete functions are removed
 - Inefficient data flows are reorganized
- New Physical
 - Represents the physical implementation of the new system

Guidelines for Drawing DFDs

- Completeness
 - DFD must include all components necessary for system.
 - Each component must be fully described in the project dictionary or CASE repository.
- Consistency
 - The extent to which information contained on one level of a set of nested DFDs is also included on other levels.
- Timing
 - Time is not represented well on DFDs.
 - Best to draw DFDs as if the system has never started and will never stop.
- Iterative Development
 - Analyst should expect to redraw diagram several times before reaching the closest approximation to the system being modeled.
- Primitive DFDs
 - Lowest logical level of decomposition
 - Decision has to be made when to stop decomposition
- Rules for stopping decomposition
 - When each process has been reduced to a single decision, calculation or database operation
 - When each data store represents data about a single entity
 - When the system user does not care to see any more detail
 - When every data flow does not need to be split further to show that data are handled in various ways
 - When you believe that you have shown each business form or transaction, online display and report as a single data flow
 - When you believe that there is a separate process for each choice on all lowest-level menu options

Advantages of data flow diagrams

- ✓ It gives further understanding of the interestedness of the system and sub-systems
- ✓ It is useful from communicating current system knowledge to the user
- ✓ Used as part of the system documentation files
- ✓ Data flow diagram helps to substantiate the logic underlining the dataflow of the organization
- ✓ It gives the summary of the system
- ✓ DFD is very easy to follow errors and it is also useful for quick reference to the development team for locating and controlling errors

Disadvantages of data flow diagram

- ✓ DFD is likely to take many alteration before agreement with the user
- ✓ Physical consideration are usually left out
- ✓ It is difficult to understand because it ambiguous to the user who have little or no knowledge

Modeling Logic with Decision Tables, Decision Trees, and Pseudocodes

- **Structured English or Pseudocodes**

- Modified form of English used to specify the logic of information processes like Pseudocode.
- Uses a subset of English vocabulary to express process procedures
 - Action verbs – read, write, print, move, merge, add, sort
 - Noun phrases – name, address
 - No adjectives or adverbs
- No specific standards – each analyst will have his own way
- File and variable names are CAPITALIZED
- Logical comparisons are spelled out and not used symbols

- Structured English is used to represent processes in a shorthand manner that is relatively easy for users and programmers to read and understand

- No specific standards.
- Decisions are made through the use of if , then ,else and so statements like If conditions, Case statements, for loop, while loop etc.
- It uses logical constructs to carry out instructions for actions .
- There are three valid constructs in this notations:
 - A sequence of single declarative statements
 - The selection of one or more declarative statements based on a decision like if then , else ,switch, case etc...
 - The repetition of one or more declarative statements . Example Looping constructs , for, while, do while etc..

- **Example of Structured English:**

```
BEGIN IF
    IF quantity in stock is less than minimum-quantity
    THEN generate new quantity order
    ELSE
    Do nothing
END IF
```

- **Rules for Pseudocodes**

- **Write only one statement per line**

- Each statement in pseudocode should express just one action for the computer. If the task list is properly drawn, then in most cases each task will correspond to one line of pseudocode.

Task List

Read name, hours worked, rate of pay

Perform calculations

gross = hours worked * rate of
pay

Write name, hours worked, gross

Pseudocode

READ name, hoursWorked, payRate

gross = hoursWorked * payRate

WRITE name, hoursWorked, gross

- **Capitalize initial keyword**
- **In the example below note the words: READ and WRITE. These are just a few of the keywords to use, others include:**

READ, WRITE, IF, ELSE, ENDIF, WHILE, ENDWHILE

- **Indent to show hierarchy**
- **End multiline structures**
- **Keep statements language independent**

Figure 8-2 Current logical DFD for Hoosier Burger’s inventory control system

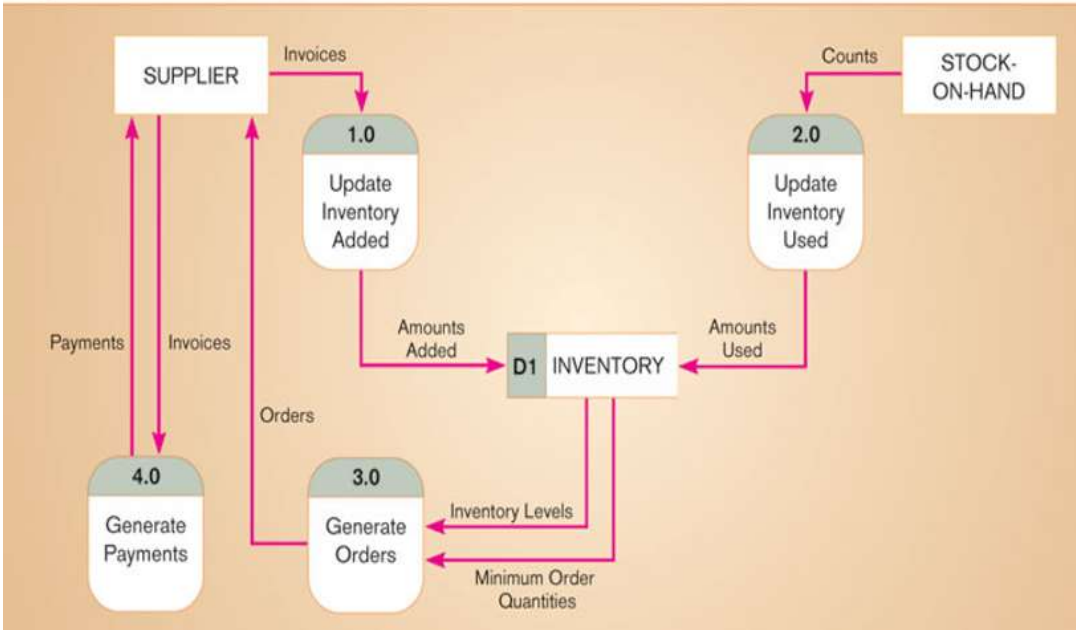


Figure 8-3 Structured English representations of the four processes depicted in Figure 8-2

Process 1.0: Update Inventory Added DO READ next Invoice-item-record FIND matching Inventory-record ADD Quantity-added from Invoice-item-record to Quantity-in-stock on Inventory-record UNTIL End-of-file
Process 2.0: Update Inventory Used DO READ next Stock-item-record FIND matching Inventory-record SUBTRACT Quantity-used on Stock-item-record from Quantity-in-stock on Inventory-record UNTIL End-of-file
Process 3.0: Generate Orders DO READ next Inventory-record BEGIN IF If Quantity-in-stock is less than Minimum-order-quantity THEN GENERATE Order END IF UNTIL End-of-file
Process 4.0: Generate Payments READ Today's-date DO SORT Invoice-records by Date READ next Invoice-record BEGIN IF IF Date is 30 days or greater than Today's-date THEN GENERATE Payments END IF UNTIL End-of-file

- Structured English is not good to represent *complicated logic* (having several different conditions) as it becomes difficult to understand. Decision Table or Decision Tree are the solution of this situation.
- **Decision Table:**
 - Decision tables describe **all possible combinations of conditions** and the decision appropriate to each combination
 - a matrix representation of the logic of a decision which specifies the possible conditions for the decision and the resulting actions.
 - Best used for complicated decision logic.
 - Specifies all the possible conditions and the resulting actions in a tabular form

3 Parts of a Decision Table

1. Condition stubs
 - Lists condition relevant to decision
2. Action stubs
 - Actions that result from a given set of conditions
3. Rules
 - Specify which actions are to be followed for a given set of conditions

Indifferent Condition

- Condition whose value *does not affect* which action is taken for two or more rules

- **Procedure for Creating Decision Table:**

- Name the conditions and values each condition can assume
 - some conditions values will be just “yes” or “no” and some may have many values (called an extended entry)
- Name all possible actions that can occur
- List all possible rules
 - Create exhaustive set of rules – every possible combination of conditions must be represented
 - Some rules may be *redundant* or make *no sense* that can be altered later
 - **Number of rules = number of values for *condition 1* X number of values for *condition 2* XX number of values for *condition n***
- Define the actions for each rule
 - If an action doesn’t make sense create an “**impossible**” row for that action
 - If the action is not known place a ? for that rule
- Simplify the table
 - Remove any rules with **impossible** actions

◆ Decision tables can also be used to specify additional decision-related information:

◆ If actions for a rule are more complicated and can’t be conveyed in one or two lines of text (or)

◆ If some conditions depend on other conditions (nested conditions)

- use separate, **linked decision** table by writing “Perform Table B” as action in the action stub
- Table B could contain an action stub that returns to the original table

◆ Use **numbers** to indicate *sequence* rather than just Xs where rules and action stub intersect

◆ Decision tables are **compact** – pack a lot of information into a small table

◆ Decision tables allow you to check for the **completeness, consistency, and redundancy** of *logic*

DECISION TABLE TESTING – EMAIL EXAMPLE

Email
Password

SoftwareAndTestingTraining

Conditions	Email	F	T	F	T
	Password	F	F	T	T
Actions	Expected result	Error: Please enter email.	Error: Please enter password.	Error: Please enter email.	Login processed

Conditions	Email	F	T	F	T
	Password	F	F	T	T
Actions	Expected result	Error: Please enter email.	Error: Please enter password.	Error: Please enter email.	Login processed

No. of combinations = No. of condition1 values * No. of condition2 values

Decision Table Example:

Consider a Doctor clinic. In a clinic the customer are classified according to their age group and their regularity. For a customer with age less than or equal to 10 give full discount, customer with age greater than 40 give 10% discount and Customer with age greater than 10 and less than or equal to 40 give 5% discount if he is regular customer otherwise do not give discount. Prepare the decision table for above case.

Doctor's Clinic

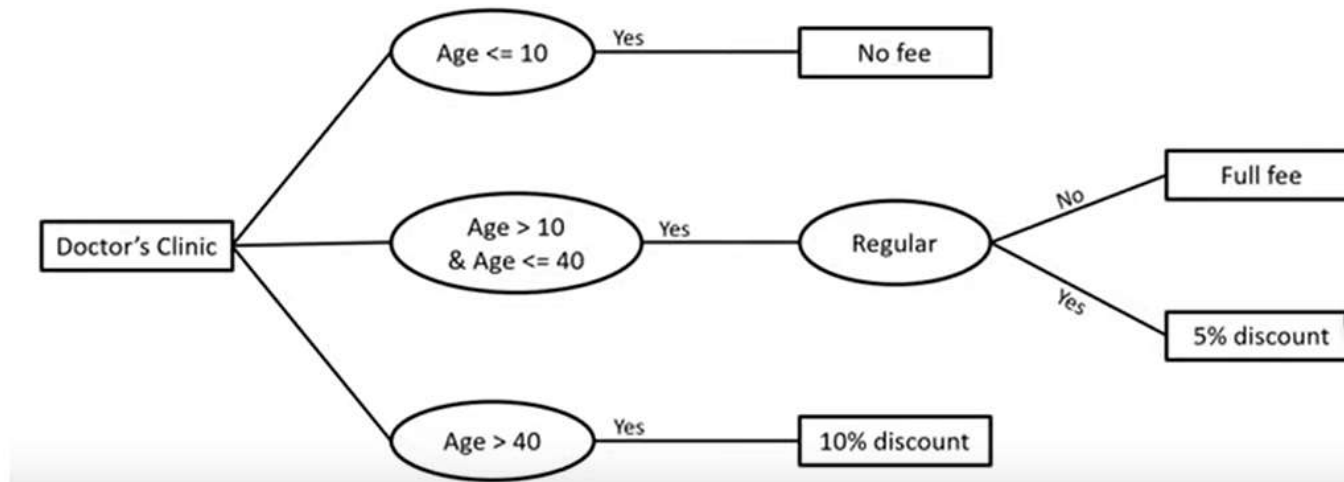
Condition	Rule 1	Rule 2	Rule 3	Rule 4
Age $\leq$ 10	Yes	-	-	-
Age $>$ 10 & Age $\leq$ 40	-	-	Yes	Yes
Age $>$ 40	-	Yes	-	-
Regular	-	-	Yes	No
Action				
Fee	No	Yes	Yes	Yes
Discount	-	10%	5%	No

Modeling Logic with Decision Trees

- A **decision tree** is a graphical representation of a decision situation
- Decision situation points (**nodes**) are connected together by **arcs** and terminate in **ovals**
- Main components
 - **Decision points** represented by **nodes**
 - **Actions** represented by **ovals**
 - Particular choices from a decision point represented by **arcs**
- To read a decision tree – begin at root node on far **left**
- Each **node** is numbered and each number corresponds to a **choice**
- Choices are spelled out in a **legend**
- From each node there are *at least two paths* leading to next step – another *decision point* or an *action*
- All possible **actions** are listed on the far **right** in **leaf nodes**
- Each **rule** is represented by tracing a series of *paths* from **root** node to the next node and so on until an **action oval** is reached

Decision Tree Example:

Consider a Doctor clinic. In a clinic the customer are classified according to their age group and their regularity. For a customer with age less than or equal to 10 give full discount, customer with age greater than 40 give 10% discount and Customer with age greater than 10 and less than or equal to 40 give 5% discount if he is regular customer otherwise do not give discount. Prepare the decision tree for above case.



Create a Structured English ,Decision Table and decision tree to represent the logic of payroll system described in the following narrative. There are two types of employees: salarized and hourly. All salarized employees get basic salary. Hourly wage is calculated for hourly worker. For hourly worker, if hours worked is less than 40 absence report is also produced and if it is greater than 40 overtime is also calculated.

SOLUTION:

STRUCTURED ENGLISH FOR THE PAYROLL SYSTEM:

PRINT "Enter employee type: (S)alaried or (H)ourly:"

GET employeeType

IF employeeType = "S" THEN

 CALL paySalary()

ELSE

 PRINT "Enter number of hours worked:"

 GET hoursWorked

 IF hoursWorked < 40 THEN

 CALL printAbsenceReport()

 CALL payRegularHours()

 ELSE IF hoursWorked = 40 THEN

 CALL payRegularHours()

 ELSE

 CALL payRegularHours()

 CALL payOvertime()

 END IF

END IF

Figure 8-4 Complete decision table for payroll system example

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	<40	<40	40	40	>40	>40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

Figure 8-9 Decision tree representation of the decision logic in the decision tables in Figures 8-4 and 8-5, with only two choices per decision point

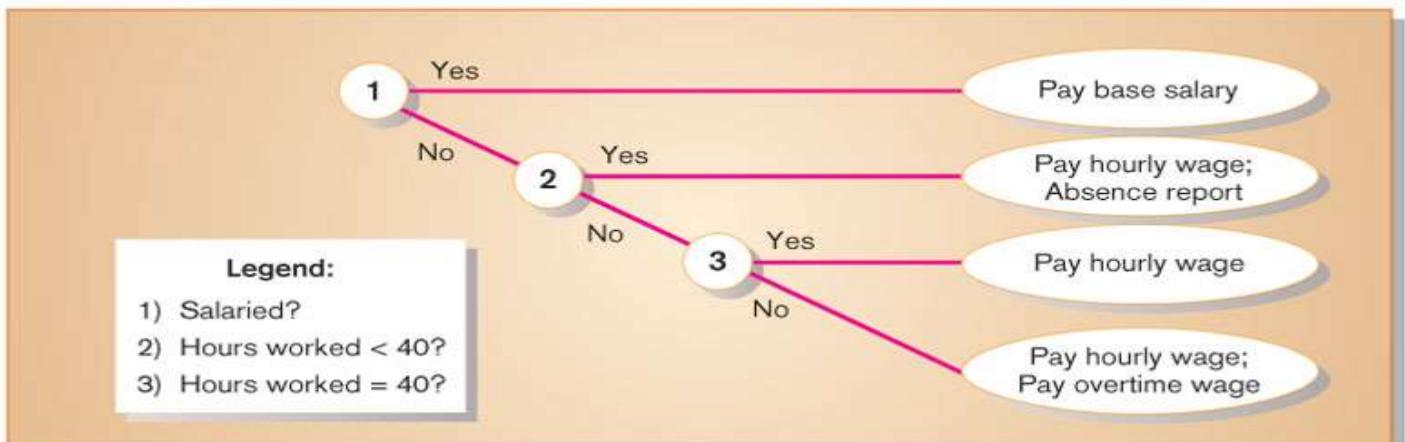
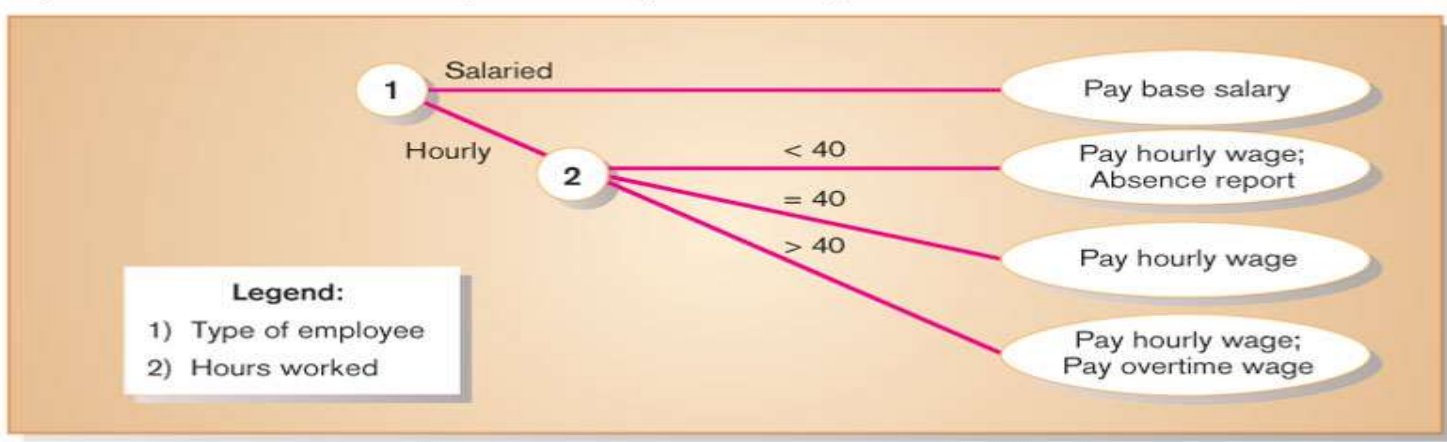


Figure 8-10 Decision tree representation of the decision logic in the decision tables in Figure 8-4 and 8-5, with multiple choices per decision point



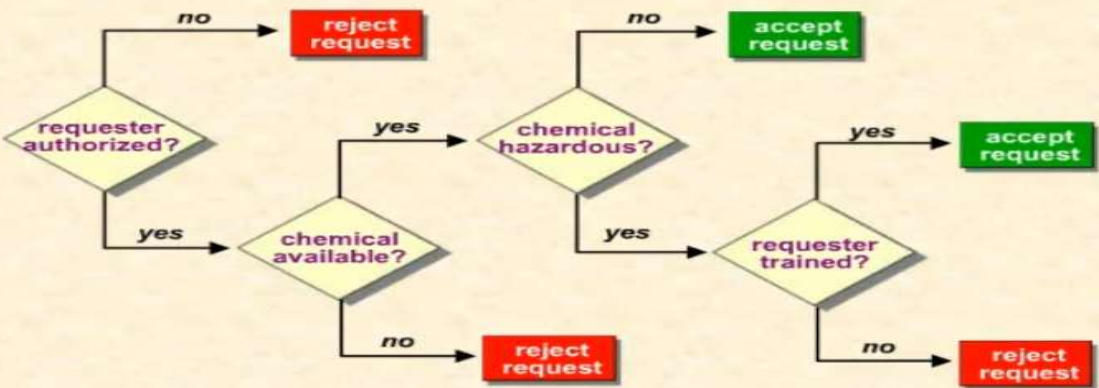
■ How does the Chemical Tracking System decide whether to approve or reject a request?

- ✓ Is the requester authorized to request chemicals?
- ✓ Is the chemical available either in the chemical stockroom or from a vendor?
- ✓ Is the chemical on the list of hazardous chemicals?
- ✓ Is the requester trained in handling hazardous chemicals?

A Sample Decision Table

Condition	Requirement Number				
	1	2	3	4	5
Requester is authorized	F	T	T	T	T
Chemical is available	—	F	T	T	T
Chemical is hazardous	—	—	F	T	T
Requester is trained	—	—	—	F	T
Action					
Accept request			X		X
Reject request	X	X		X	

A Sample Decision Tree



Deciding Among Structured English, Decision Tables, and Decision Trees

Criteria	Structured English	Decision Tables	Decision Trees
Determining Conditions and Actions	Second Best	Third Best	Best
Transforming Conditions and Actions into Sequence	Best	Third Best	Best
Checking Consistency and Completeness	Third Best	Best	Best

Deciding Between Decision Tables and Decision Trees

Criteria	Decision Tables	Decision Trees
Portraying complex logic	Best	Worst
Portraying simple rules	Worst	Best
Making decisions	Worst	Best
More compact	Best	Worst
Easier to manipulate	Best	Worst

3.3.Structuring System Data Requirements

- Introduction;
- Conceptual Data Modeling;
- Gathering Information for Conceptual Data Modeling;
- Introduction to E-R Modeling

Learning Objectives

- Define key data modeling terms:
 - *entity type, attribute, multivalued attribute, relationship, degree, cardinality, business rule, associative entity, trigger, supertype, subtype.*
- Draw an E-R diagram to represent common business situations.
- Explain the role of conceptual data modeling in the overall analysis and design of an IS.

Learning Objectives

- Explain the role of prepackaged DB models (patterns) in data modeling.
- Distinguish between unary, binary, and ternary relationships and give an example of each.
- Define four basic types of business rules in a conceptual data model.
- Relate data modeling to process and logic modeling as different views of describing an IS

Conceptual Data Modeling

- **Conceptual data modeling:**

a detailed model that captures the overall structure of data in an organization

- Independent of any database management system (DBMS) or other implementation considerations

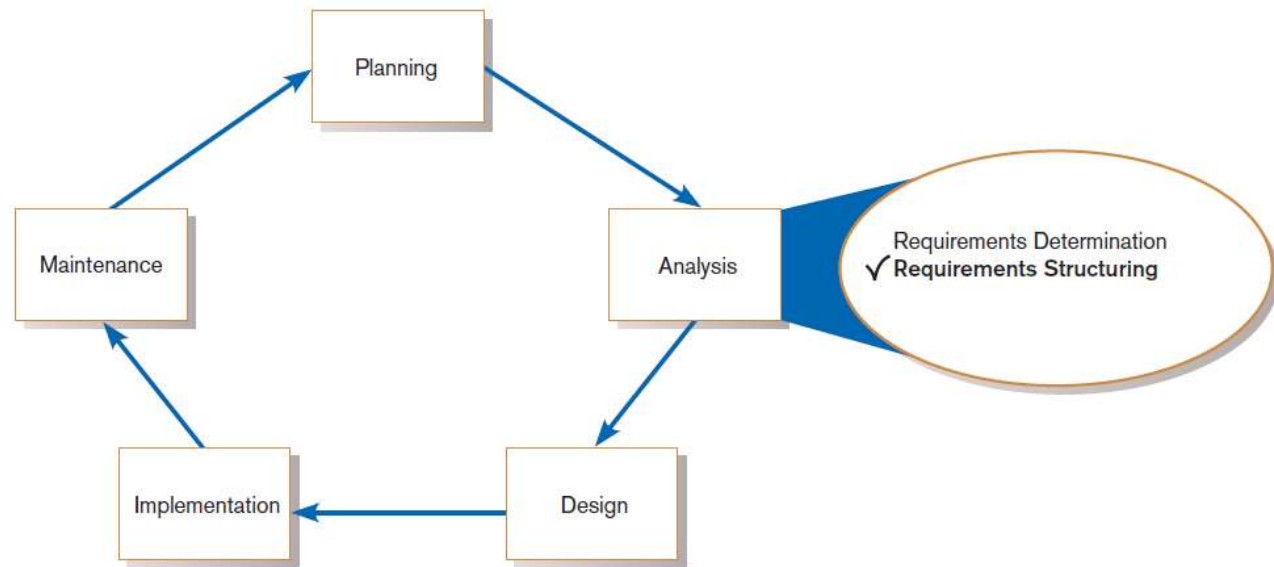


FIGURE: Systems development life cycle with analysis phase highlighted

Conceptual Data Modeling Process

- Develop a data model for the current system.
- Develop a new conceptual data model that includes all requirements of the new system.
- In the design stage, the conceptual data model is translated into a physical design.
- Project repository links all design and data modeling steps performed during SDLC.

Conceptual Data Modeling

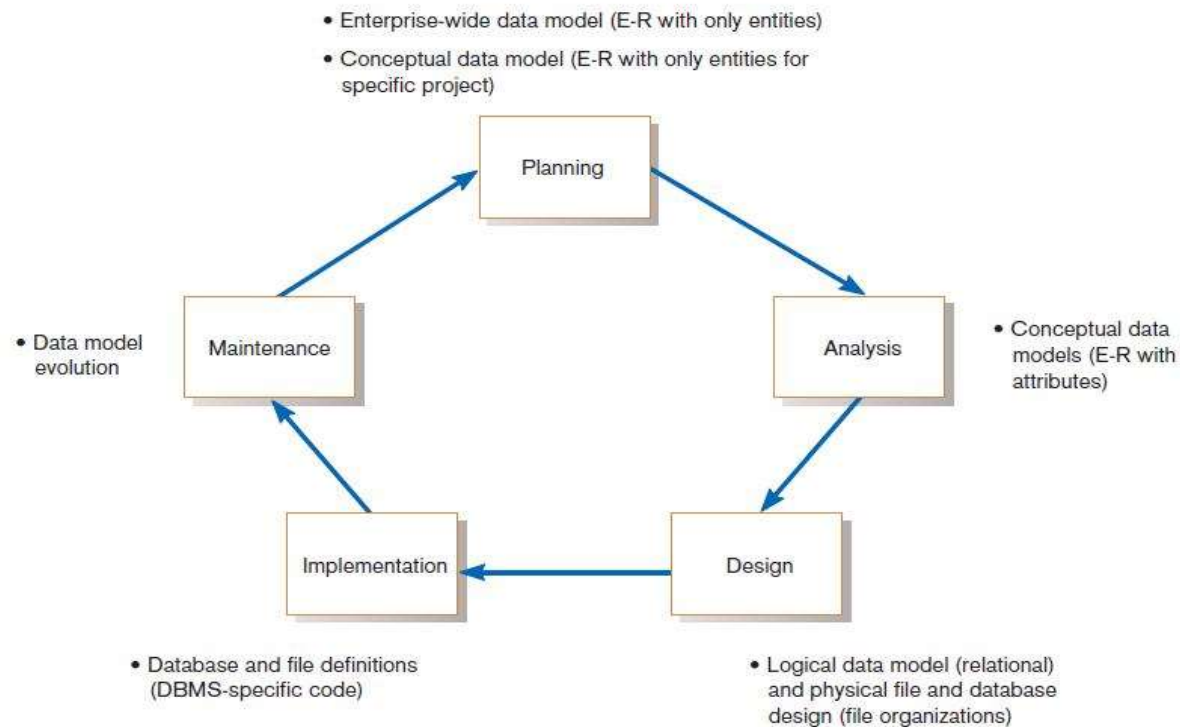
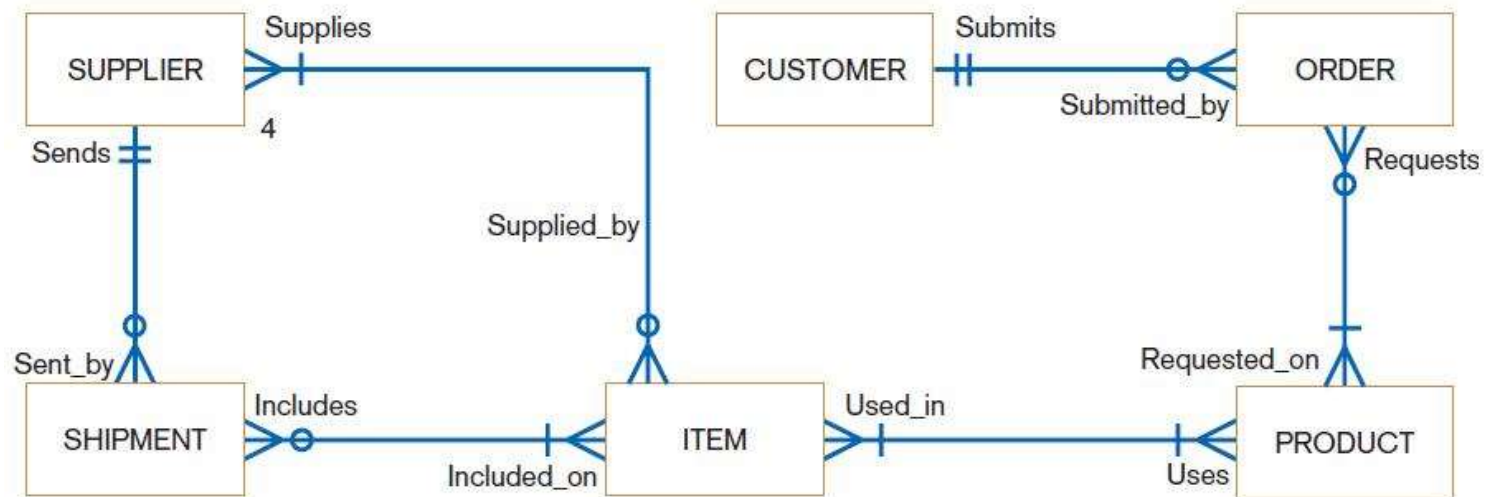


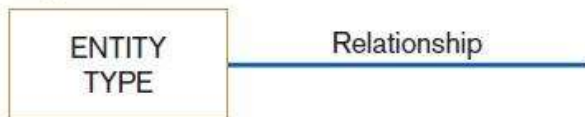
FIGURE 8-2
Relationship between data modeling and the SDLC

Deliverables and Outcome

- **Entity-relationship (E-R) diagram or UML class diagram**
 - **Entities** (or classes) – categories of data, represented as rectangles
 - **Relationships** (or associations) – lines between the entities
- Set of entries about data objects to be stored in repository project dictionary, or data modeling software
 - Repository links data, process, and logic models of an information system.
 - Data elements included in the data flow diagram (DFD) must appear in the data model and vice versa.
 - Each data store in a process model must relate to business objects represented in the data model.



Key



Cardinalities

—||—
Mandatory One

—|<—
Mandatory Many

—○|—
Optional One

—○<—
Optional Many

FIGURE 8-3

Sample conceptual data model

Gathering Information for Conceptual Data Modeling

- Two perspectives on data modeling:
 - Top-down approach for a data model is derived from an intimate understanding of the business.
 - Bottom-up approach for a data model is derived by reviewing specifications and business documents.

Gathering Information for Conceptual Data Modeling

- Requirements Determination Questions for Data Modeling:

- What are subjects/objects of the business?
 - Data entities and descriptions
- What unique characteristics distinguish between subjects/objects of the same type?
 - Primary keys
- What characteristics describe each subject/object?
 - Attributes and secondary keys
- How do you use the data?
 - Security controls and user access privileges
- Over what period of time are you interested in the data?
 - Cardinality and time dimensions
- Are all instances of each object the same?
 - Supertypes, subtypes, and aggregations
- What events occur that imply associations between objects?
 - Relationships and cardinalities
- Are there special circumstances that affect the way events are handled?
 - Integrity rules, cardinalities, time dimensions

Introduction to E-R Modeling

- **Entity-Relationship data model** (E-R model): a detailed, logical representation of the entities, associations and data elements for an organization or business area
- **Entity-relationship diagram** (E-R diagram): a graphical representation of an E-R model
- The E-R model is expressed in terms of:
 - Data entities in the business environment.
 - Relationships or associations among those entities.
 - Attributes or properties of both the entities and their relationships.

Introduction to E-R Modeling

- **Entity**: a person, place, object, event or concept in the user environment about which data is to be maintained
- **Entity type**: collection of entities that share common properties or characteristics
- **Entity instance**: single occurrence of an entity type

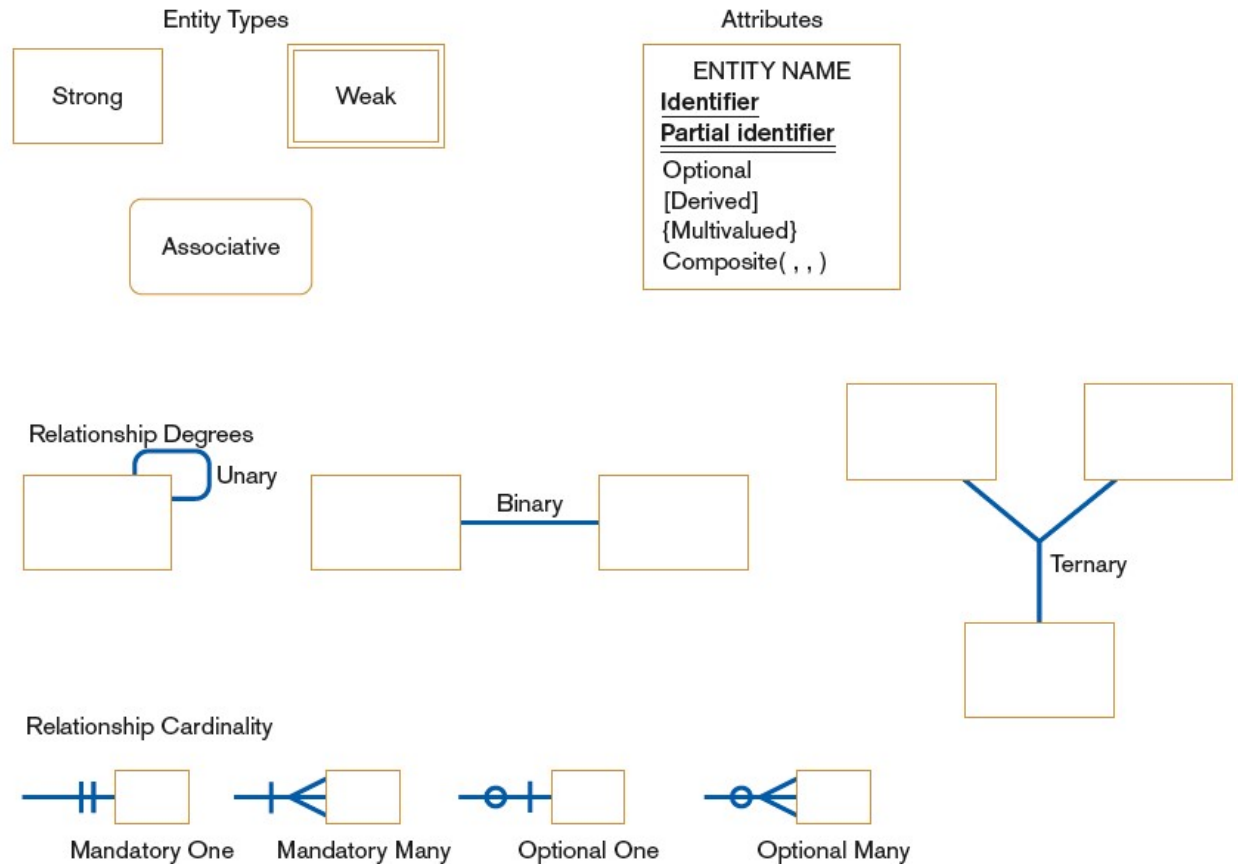


Fig: Basic E-R notation

Naming and Defining Entity Types

- An entity type name should be:
 - A singular noun.
 - Descriptive and specific to the organization.
 - Concise.
- Event entity type should be named for the result of the event, not the activity or process of the event.
- An entity type definition:
 - Includes a statement of what the unique characteristic(s) is (are) for each instance.
 - Makes clear what entity instances are included and not included in the entity type.
 - Often includes a description of when an instance of the entity type is created or deleted.
- For some entity types the definition must specify:
 - When an instance might change into an instance of another entity type.
 - What history is to be kept about entity instances.

Attributes

- **Attribute:** a named property or characteristic of an entity that is of interest to the organization
 - Naming an attribute: i.e. Vehicle_ID
 - Place its name inside the rectangle for the associated entity in the E-R diagram.
- **Naming and Defining Attributes**
 - An attribute name is a noun and should be unique.
 - To make an attribute name unique and for clarity, each attribute name should follow a standard format.
 - Similar attributes of different entity types should use similar but distinguishing names.
 - An attribute definition:
 - States what the attribute is and possibly why it is important.
 - Should make it clear what is included and what is not included.
 - Contains any aliases or alternative names.
 - States the source of values for the attribute.
 - An attribute definition should indicate:
 - If a value for the attribute is required or optional.
 - If a value for the attribute may change.
 - Any relationships that attribute has with other attributes.

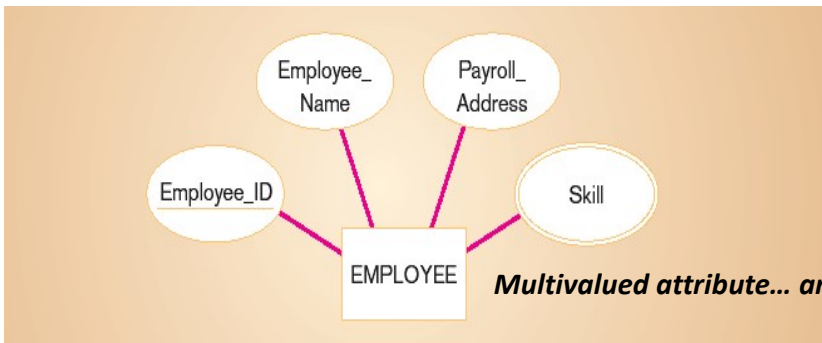
Candidate Keys and Identifiers.

- **Candidate key:** an attribute (or combination of attributes) that uniquely identifies each instance of an entity type
- **Identifier:** a candidate key that has been selected as the unique, identifying characteristic for an entity type
- Selection rules for an identifier
 - Choose a candidate key that will not change its value.
 - Choose a candidate key that will never be null.
 - Avoid using intelligent keys.
 - Consider substituting single value surrogate keys for large composite keys.

Other Attribute Types

- **Other attribute types:**

- **Multivalued attribute:** an attribute that may take on more than one value for each entity instance.
 - Represented on E-R Diagram in two ways:
 - double-lined ellipse
 - weak entity
- **Repeating group:** a set of two or more multivalued attributes that are logically related
- **Required attribute:** an attribute that must have a value for every entity instance
- **Optional attribute:** an attribute that may not have a value for every entity instance
- **Composite attribute:** an attribute that has meaningful component parts
- **Derived attribute:** an attribute whose value can be computed from related attribute values

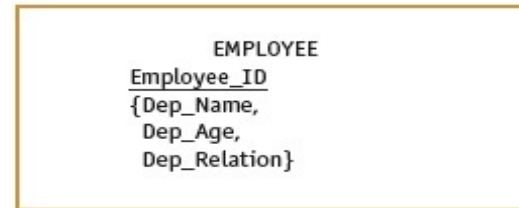


Multivalued attribute... an employee may have more than one skill.

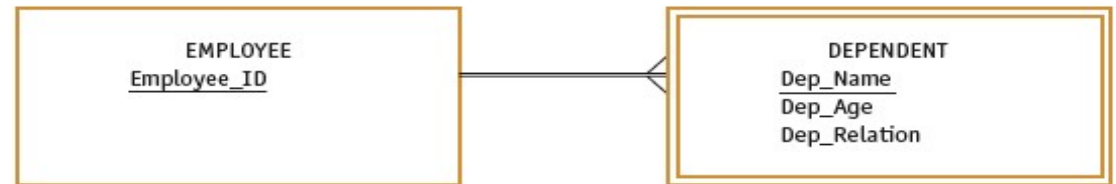
Employee_ID: Identifier attribute... each employee has a unique ID.



(a) Multivalued attribute skill



(b) Repeating group of dependent data



(c) Weak entity for dependent data

Fig: Multivalued attributes and repeating groups

Relationships

- **Relationship:** an association between the instances of one or more entity types that is of interest to the organization
- **Degree:** the number of entity types that participate in a relationship
 - **Unary relationship:** a relationship between the instances of one entity type
 - Also called a recursive relationship
 - **Binary relationship:** a relationship between instances of two entity types
 - Most common type of relationship encountered in data modeling
 - **Ternary relationship:** a simultaneous relationship among instances of three entity types

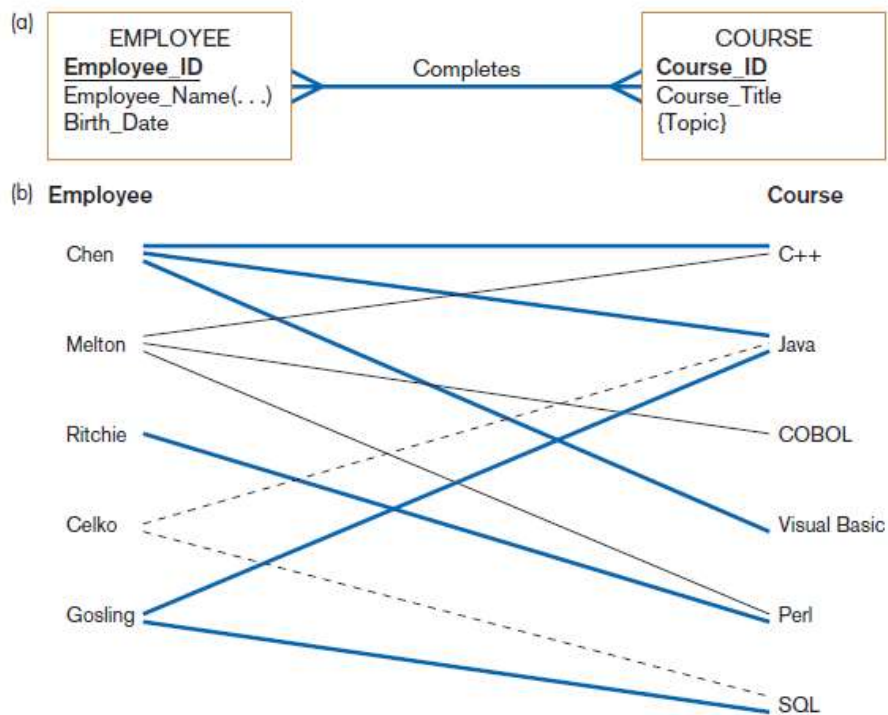


Figure :
 Relationship type and instances
 (a) Relationship type (Completes)
 (b) Relationship instances

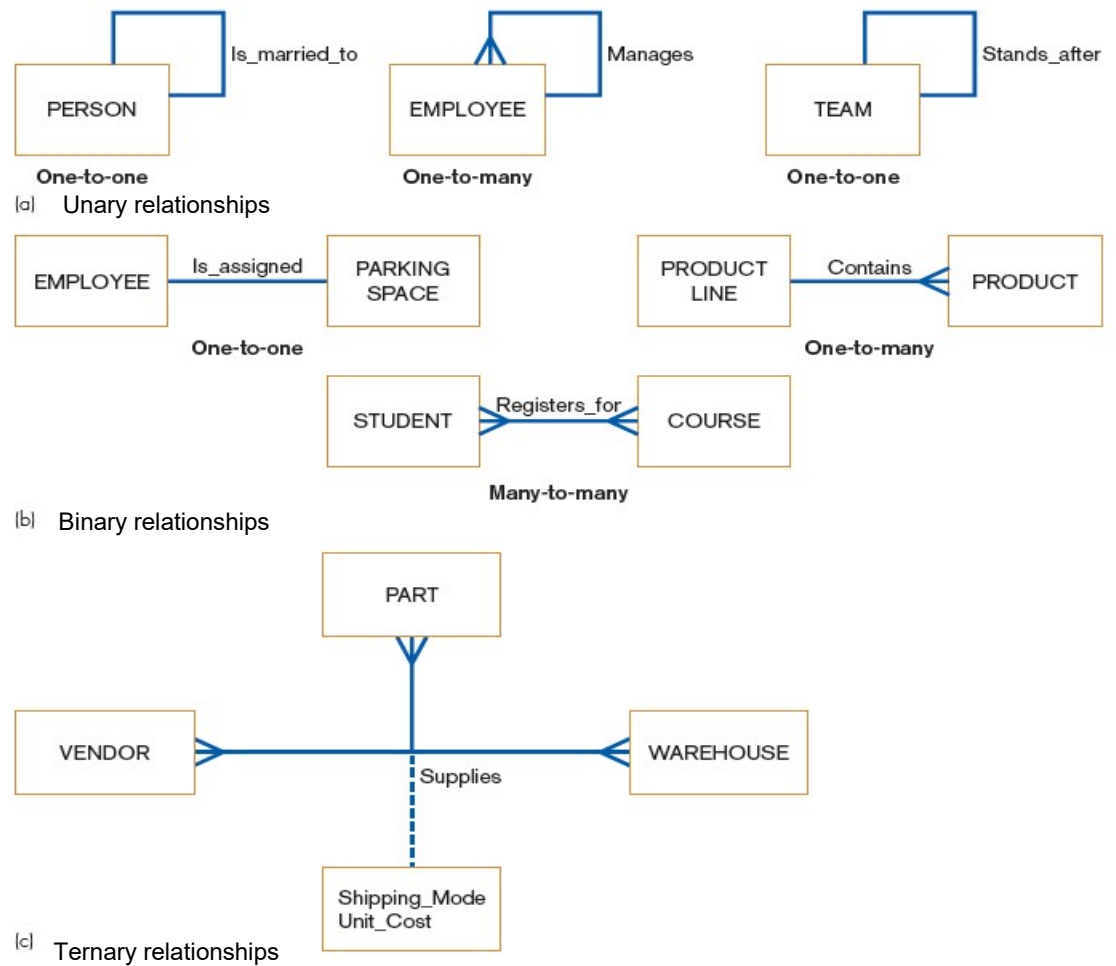
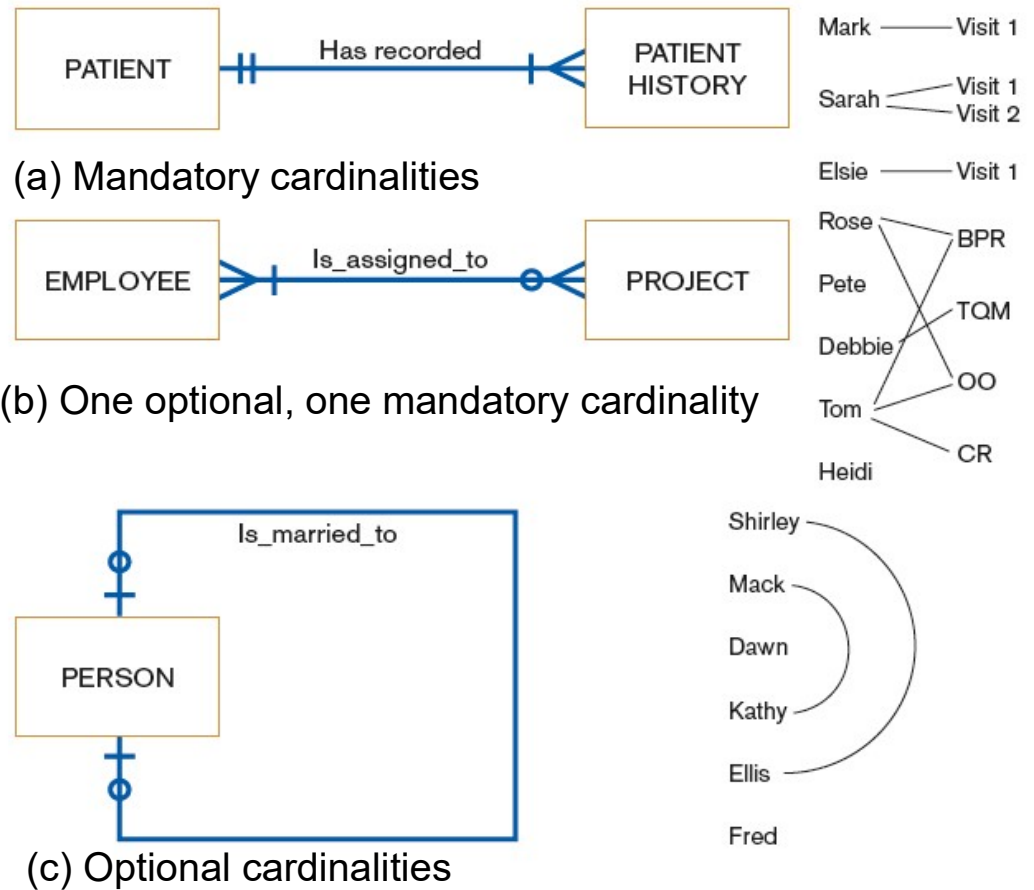


FIGURE :Examples of relationships of different degrees

Cardinalities in Relationships

- **Cardinality:** the number of instances of entity B that can (or must) be associated with each instance of entity A
- **Minimum Cardinality**
 - The minimum number of instances of entity B that may be associated with each instance of entity A
- **Maximum Cardinality**
 - The maximum number of instances of entity B that may be associated with each instance of entity A
- **Mandatory vs. Optional Cardinalities**
 - Specifies whether an instance must exist or can be absent in the relationship

FIGURE Examples of cardinality constraints



Naming and Defining Relationships

- A relationship name is a verb phrase; avoid vague names.
- A relationship definition:
 - Explains what action is to be taken and possibly why it is important.
 - Gives examples to clarify the action.
- A relationship definition should:
 - Explain any optional participation.
 - Explain the reason for any explicit maximum cardinality other than many.
 - Explain any restrictions on participation in the relationship.
 - Explain the extent of history that is kept in the relationship.
 - Explain whether an entity instance involved in a relationship instance can transfer participation to another relationship instance.

Steps in Creating ER-Diagram

Creating an E-R Diagram

A number of steps are required to create an E-R diagram that accurately represents the organizational data. These steps are summarized in Figure 8 and discussed in this section.

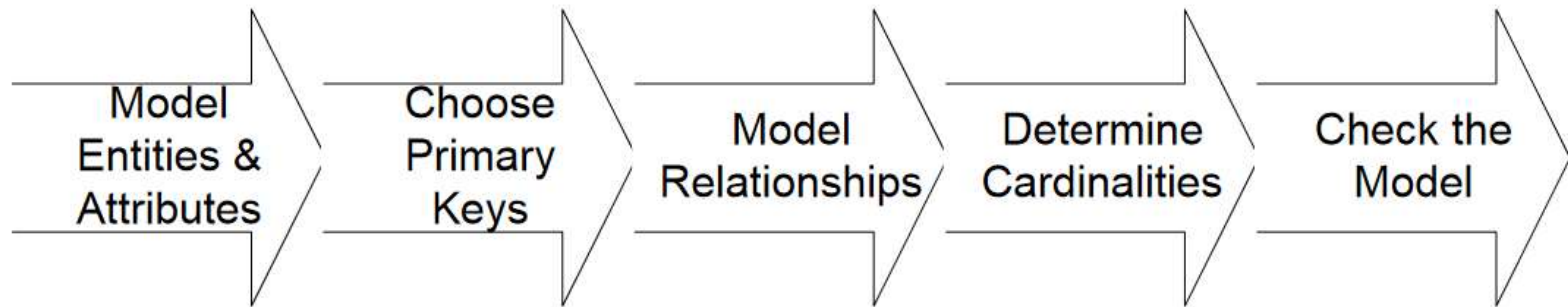


Figure 8: Steps in Building an E-R Diagram

EXAMPLE:

ORDER-NO:	44-44-4444	CUSTOMER-ID:	1002	
DATE:	10/31/98	CUST-NAME:	ABC Inc.	
PROD-ID	DESCRIPTION	PRICE	QTY	EXT
A123	STEREO SYSTEM	375.00	2	750.00
C235	8" SPEAKER	150.00	8	1,200.00
X002	SPEAKER WIRE	10.00	5	50.00
	TOTAL			2,000.00

Figure 9: Order Entry Form**Step 1: Model the Entities: List the Entities and attributes used in the system**

Entity	Attributes
ORDER	ORDER-NO, DATE, TOTAL
PRODUCT	PROD-ID, DESCRIPTION, PRICE
CUSTOMER	CUSTOMER-ID, CUST-NAME

Step 2: Choose Primary Keys

Desirable Primary Key Characteristics

1. Uniquely identifies an entity instance.
2. Non-null (always has a value)
3. Data-less
4. Never changes

Step 3: Model Relationships

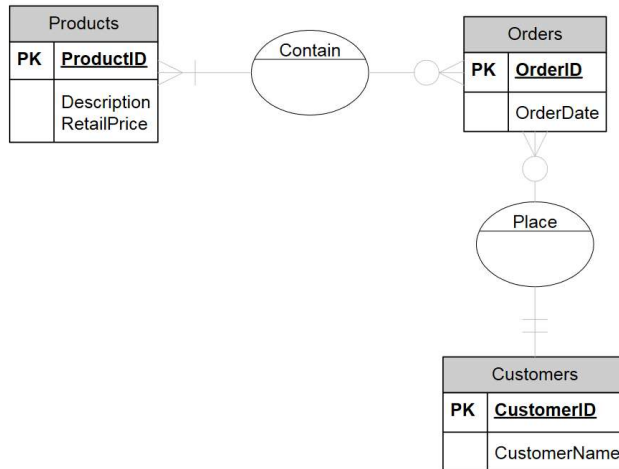


Figure 12: Relationships for Order Entry Database

Step 4: Determine Cardinalities

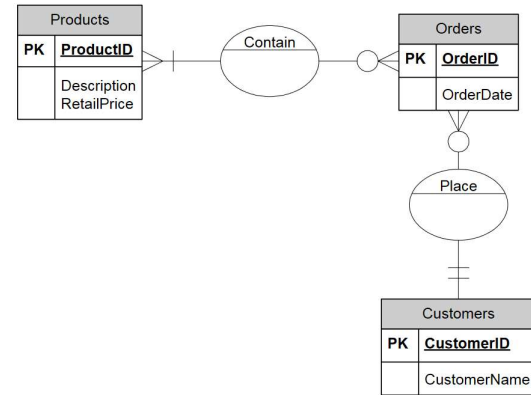


Figure 13: Incomplete Order Entry ERD with Cardinalities

Step 5 - Check the Model and complete

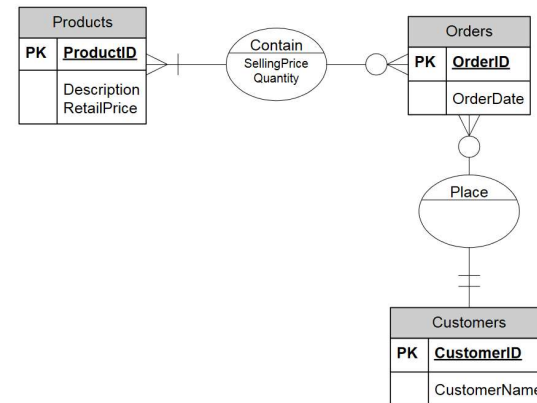


Figure 15: Complete Order Entry ERD

Special topics in ERD

- **Resolving Many-to-Many Relationships (Associative Entities)**

- Some database analysts find it useful to resolve many-to-many relationships (M:N) in the ERD.
- This makes the process of converting the ERD to a set of normalized relations easier.
- In order to resolve M:N relationships, you must create an associative entity (which is sometimes called a gerund), which is essentially an entity.
- It is called an associative entity because it is an entity that associates two instances of other entities with one another.
- An associative entity is shown in the ERD in the same way any other entity is shown.
- We use the many-to-many relationship between PRODUCTS and ORDERS as an example to illustrate how to create an associative entity.
- **Associative entities are relatively easy to create.** Start by creating a new entity (the associative entity) and using it to replace the many-to-many relationship.
- **Next, we need to decide what attributes to give the associative entity.**
- Any attributes that were part of the converted relationship should also be in the associative entity.
- In our example, the attributes "Quantity" and "SellingPrice" are attributes of the relationship, so we'll include these in the associative entity.
- **Next, we need to decide on a primary key for the associative entity.** A standard practice is to use the primary keys from the two original entities (PRODUCTS and ORDERS). In our example, these primary keys are ProductID and OrderID. We can use the composite key (ProductID+OrderID) if this can be uniquely expressed; otherwise need to select the key that can uniquely identified and expressed both the relationships.
- **The next step in creating the associative entity is to determine the cardinalities.** Maximum cardinalities are always the same when converting the M:N relationship. The maximum cardinalities coming into both sides of the associative entity are both many, while the maximum cardinalities going into the original entities is always a one.

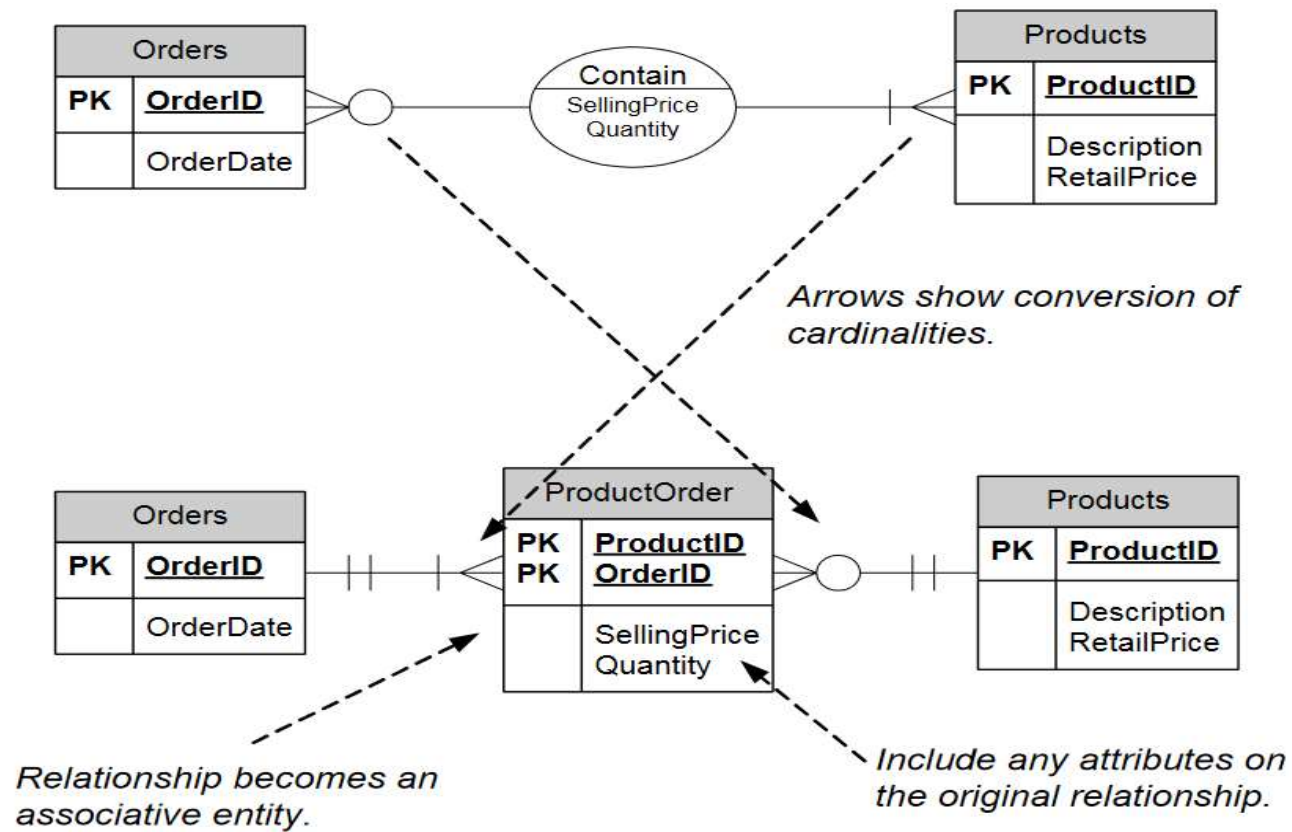


Figure 16: Converting a Many-to-Many Relationship

Associative Entities Example

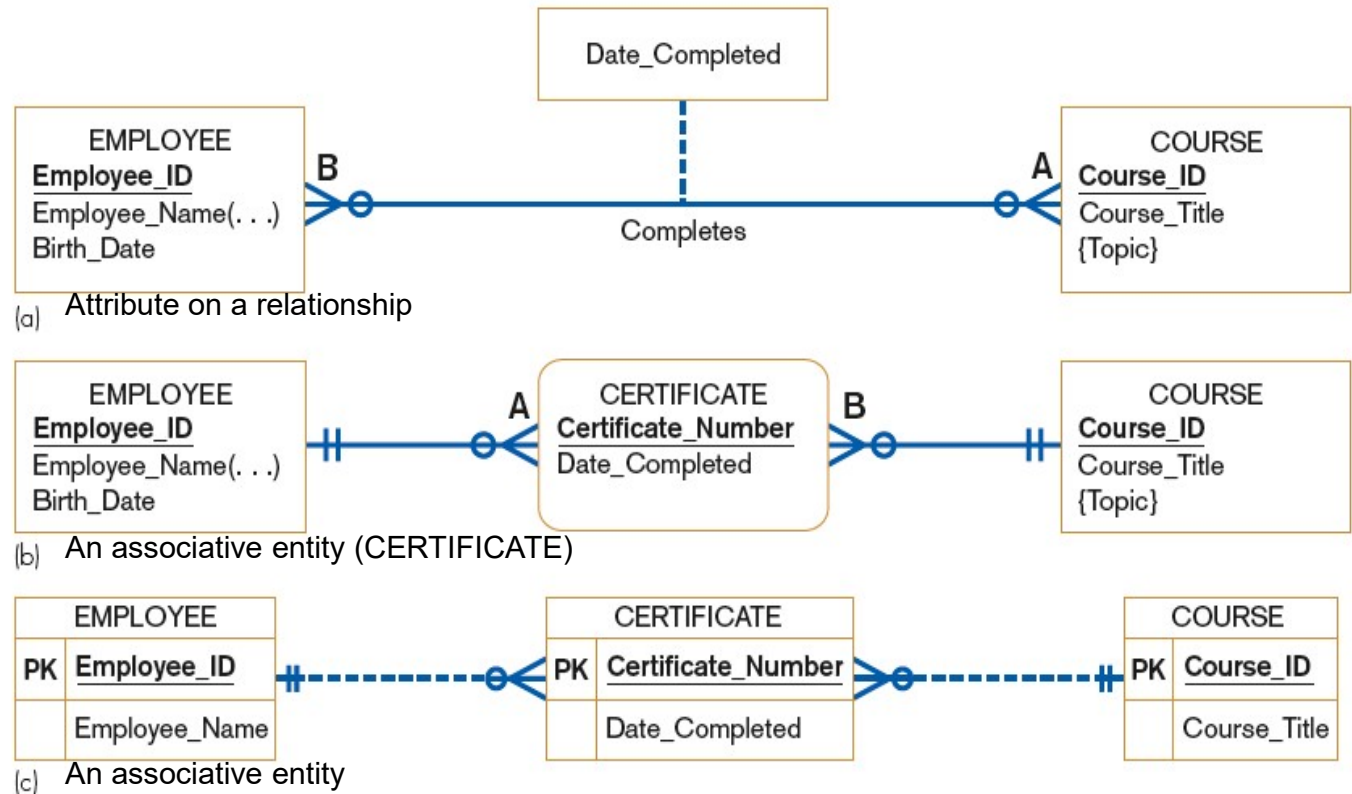


FIGURE : An associative entity

Representing Supertypes and Subtypes

- **Subtype:** a subgrouping of the entities in an entity type
 - Is meaningful to the organization
 - Shares common attributes or relationships distinct from other subgroupings
- **Supertype:** a generic entity type that has a relationship with one or more subtypes
- Business Rules for Supertype/subtype Relationships:
 - **Total specialization** specifies that each entity instance of the supertype must be a member of some subtype in the relationship.
 - **Partial specialization** specifies that an entity instance of the supertype does not have to belong to any subtype, and may or may not be an instance of one of the subtypes.
 - **Disjoint rule** specifies that if an entity instance of the supertype is a member of one subtype, it cannot simultaneously be a member of any other subtype.
 - **Overlap rule** specifies that an entity instance can simultaneously be a member of two (or more) subtypes.

- Disjoint and overlap combination**
- disjoint with partial specialization,
 - disjoint with total specialization
 - overlap with partial specialization, or
 - overlap with total specialization.

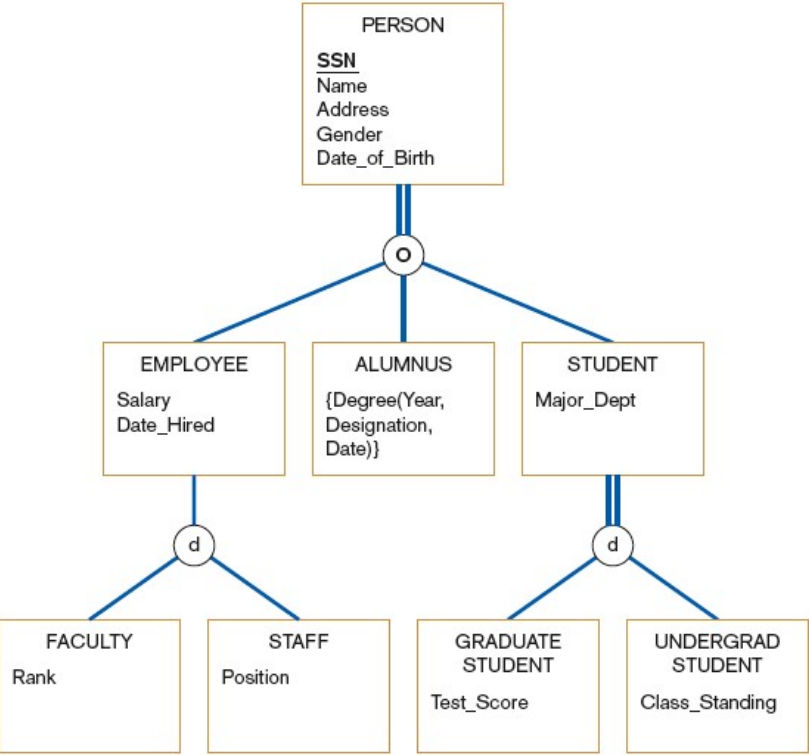


FIGURE :Example of super type/subtype hierarchy

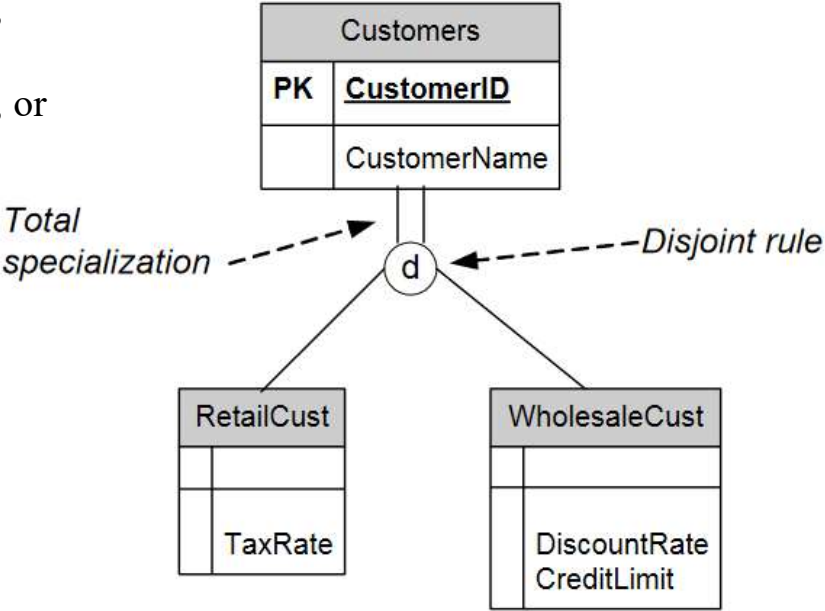


Figure 17: Supertype/Subtype Hierarchy

- In case of a disjoint subtype, which is indicated by the "d" in the circle underneath CUSTOMERS. This means that each super type can be only one subtype--it is not possible for one instance of CUSTOMERS to be both a RETAIL and a WHOLESALE customer.
- Sometimes subtypes follow the overlap rule, which means that an instance of the super type may be an instance of more than one subtype. When the overlap rule is in effect, an "o" is placed in the circle.

Unary Relationships

- Representing unary relationships in an ERD is done by drawing an ellipse to represent the relationship, and then connecting it to the entity twice.
- Note that maximum and minimum cardinality symbols are shown for the unary relationships.
- From Figure 20, we can tell that a given employee may manage many other employees, or may not manage any.
- We can also tell that each employee must be managed by exactly one employee.
- From the diagram of the PREREQ relationship we can tell that a given course can have many prerequisites or may have none, and that a course can be a prerequisite for many courses or may not be a prerequisite for any other courses.
- The type attribute is used to store the nature of the prerequisite (whether it's a prerequisite or a co-requisite).
- You do not have to have any attributes in a many-to-many unary relationship.

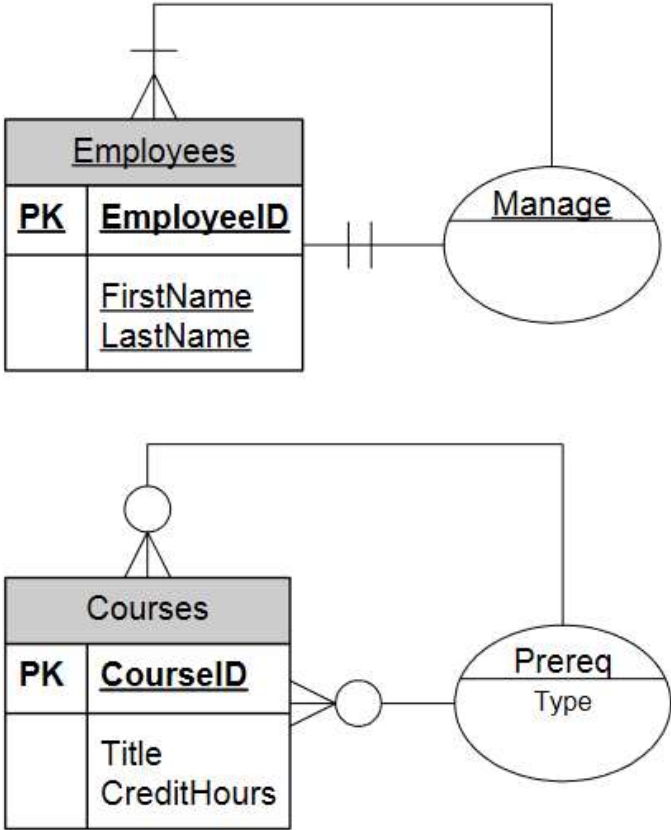
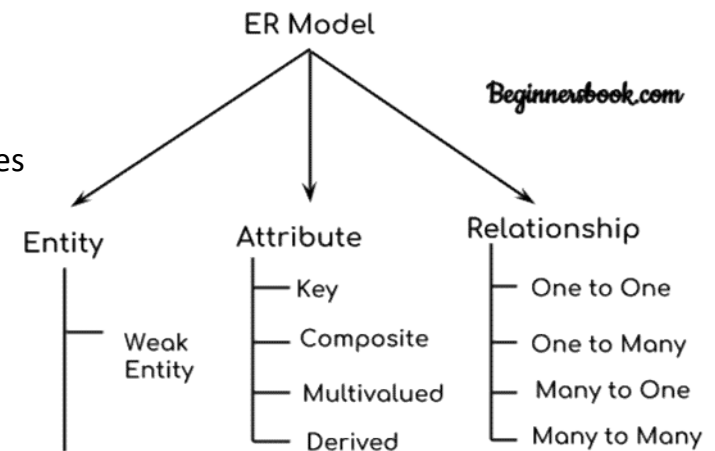


Figure 20 – Unary (recursive) Relationships

Some More Point/Ways on Designing ERD

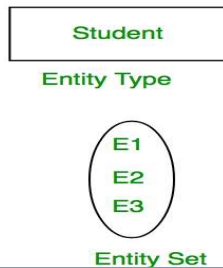
Entity Relationship Diagram(ERD)

- A detailed, logical representation of the entities, associations and data elements for an organization or business area.
- **ER Diagram** stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships.
- ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shapes to represent relationships.
- **Components of ERD**
 - **Rectangles:** This Entity Relationship Diagram symbol represents entity types
 - **Ellipses :** Symbol represent attributes
 - **Diamonds:** This symbol represents relationship types
 - **Lines:** It links attributes to entity types and entity types with other relationship types
 - **Primary key:** attributes are underlined
 - **Double Ellipses:** Represent multi-valued attributes



Components of ER Diagram

- An **Entity** may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence – a company, a job, or a university course.
- An **Entity** is an object of Entity Type and set of all entities is called as entity set. e.g.; E1 is an entity having Entity Type Student and set of all students is called Entity Set. Eg:

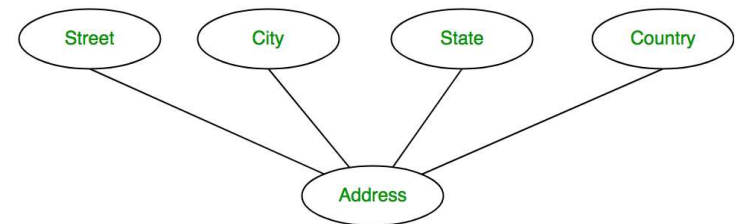


- **Attribute(s):**
Attributes are the **properties which define the entity type**. For example, Roll_No, Name, DOB, Age, Address, Mobile_No are the attributes which defines entity type Student. In ER diagram, attribute is represented by an oval.



Types of Attributes

- **Key Attribute**
 - The attribute which **uniquely identifies each entity** in the entity set is called key attribute. For example, Roll_No will be unique for each student. In ER diagram, key attribute is represented by an oval with underlying lines.
- **Composite Attribute**
 - An attribute **composed of many other attribute** is called as composite attribute. For example, Address attribute of student Entity type consists of Street, City, State, and Country. In ER diagram, composite attribute is represented by an oval comprising of ovals.



Multivalued Attribute

- An attribute consisting **more than one value** for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, multivalued attribute is represented by double oval.



Derived Attribute

- An attribute which can be **derived from other attributes** of the entity type is known as derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, derived attribute is represented by dashed oval.

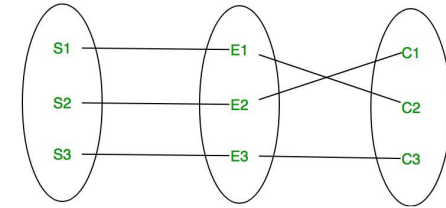


Relationship Type and Relationship Set:

- A relationship type represents the **association between entity types**. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.



A set of relationships of **same type** is known as **relationship set**. The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.

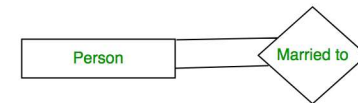


Degree of a relationship set:

The number of different entity sets **participating in a relationship** set is called as degree of a relationship set.

Unary Relationship

- When there is **only ONE entity set participating in a relation**, the relationship is called as unary relationship. For example, one person is married to only one person.



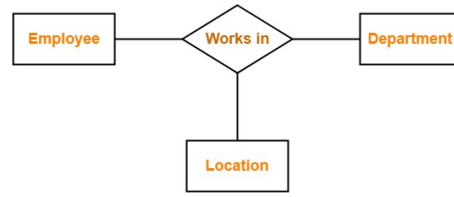
Binary Relationship

- When there are **TWO entities set participating in a relation**, the relationship is called as binary relationship. For example, Student is enrolled in Course



Ternary Relationships:

- Ternary relationship set is a relationship set where three entity sets participate in a relationship set.



Ternary Relationship Set

n-ary Relationship

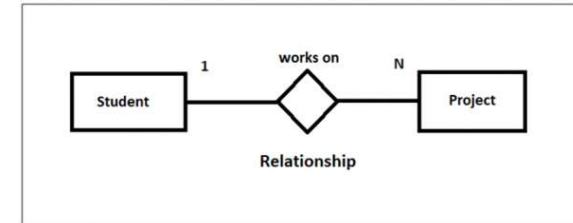
- When there are n entities set participating in a relation, the relationship is called as n-ary relationship.

Cardinality:

- The **number of times an entity of an entity set participates in a relationship** set is known as cardinality. Cardinality can be of different types:
- One to one** – When each entity in each entity set can take part **only once in the relationship**, the cardinality is one to one. Let us assume that a male can marry to one female and a female can marry to one male. So the relationship will be one to one.



- One to Many**-When each entity in each entity set can **take part more than once in the relationship** cardinality is one to many



- Many to one** – When entities in one entity set **can take part only once in the relationship set** and **entities in other entity set can take part more than once in the relationship set**, cardinality is many to one. Let us assume that a student can take only one course but one course can be taken by many students. So the cardinality will be n to 1. It means that for one course there can be n students but for one student, there will be only one course.



- Many to many** – When entities in all entity sets can **take part more than once in the relationship** cardinality is many to many. Let us assume that a student can take more than one course and one course can be taken by many students. So the relationship will be many to many.



Participation Constraint:

Participation Constraint is applied on the entity participating in the relationship set.

Total Participation –

- It specifies that each entity in the entity set must compulsorily participate in at least one relationship instance in that relationship set.
- That is why, it is also called as **mandatory participation**.
- Total participation is represented using a double line between the entity set and relationship set.



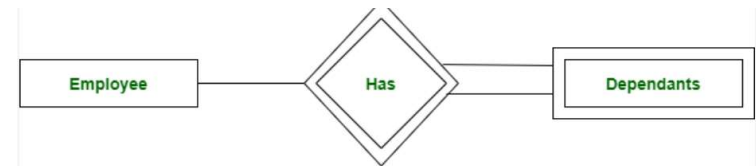
Partial Participation –

- It specifies that each entity in the entity set may or may not participate in the relationship instance in that relationship set.
- That is why, it is also called as **optional participation**.
- Partial participation is represented using a single line between the entity set and relationship set.



Weak Entity Type and Identifying Relationship:

weak entity type is represented by a double rectangle. The participation of weak entity type is always total. The relationship between weak entity type and its identifying strong entity type is called identifying relationship and it is represented by double diamond.



How to Create ERD(Designing concept on ERD)

- 1.Identification and listing of the Entities exists in the System
- 2.Identification and listing of relationships exists within the system
- 3.Identification and assignment of the cardinality exists in the system
- 4.Identification of the attributes exists in the system and assign them with proper symbol example primary key attribute,composite key attributes etc.

- 5.Create the ER diagram on the basis of above identifications.

- Example case:Create ERD for following case

In a university, a **Student** enrolls in **Courses**.

A student must be assigned to at least one or more Courses.

Each course is taught by a single **Professor**.

To maintain instruction quality, a Professor can deliver only one course

• Solutions:

- 1.List of Entities:Student,Course,Professor

- 2.List of Relationships:

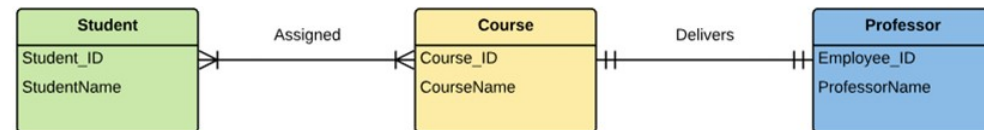
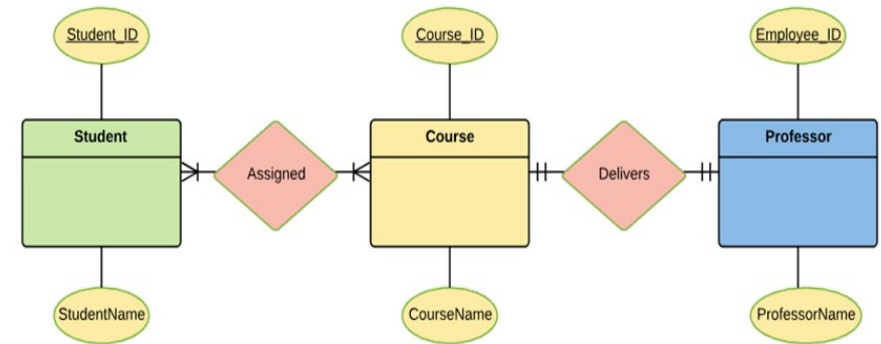
- student **assigned** to course.
- A professor can **deliver** only one course.

- 3.Cardinalities:

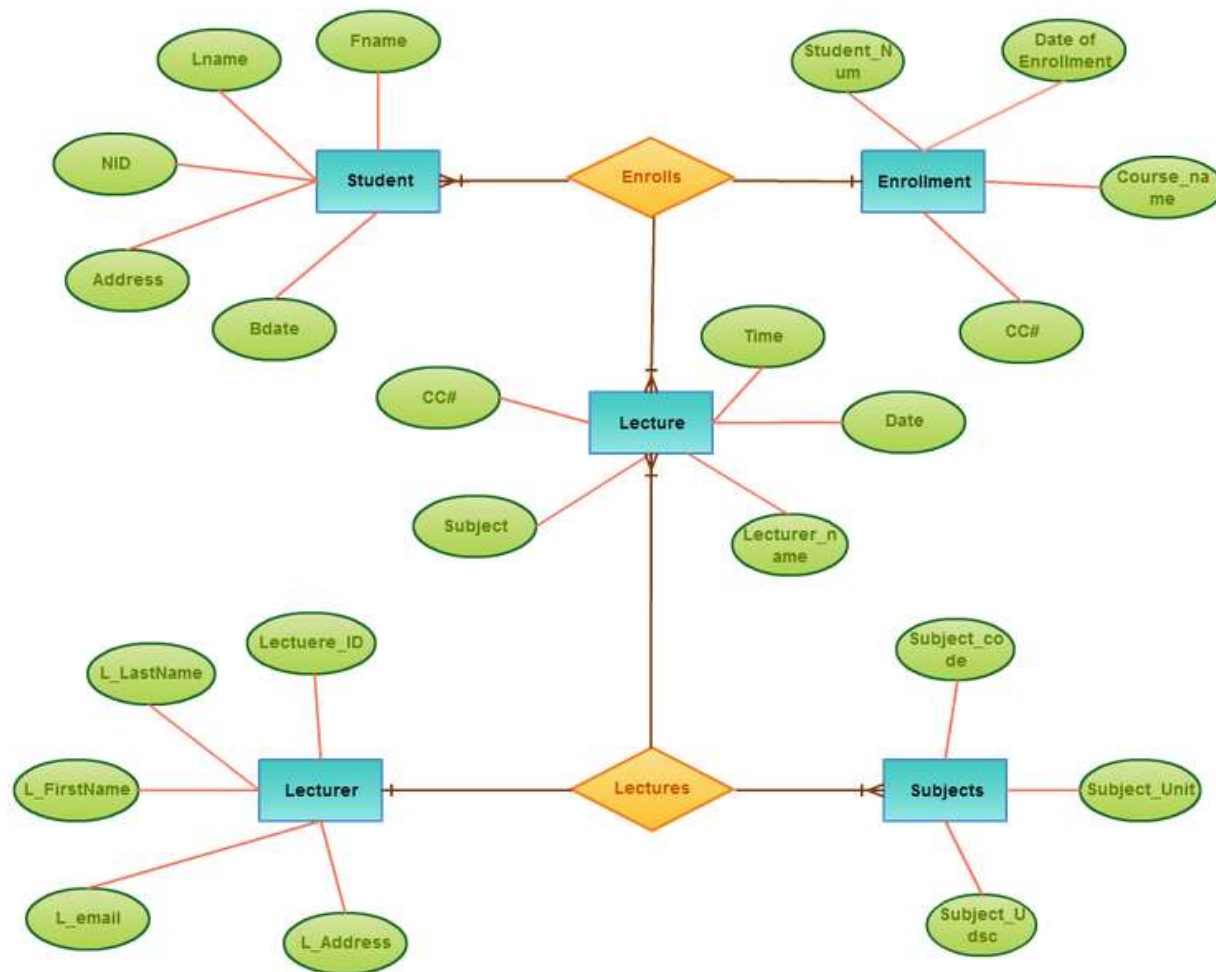
- A student can be assigned **multiple** courses
- A Professor can deliver only **one** course

- 4.Attributes Identification: student,professor and Course can be characterized by their name,id etc.

- 5.Prepare the ERD



ER DIAGRAM FOR STUDENT ENROLLMENT SYSTEM



More Examples on DFD

Case study:

The existing system in any medical shop is entirely manual. The shop gets its products from a dealer who in turn may produce them from a higher level dealer or directly from the manufacturer. It keeps track of all available medicines. When a customer approaches for a particular medicine, the stock is checked to verify the availability of the product and if it is available, then it is supplied to the customer who is billed for it.

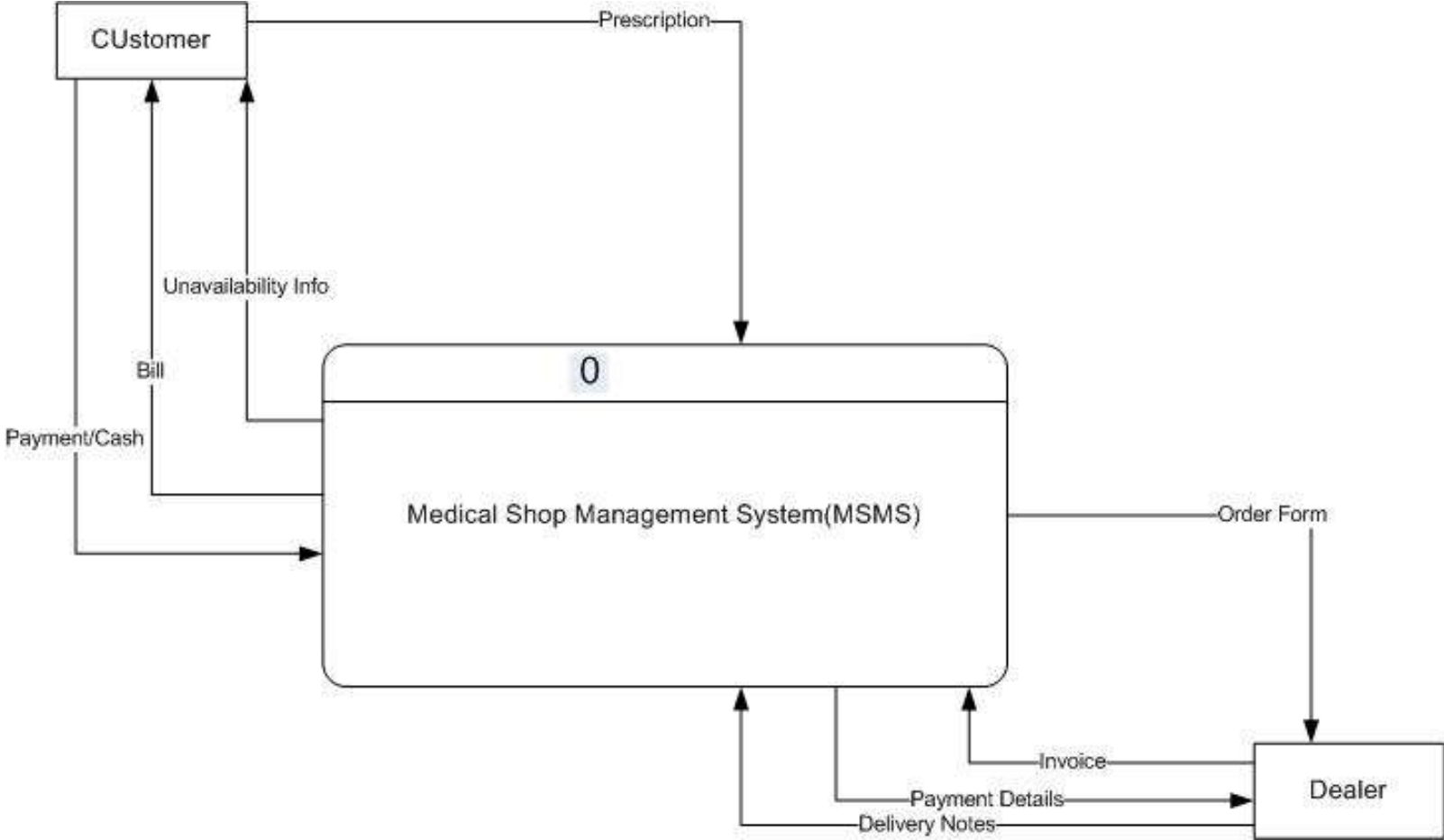
The system suffers from all the drawbacks that a manual system suffers i.e it is highly employee dependent. Right from the purchase of products from the dealers to selling it to the customer and other administrative functions are performed manually. Accounting and billing is also manual. Manual inventory suffers from the drawbacks that the shopkeeper has to track the status of the stock of many products. If the stock is diminished.

An automated system would eliminate the above drawbacks. Apart from computerizing all the manual tasks that are performed in the shop, it can also keep a track of the inventories and when the stock of a particular product reaches below a predetermined level. It can alert some person or even automatically order it in a requisite quantity.

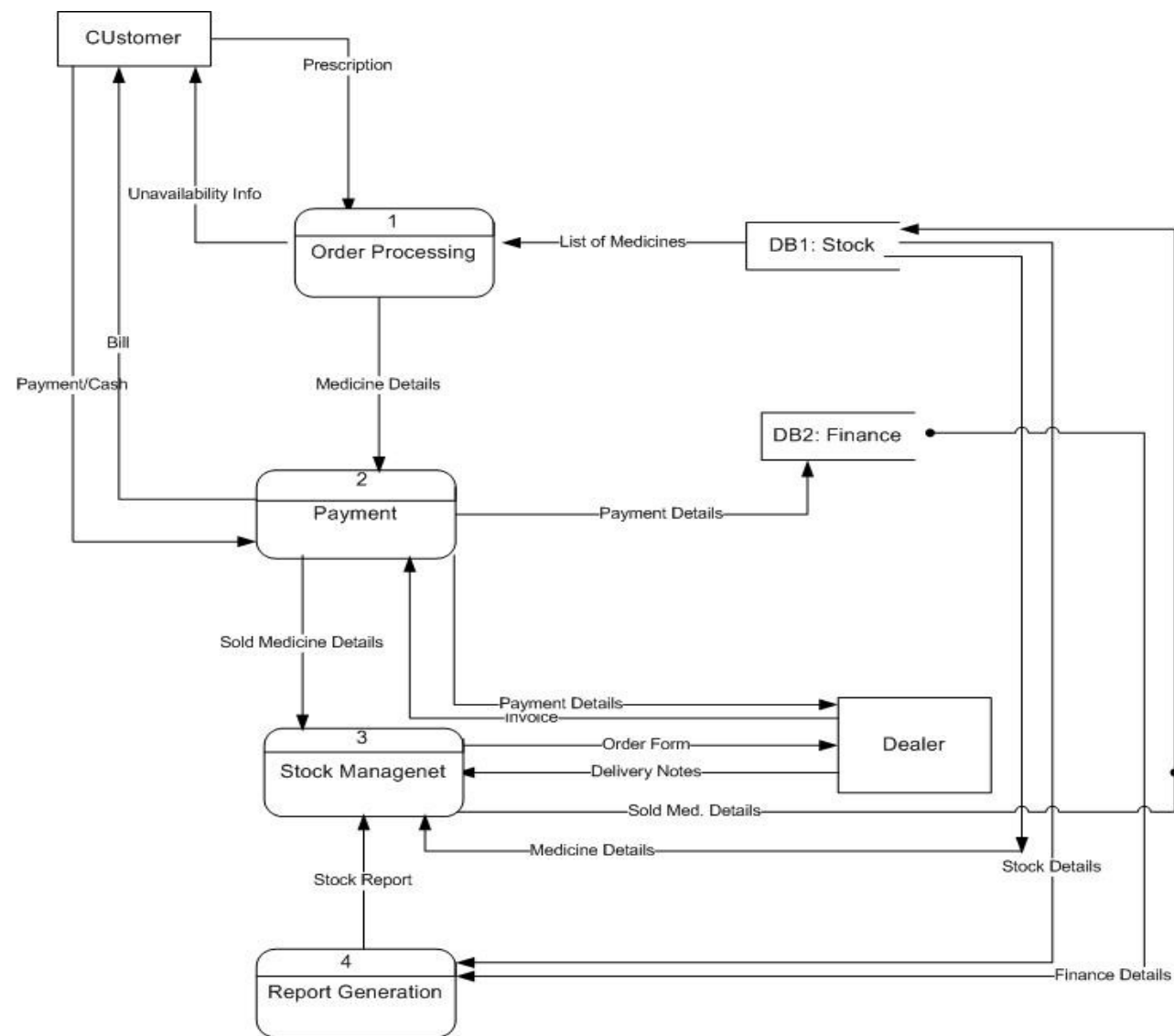
1. Construct Context Diagram and level 1 DFD

2. Construct level 2 DFD for one key process

Context Diagram



Level 0 DFD

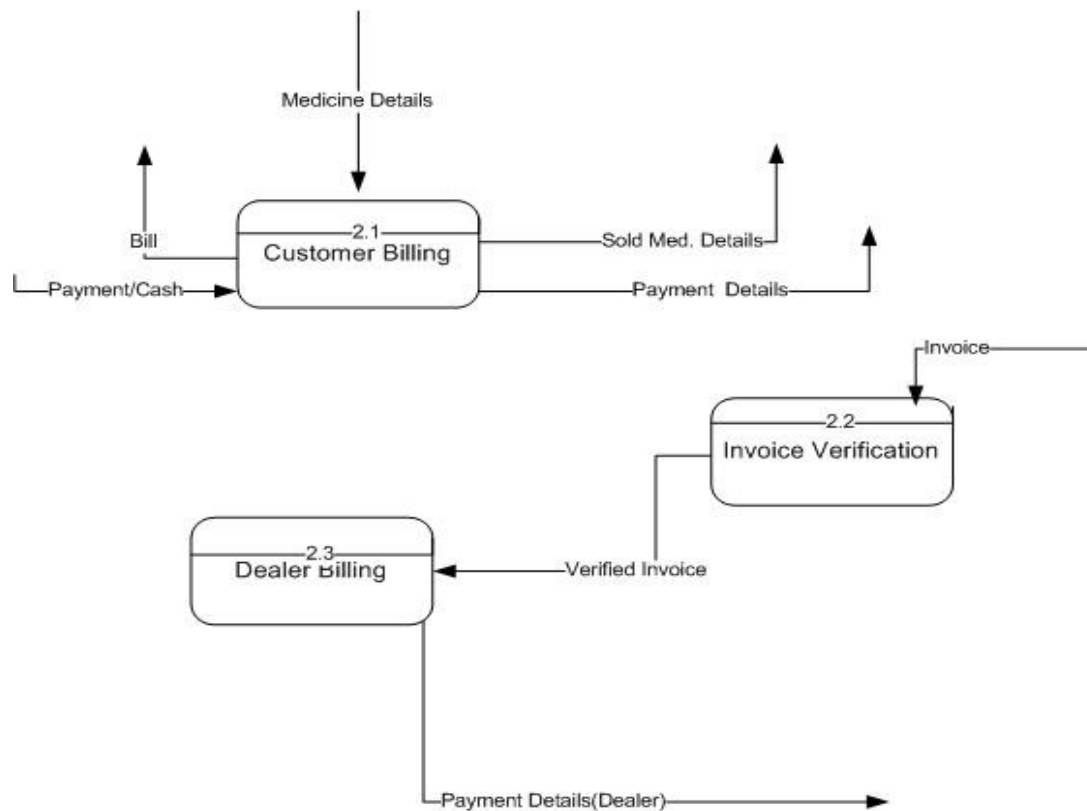


Level 1 DFD

for Process no 2(Payment Process) : Customer Payment(Billing) , Dealer Payment(Billing), Verification of Dealer Payment(Invoice Verification)

Note: In Level 1 or higher level DFD generally Entities and Data Store are not represented.

Two arrows with same name should consider one input or output while balancing DFD.



Identify Entities, Process, Data Stores & Data Flow

- **Case study : Order Processing System**

- Precision Tools sells a line of high-quality woodworking tools. When customers place orders on the company's Web site, the system checks to see if the items are in stock, issues a status message to the customer, and generates a shipping order to the warehouse, which fills the order. When the order is shipped, the customer is billed. The system also produces various reports.
 1. Draw a context diagram for the order system
- Draw DFD diagram 1 for the order system

Order Processing System

Identify Entities, Process, Data Stores & Data Flow

■ Entities

- ☐ Customer
- ☐ Warehouse
- ☐ Accounting

■ Processes

- ☐ 1.0 Check Status
- ☐ 2.0 Issue Status Messages
- ☐ 3.0 Generate Shipping Order
- ☐ 4.0 Manage Accounts Receivable
- ☐ 5.0 Produce Reports

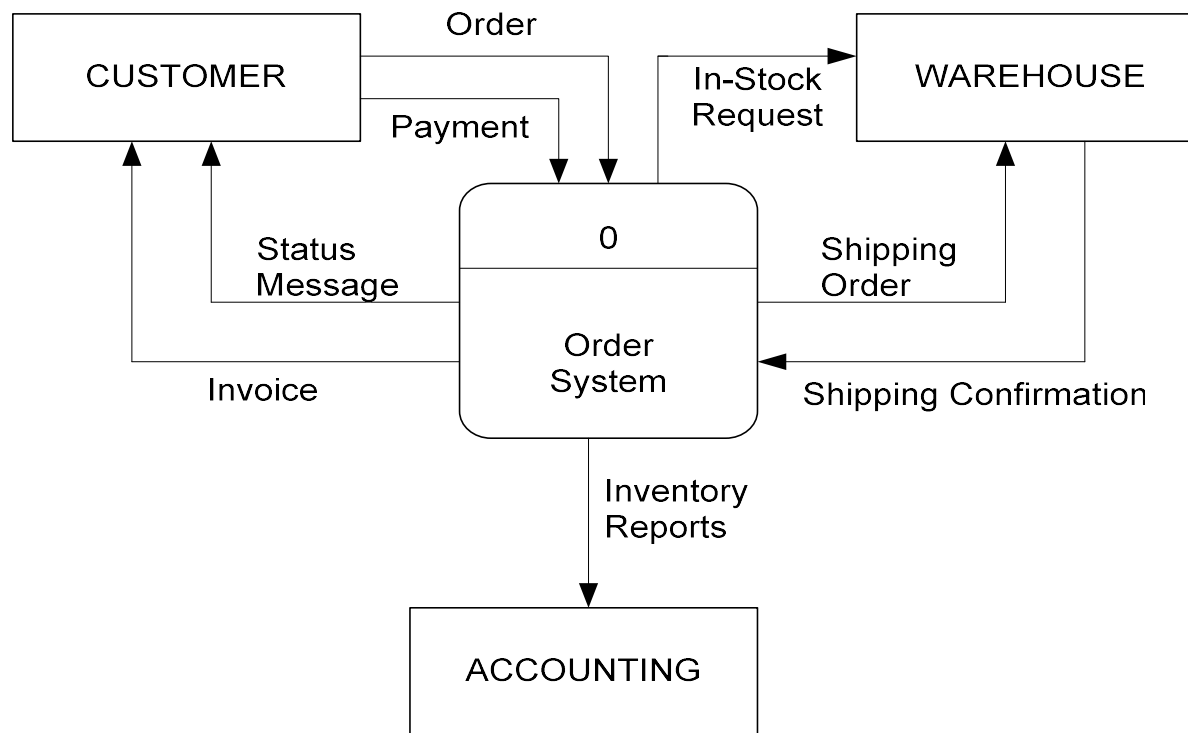
■ Data Stores

- ☐ D1 Pending Orders
- ☐ D2 Accounts Receivable

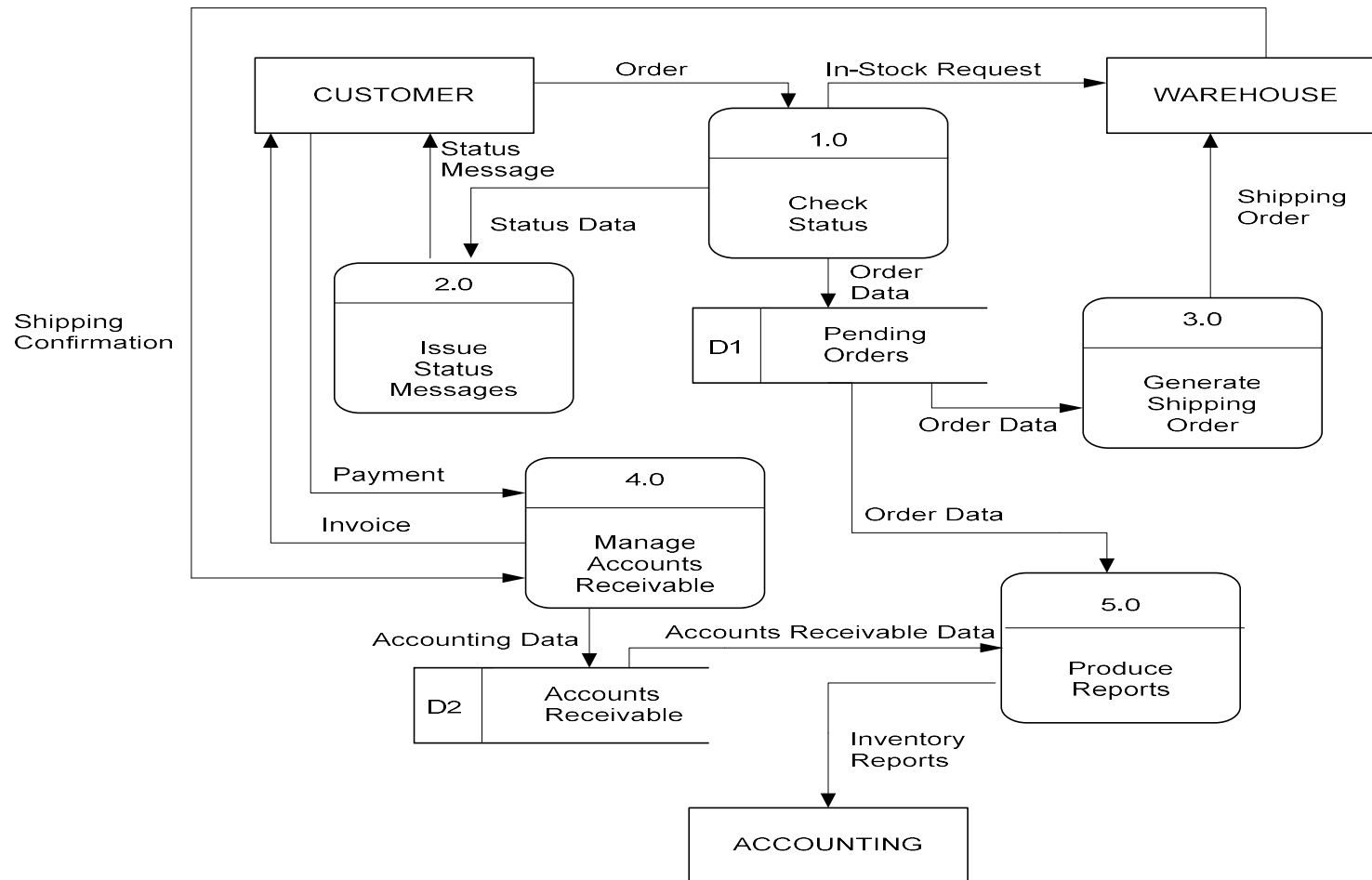
■ Data Flows

- ☐ Order
- ☐ In-Stock Request
- ☐ Order Data
- ☐ Status Data
- ☐ Status Message
- ☐ Shipping Order
- ☐ Order Data
- ☐ Invoice
- ☐ Shipping Confirmation
- ☐ Payment
- ☐ Accounting Data
- ☐ Accounts Receivable Data
- ☐ Order Data
- ☐ Inventory Reports

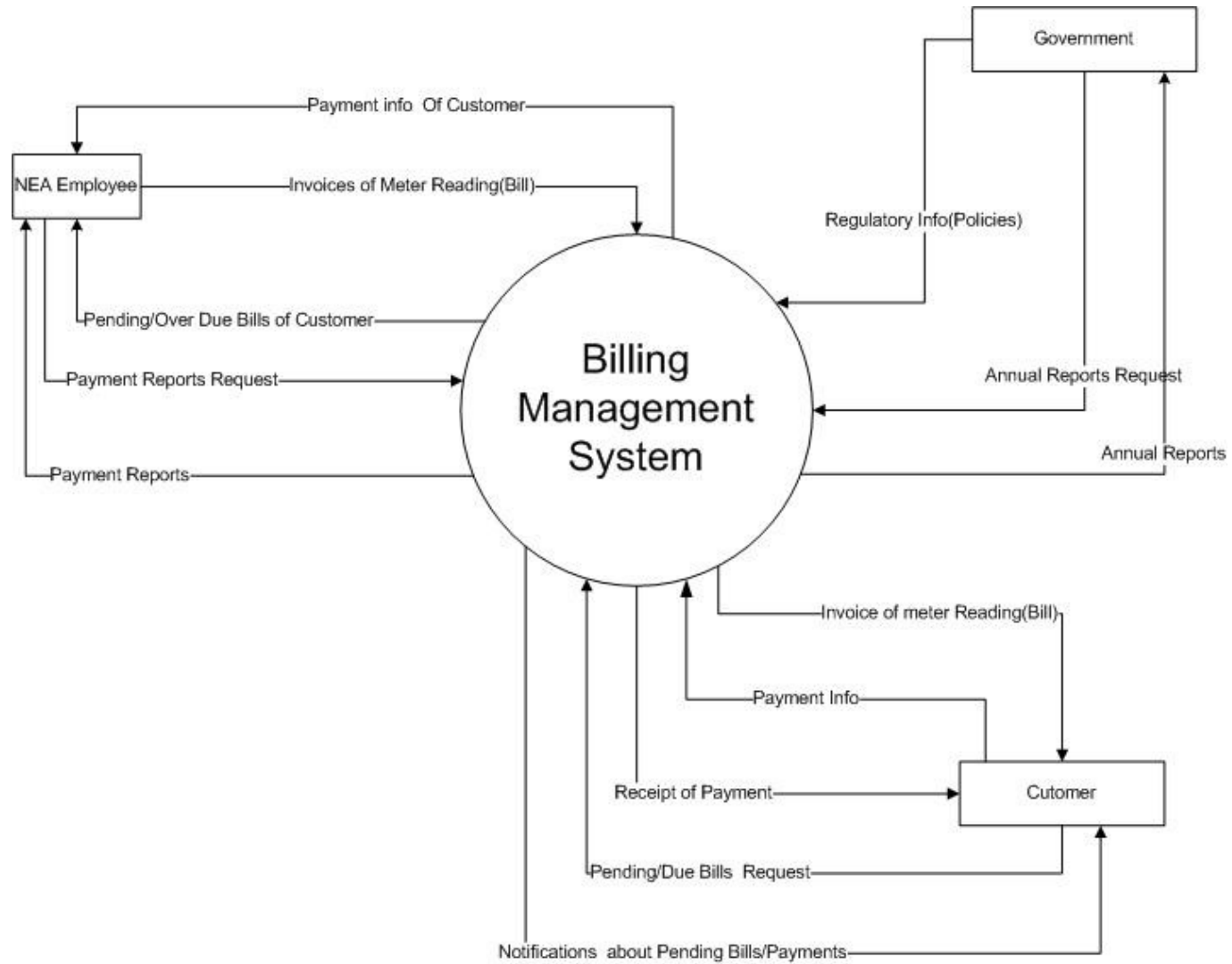
Context Diagram



Level 0 DFD

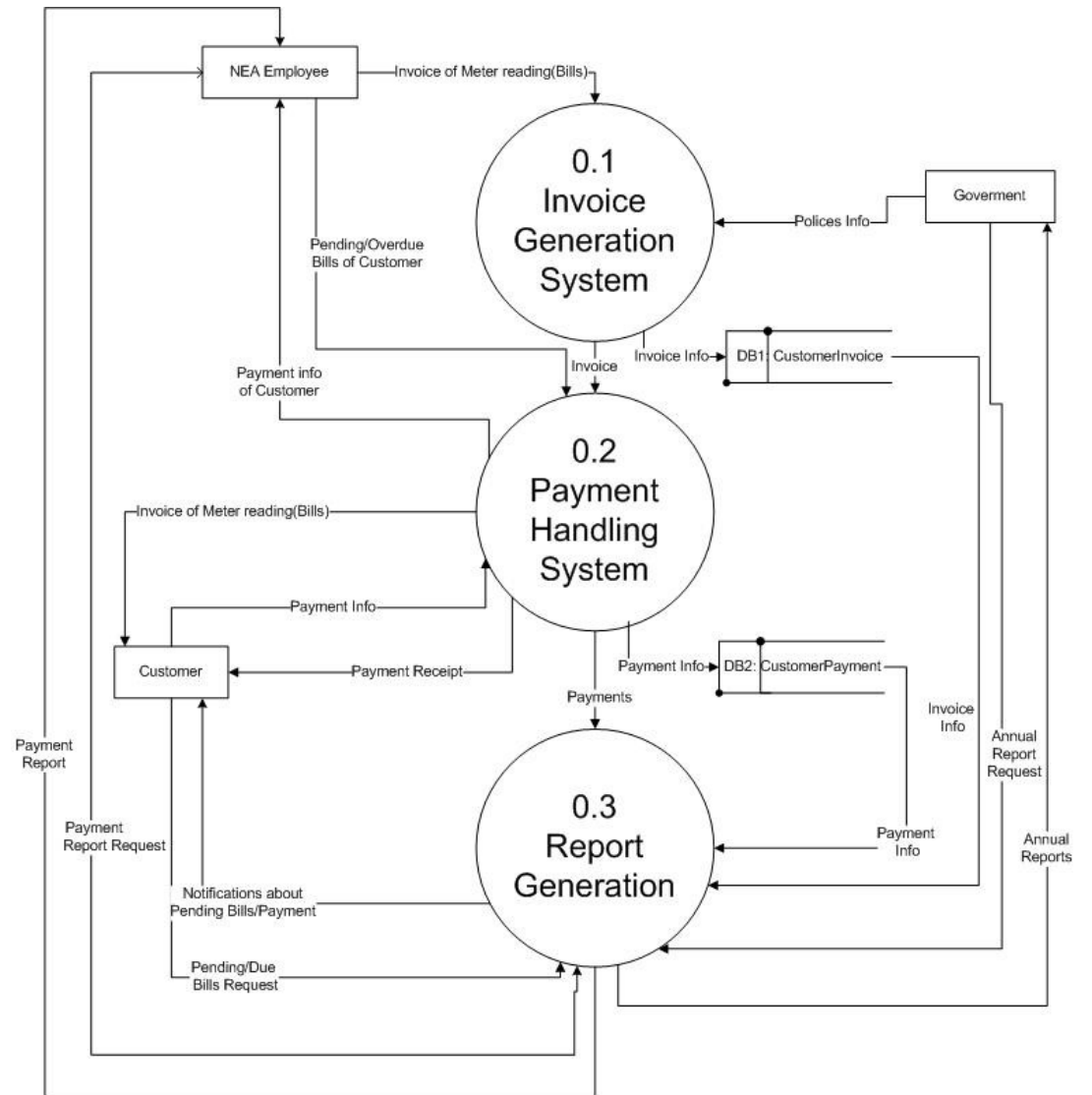


NEA Billing System
DFD : Context Diagram



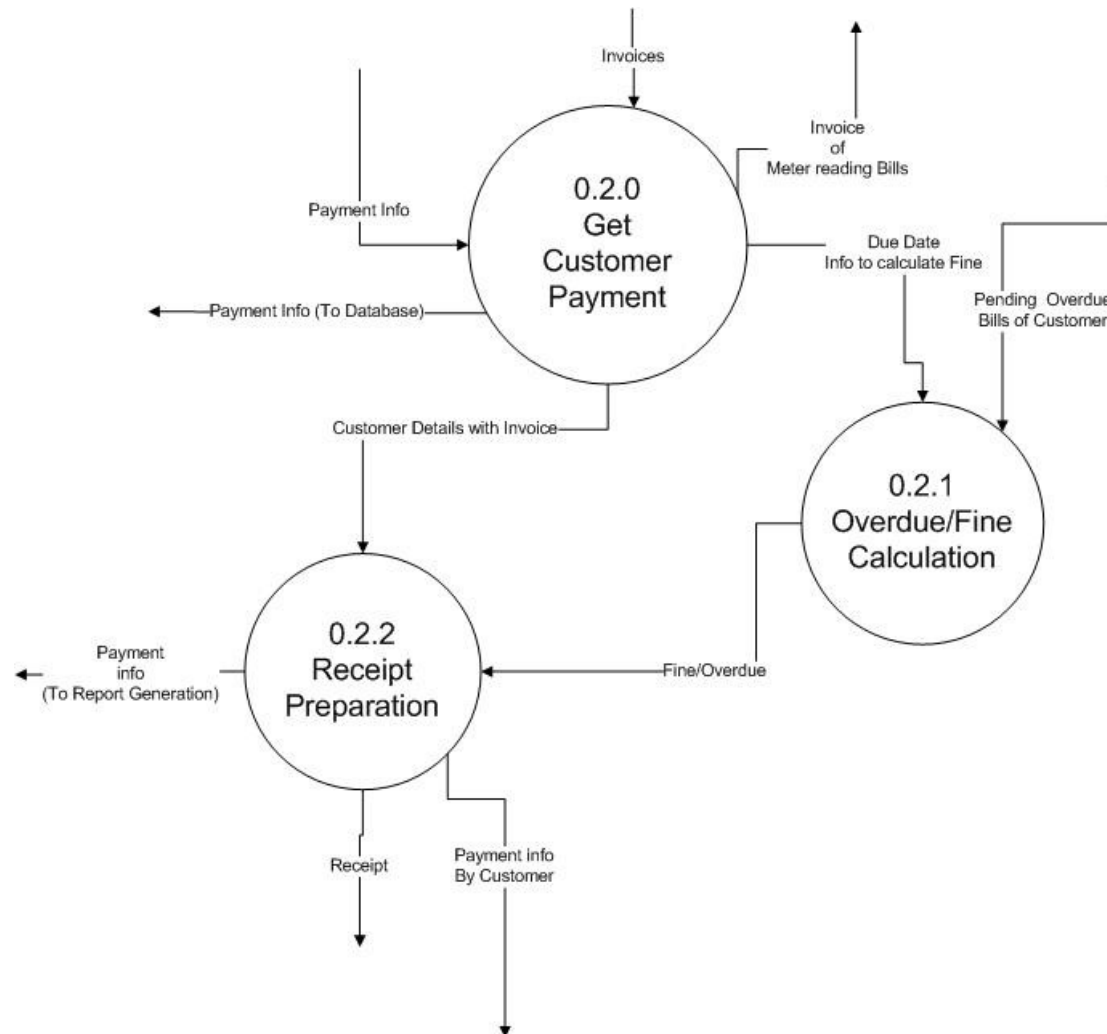
NEA Billing System

DFD : Level 0



NEA Billing System

DFD : Level 1



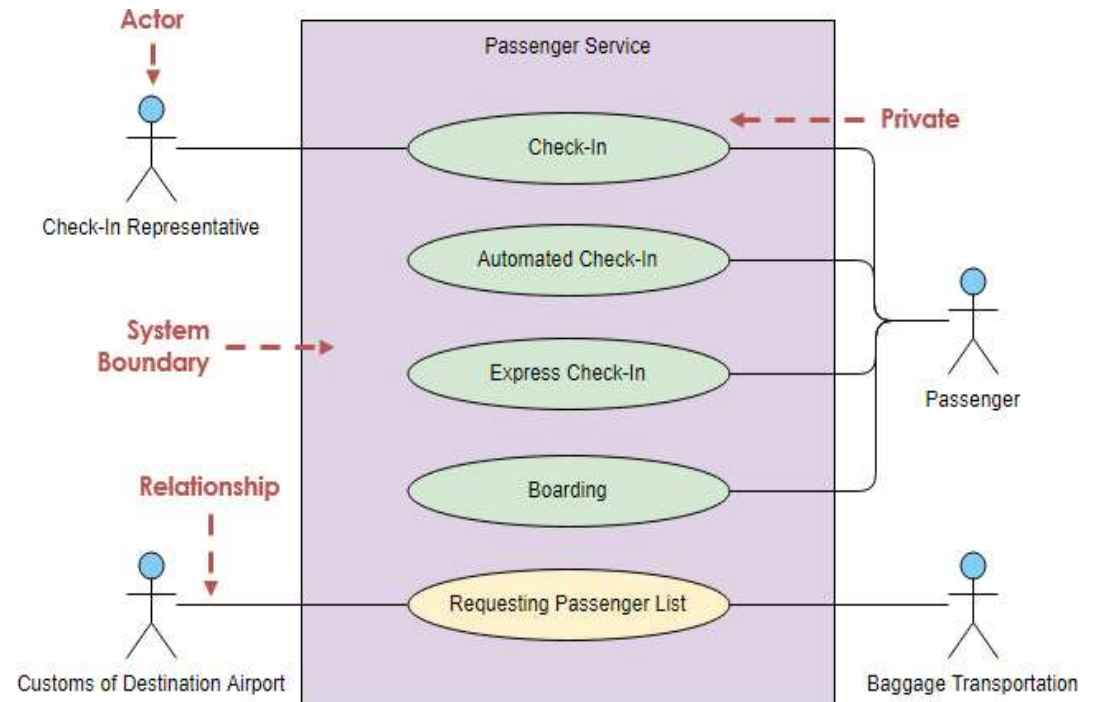
More On Use Case

Use Case Modeling

- Use Case is the Depiction of a system's behavior or functionality under various conditions as the system responds to requests from users
- A use case describes how a user uses a system to accomplish a particular goal.
- A use case diagram consists of the system, the related use cases and actors and relates these to each other to visualize: what is being described? (**system**), who is using the system? (**actors**) and what do the actors want to achieve? (**use cases**), thus, use cases help ensure that the correct system is developed by capturing the requirements from the user's point of view.
- A use-case diagram can help provide a higher-level view of the system, providing the simplified and graphical representation of what the system must actually do.
- A use case (or set of use cases) has these **characteristics**:
 - Organizes functional requirements
 - Models the goals of system/actor (user) interactions
 - Describes one main flow of events (main scenarios) and possibly other exceptional flows (alternatives), also called paths or user scenarios
- **Purpose of Use Case Diagram**
 - Specify the context of a system
 - Capture the requirements of a system
 - Validate a systems architecture
 - Drive implementation and generate test cases
 - Developed by analysts together with domain experts

• A use case diagram notations

- Use cases
- Actor
- Use Case relationships
- System Boundary



Use Case Components

- **Actor**

- Actor is an external entity that interacts with the system.
- Most actors represent user roles, but actors can also be external systems.
- An actor is a role, not a specific user; one user may play many roles, and an actor may represent many users.
- An Actor is outside or external the system.
- It can be a:
 - Human
 - Peripheral device (hardware)
 - External system or subsystem
 - Time or time-based event
- Represented by stick figure

- **Boundary**

- A boundary is the dividing line between the system and its environment.
- Use cases are within the boundary.
- Actors are outside of the boundary.

- **Cases:**

- A use case describes how actors uses a system to accomplish a particular goal
- Use cases are typically initiated by a user to fulfill goals describing the activities and variants involved in attaining the goal.
- Document containing detailed specifications for a use case
- Contents can be written as simple text or in a specified format

- **Relationships**

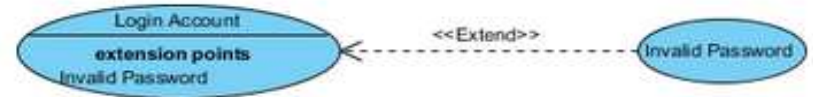
- **Association**

- Associations are used to describe the relationships between actors and the use cases they participate in.
- This relationship is commonly known as a “communicates-association”.
- Represented by solid line



- **Extends**

- <<extend>> is used to include optional behavior from an extending use case in an extended use case.



- Indicate that an "**Invalid Password**" use case may include (subject to specified in the extension) the behavior specified by base use case "**Login Account**".
- Depict with a directed arrow having a dotted line. The tip of arrowhead points to the base use case and the child use case is connected at the base of the arrow.
- The stereotype "<<extends>>" identifies as an extend relationship

Use Case Relationships

- **Includes**

- `<<include>>` is used to include common behavior from an included use case into a base use case in order to support re-use of common behavior.
- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.
- A use case includes the functionality described in another use case as a part of its business process flow.
- An include relationship is depicted with a directed arrow having a dotted line. The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow.
- The stereotype "`<<include>>`" identifies the relationship as an include relationship.



- **Inheritance(Generalization/Specialization)**

- A generalization relationship is a parent-child relationship between use cases.
- The child use case is an enhancement of the parent use case.
- Generalization is shown as a directed arrow with a triangle arrowhead.
- The child use case is connected at the base of the arrow. The tip of the arrow is connected to the parent use case.
- A generalization relationship means that a child use case inherits the behavior and meaning of the parent use case.
- The child may add or override the behavior of the parent.

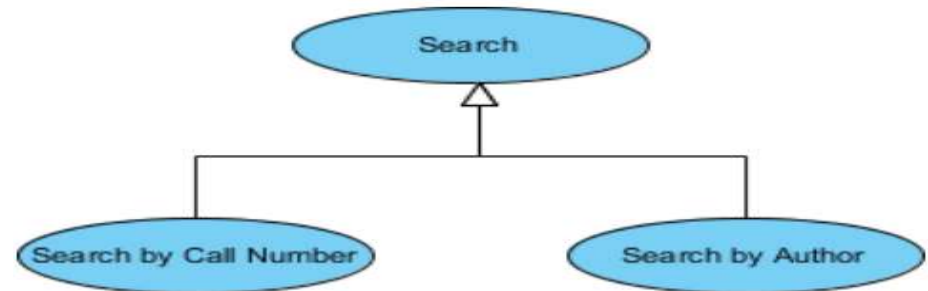
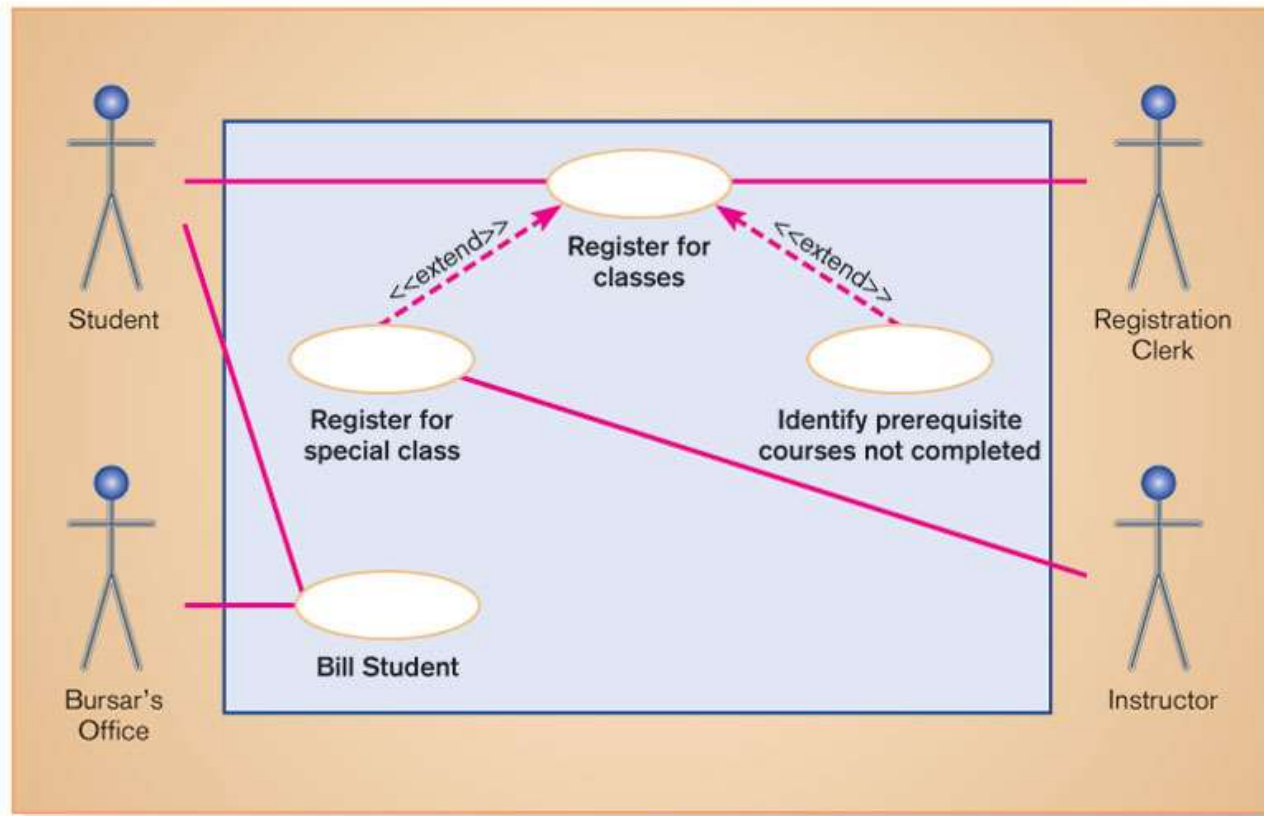


Figure 7-22 A use case diagram for a university registration system

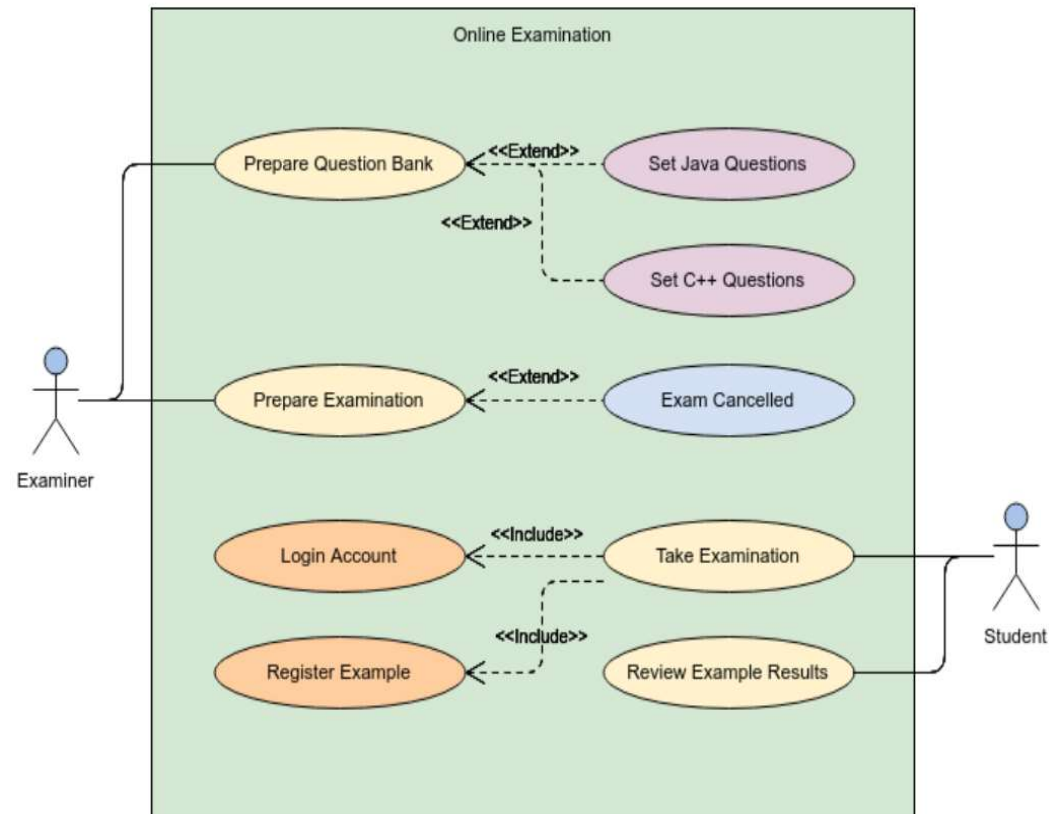


Guidelines to draw Use Case Diagram

- Identify the Actors (role of users) of the system.
- For each category of users, identify all roles played by the users relevant to the system. Identify the cases exists in the system.
- Create use cases for every goal and assign the different types of relationships as per needed.
- Structure the use cases.i.e organize different actors, cases and cases relationships.
- Prioritize, review, estimate and validate the users.

Advantages of Use case diagram

- Use cases is a powerful technique for the elicitation and documentation of functional requirements.
- Because, use cases are easy to understand and provide an excellent way for communicating with customers and users as they are written in natural language.
- Use cases can help manage the complexity of large projects by partitioning the problem into major user features (i.e., use cases) and by specifying applications from the users' perspective.
- A use case scenario, often represented by a sequence diagram, involves the collaboration of multiple objects and classes, use cases help identify the messages (operations and the information or data required — parameters) that glue the objects and classes together.
- Use cases provide a good basis to link between the verification of the higher-level models (i.e. interaction between actors and a set of collaborative objects), and subsequently, for the validation of the functional requirements (i.e. blueprint of white-box test).
- Use case driven approach provides an traceable links for project tracking in which the key development activities such as the use cases implemented, tested, and delivered fulfilling the goals and objectives from the user point of views.



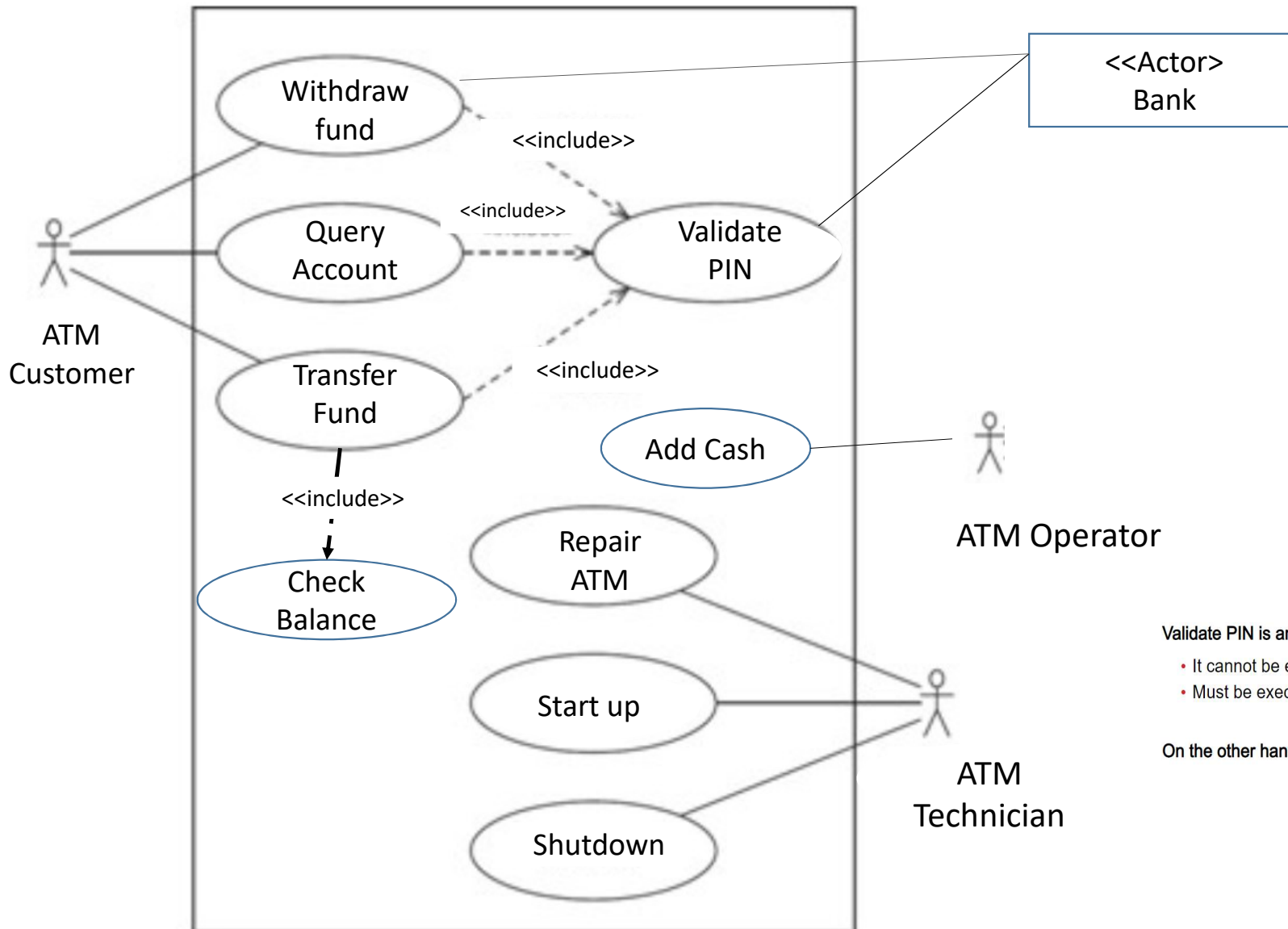
USE CASE ATM EXAMPLE (Complete example)

Actors:

- ATM Customer
- ATM Operator
- ATM Technician
- Bank

Use Cases:

- *The customer can*
 - withdraw funds from a checking or savings account
 - query the balance of the account
 - transfer funds from one account to another
- *The ATM operator can*
 - Replenish/load the ATM cash dispenser
- *The ATM Technician can*
 - Shut down the ATM
 - Start the ATM
 - Repair ATM



Validate PIN is an *Inclusion Use Case*

- It cannot be executed on its own
- Must be executed as part of a *Concrete Use Case*

On the other hand, a *Concrete Use Case* can be executed

Fully Dressed Use case description for Validate PIN Use case

USE CASE No: 1

Use case name: Validate PIN

Description: System validates customer PIN

Actor: ATM Customer

Precondition: ATM is idle, displaying a Welcome message.

Postcondition: Customer PIN has been validated.

Main sequence(Primary Flow of events):

1. Customer inserts the ATM card into the card reader.
2. If system recognizes the card, it reads the card number.
3. System prompts customer for PIN.
4. Customer enters PIN.
5. System checks the card's expiration date and whether the card has been reported as lost or stolen.
6. If card is valid, system then checks whether the user-entered PIN matches the card PIN maintained by the system.
7. If PIN numbers match, system checks what accounts are accessible with the ATM card.
8. System displays customer accounts and prompts customer for transaction type: withdrawal, query, or transfer.

Fully Dressed Use case description for Validate PIN Use case(contd..)

Alternative sequences/flow:

- Step 2: If the system does not recognize the card, the system ejects the card. Step 5: If the system determines that the card date has expired, the system confiscates the card.
- Step 5: If the system determines that the card has been reported lost or stolen, the system confiscates the card.
- Step 7: If the customer-entered PIN does not match the PIN number for this card, the system re-prompts for the PIN.
- Step 7: If the customer enters the incorrect PIN three times, the system confiscates the card.
- Steps 4-8: If the customer enters Cancel, the system cancels the transaction and ejects the card.

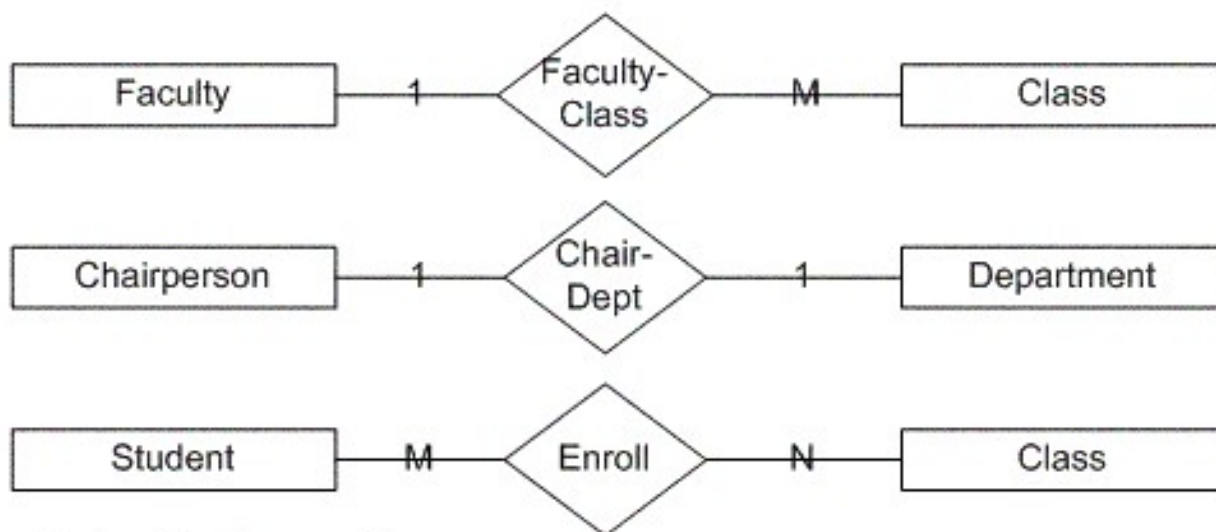
Frequency of Occurrence:

- Very frequent - Every Time a customer interact with ATM system

More on ERD

Mapping Cardinality:

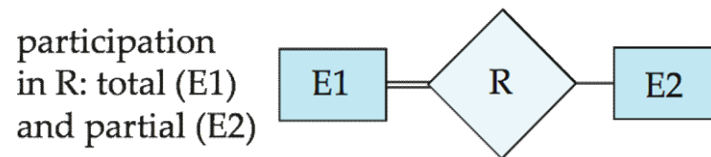
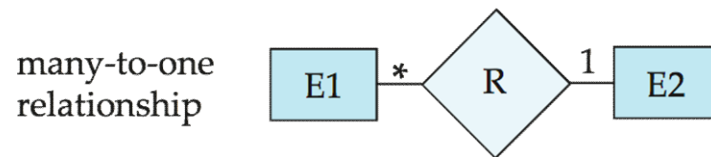
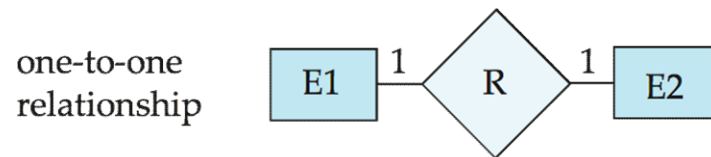
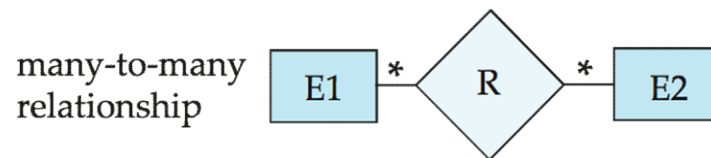
- The **number of times an entity of an entity set participates in a relationship set** is known as cardinality.
- Express the number of entities to which another entity can be associated via a relationship set.
- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line (—), signifying “many,” between the relationship set and the entity set.
- Cardinality can be of different types:
 - One to one
 - One to many
 - Many to one
 - Many to many



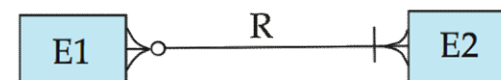
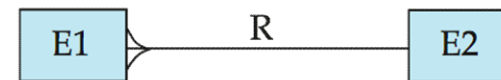
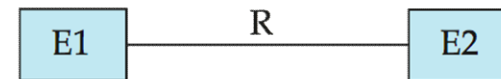
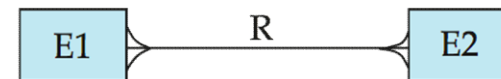
Method 1: One: 1
Many M,N

Alternative ER Notations

Chen



IDE1FX (Crows feet notation)



SYMBOL	MEANING	UML REPRESENTATION
	One and only one	1
	One or many	1..*
	Zero, or one, or many	0..*
	Zero, or one	0..1

EXAMPLES



One and only one CUSTOMER can place anywhere from zero to many of the ORDER entity.



One and only one ORDER can include one ITEM ORDERED or many.

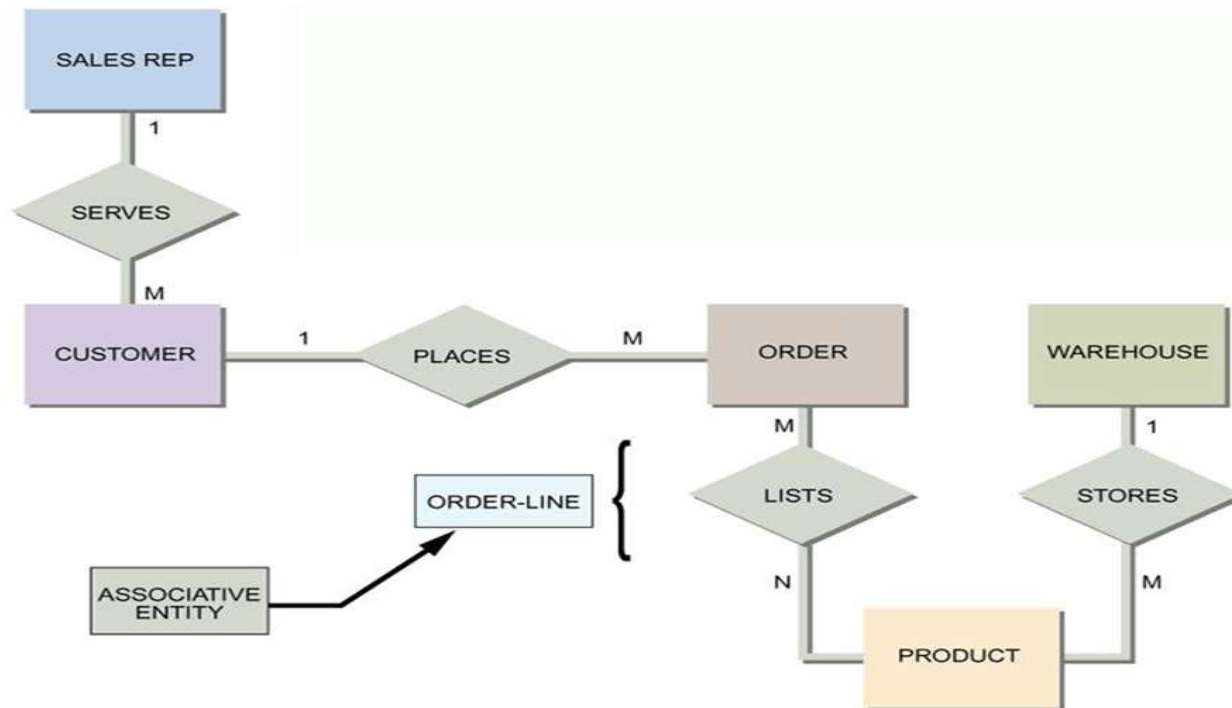


One and only one EMPLOYEE can have one SPOUSE or NONE.

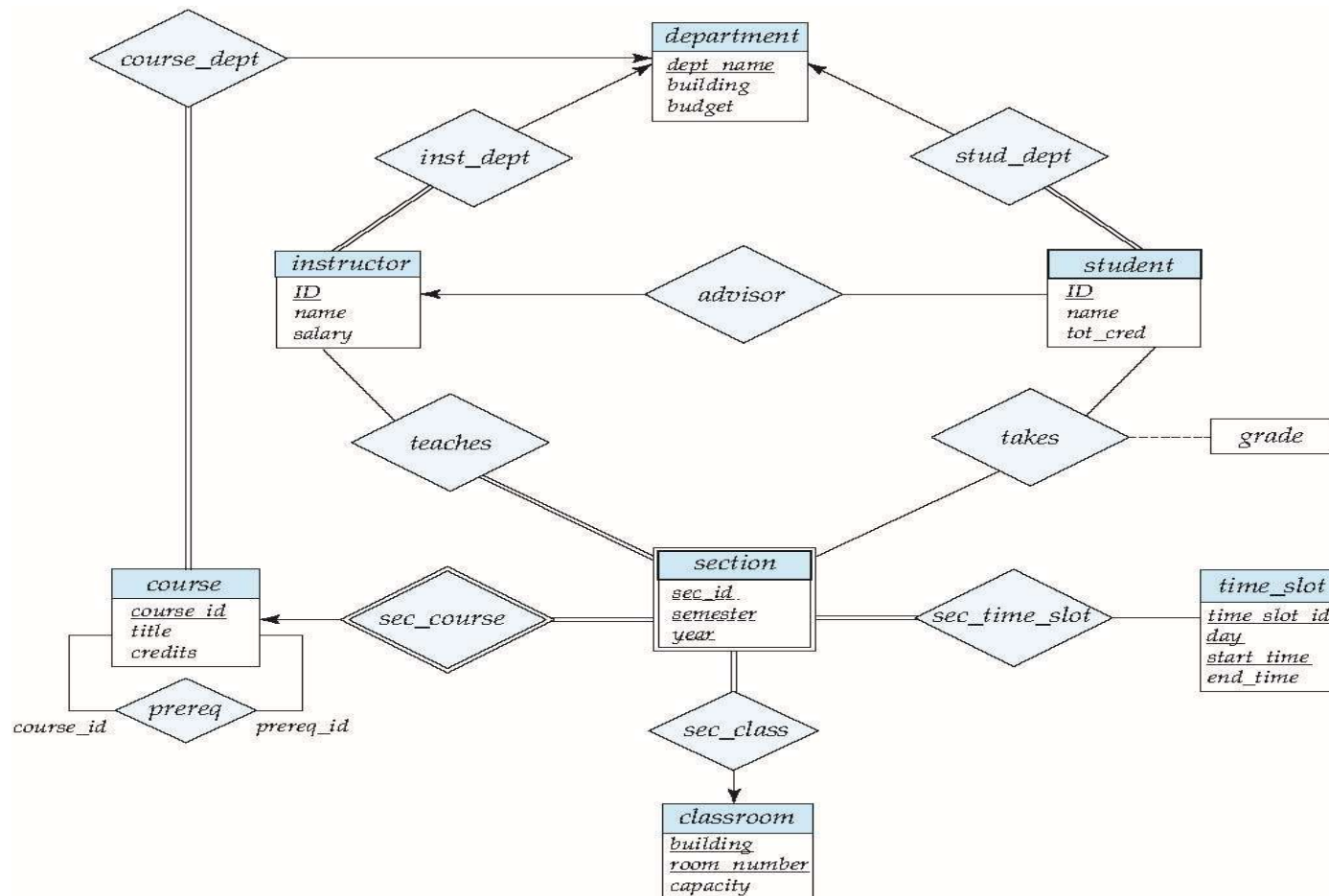


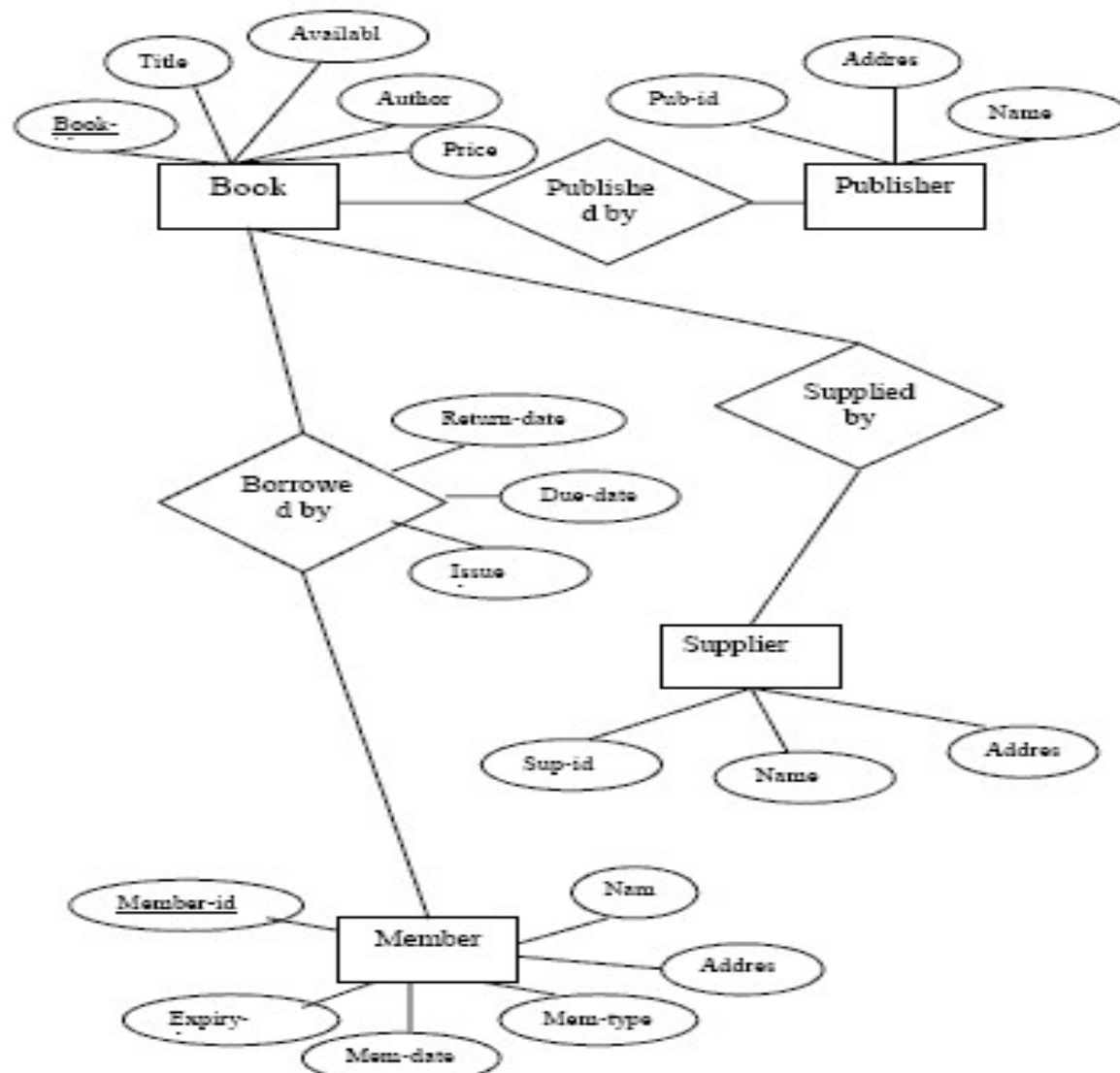
One EMPLOYEE, or many employees, or none, can be assigned to one PROJECT, or many projects, or none.

ERD for Order System



ERD for University Database





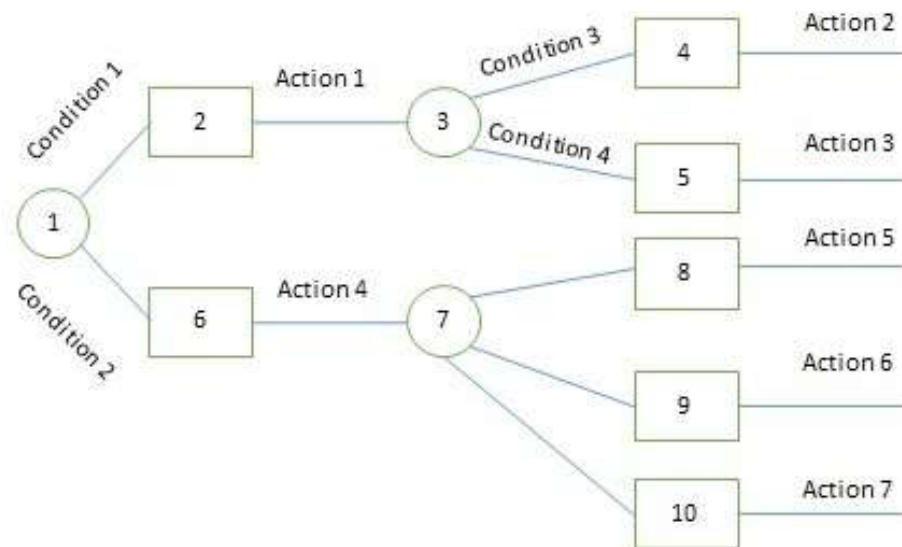
Identify the Entities in following Case study

- A university may have some departments. All these departments employ various lecturers and offer several programs.
- Some courses make up each program. Students register in a particular program and enroll in various courses. A lecturer from the specific department takes each course, and each lecturer teaches a various group of students

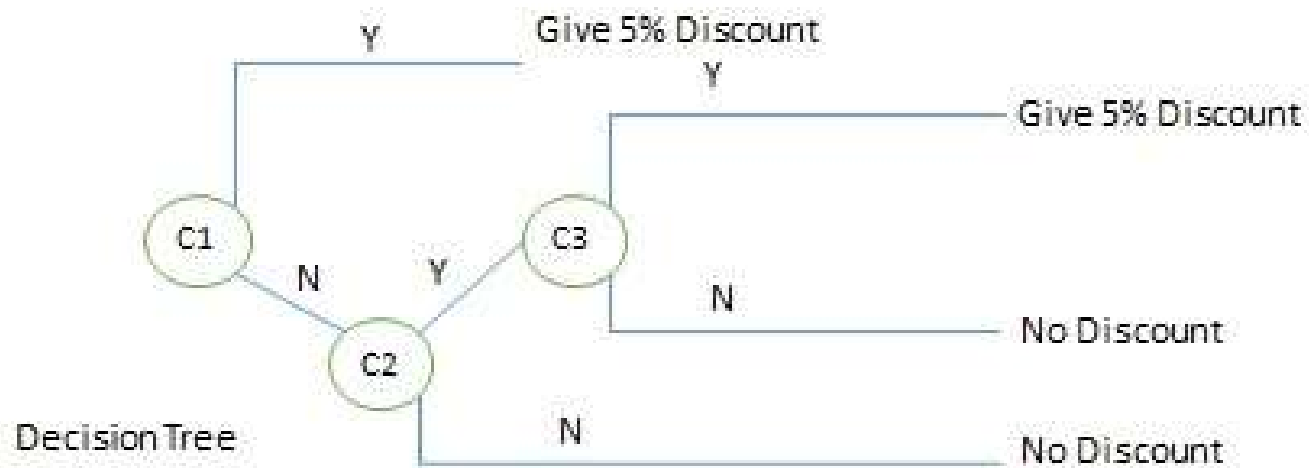
More on Decision Tree

A decision tree is a diagram that shows alternative actions and conditions within horizontal tree framework. Thus, it depicts which conditions to consider first, second, and so on.

Decision trees depict the relationship of each condition and their permissible actions. A square node indicates an action and a circle indicates a condition. It forces analysts to consider the sequence of decisions and identifies the actual decision that must be made.



Decision Tree Examples



Y = Yes

N = No

C1 : Advance Payment Mode

C2 : Purchase Amount $\geq$ 10,000

C3 : Regular Customer

More on Decision Table

- Decision tables are a method of describing the complex logical relationship in a precise manner which is easily understandable.
- It is useful in situations where the resulting actions depend on the occurrence of one or several combinations of independent conditions.
- It is a matrix containing row or columns for defining a problem and the actions.
- **Components of a Decision Table**
 - **Condition Stub** – It is in the upper left quadrant which lists all the condition to be checked.
 - **Action Stub** – It is in the lower left quadrant which outlines all the action to be carried out to meet such condition.
 - **Condition Entry** – It is in upper right quadrant which provides answers to questions asked in condition stub quadrant.
 - **Action Entry** – It is in lower right quadrant which indicates the appropriate action resulting from the answers to the conditions in the condition entry quadrant.

CONDITIONS	Rule 1	Rule 2	Rule 3	Rule 4
Advance payment made	Y	N	N	N
Purchase amount = Rs 10,000/-	-	Y	Y	N
Regular Customer	-	Y	N	-
ACTIONS				
Give 5% discount	X	X	-	-
Give no discount	-	-	X	X

More on Structured English

- Structure English is derived from structured programming language which gives more understandable and precise description of process. It is based on procedural logic that uses construction and imperative sentences designed to perform operation for action.
- It is best used when sequences and loops in a program must be considered and the problem needs sequences of actions with decisions.
- It does not have strict syntax rule. It expresses all logic in terms of sequential decision structures and iterations.

Structured English

```
if customer pays advance
  then
    Give 5% Discount
  else
    if purchase amount >=10,000
      then
        if the customer is a regular customer
          then Give 5% Discount
          else No Discount
        end if
      else No Discount
    end if
  end if
```


Pseudocode

- A pseudocode does not conform to any programming language and expresses logic in plain English.
- It may specify the physical programming logic without actual coding during and after the physical design.
- It is used in conjunction with structured programming.
- It replaces the flowcharts of a program.

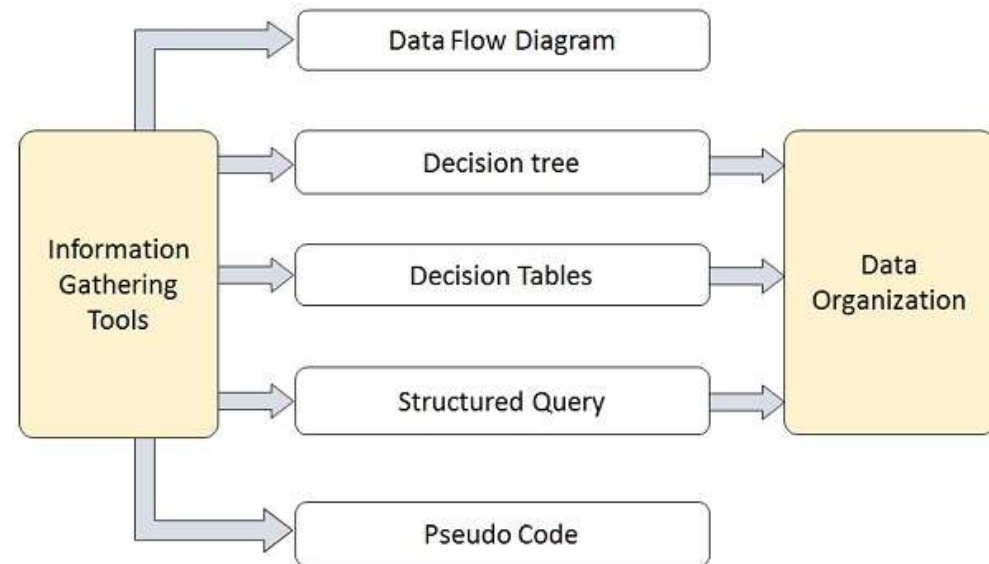
Data Dictionary

- A data dictionary is a structured repository of data elements in the system. It stores the descriptions of all DFD data elements that is, details and definitions of data flows, data stores, data stored in data stores, and the processes.
- A data dictionary improves the communication between the analyst and the user. It plays an important role in building a database. Most DBMSs have a data dictionary as a standard feature. For example, refer the following table –

Sr.No.	Data Name	Description	No. of Characters
1	ISBN	ISBN Number	10
2	TITLE	title	60
3	SUB	Book Subjects	80
4	ANAME	Author Name	15

Structured Analysis Tools

- During Structured Analysis, various tools and techniques are used for system development. They are –
 - Data Flow Diagrams
 - Data Dictionary
 - Decision Trees
 - Decision Tables
 - Structured English
 - Pseudocode



Guidelines for Selecting Appropriate Tools

- Use the following guidelines for selecting the most appropriate tool that would suit your requirements –
- Use DFD at high or low level analysis for providing good system documentations.
- Use data dictionary to simplify the structure for meeting the data requirement of the system.
- Use structured English if there are many loops and actions are complex.
- Use decision tables when there are a large number of conditions to check and logic is complex.
- Use decision trees when sequencing of conditions is important and if there are few conditions to be tested.